

TOPEET 迅为

www.topeetboard.com

「新一代 AIOT 设备平台的优秀典范」

超长供货周期 7 X 24 小时稳定运行
软硬件全开源 丰富接口和高扩展性

iTOP-RK3568

迅为电子团队编著



官方微信公众号



迅为商城

 010-8527-0708

 www.topeetboard.com

 Beijing TOPEET Electronics Co.Ltd

更新记录

更新版本	修改内容
V1.0	初版
V1.1	新增 Linux 及 Android 驱动源码位置附录
V1.2	1.2 小节修改了屏幕选择部分，区分新旧版本的 LVDS 10.1 寸 1280*800 屏幕
V1.3	新增 4.2 小节 SDK 包说明、第 6 章、第 7 章、第 9 章
V1.4	新增附录一 视频链接
V1.5	添加第五章 5.10 内核源码的编译
V1.6	添加 4.5 小节中 ubuntu 和 debian 系统版本选择章节 修改第 10 章 Ubuntu 和 Debian 文件系统的构建章节 修改优化第 10 章 Ubuntu 和 Debian 文件系统的构建章节
V1.7	修改第四章编译 Linux 源码包相关章节，加入图形化界面编译部分
V1.8	修改第五章编译 Linux 源码包相关章节，加入图形化界面编译部分
V1.9	更新第 10 章 ubuntu debian 定制相关章节

目录

更新记录	2
目录	4
注意事项与售后维修	6
技术支持与开发定制	8
资料获取与后续更新	10
第 1 章 编译 Android11 源码包	11
1.1 获取 Android11 源码	11
1.2 设置屏幕配置	12
1.3 修改编译线程数	15
1.4 整体编译 Android 固件	15
1.5 单独编译 Android 固件	18
1.5.1 编译 uboot	19
1.5.2 编译 Android 内核	21
1.5.3 编译 Android 文件系统	23
1.6 打包 update.img	27
第 2 章 编译 Android11 问题总结	28
第 3 章 编译 Android12 源码包	30
3.1 获取 Android12 源码	30
3.2 设置屏幕配置	31
3.3 修改编译线程数	33
3.4 整体编译 Android 固件	34
3.5 单独编译 Android 固件	36
3.5.1 编译 uboot	37
3.5.2 编译 Android 内核	40
3.5.3 编译 Android 文件系统	41
3.6 打包 update.img	45
第 4 章 编译 Linux 源码包	46
4.1 获取 Linux 源码包	46
4.2 SDK 包说明	47
4.2.1 SDK 包源码简要	47
4.2.2 模块代码目录相关说明	48
4.3 设置屏幕配置	48
4.4 编译 Buildroot	51
4.4.1 单独编译	51
4.4.2 全自动编译	64
4.5 编译 Ubuntu	65
4.5.1 单独编译	65
4.5.2 全自动编译	78
4.6 编译 Debian	80

4.6.1 单独编译	80
4.6.2 全自动编译	94
4.7 编译 Yocto	95
4.7.1 全自动编译	95
4.7.2 单独编译	96
4.8 编译其他系统	100
第5章 编译 Linux-5.10 内核源码包	102
5.1 获取 Linux 源码包	102
5.2 SDK 包说明	103
5.2.1 SDK 包源码简要	103
5.2.2 模块代码目录相关说明	104
5.3 设置屏幕配置	104
5.4 编译 Buildroot	105
5.4.1 单独编译	105
5.4.2 全自动编译	119
5.5 编译 Ubuntu	120
5.5.1 单独编译	120
5.5.2 全自动编译	133
5.6 编译 Debian	135
5.6.1 单独编译	135
5.6.2 全自动编译	149
5.7 编译其他系统	150
第6章 编译 Linux 源码问题总结	152
第7章 Android11/12/Linux 设备树简介	155
第8章 如何修改默认配置并保存	156
8.1 Android11/12/Linux 默认配置文件	156
8.2 Android11/12/Linux 编译驱动到 uboot	156
8.3 Android11/12/Linux 编译驱动到内核	157
第9章 编译 Linux_NVR_SDK 源码	159
9.1 编译环境说明	159
9.2 获取 Linux_NVR_SDK 源码	159
9.3 Linux_NVR_SDK 简介	160
9.4 Linux_NVR_SDK 编译源码	162
9.5 Linux_NVR_SDK 编译应用	164
第10章 定制 Ubuntu 和 Debian 系统	167
10.1 安装 docker 并配置	错误！未定义书签。
10.2 使用 docker 构建系统	错误！未定义书签。
10.3 系统定制（正常情况）	错误！未定义书签。
10.3.1 方法 1	错误！未定义书签。
10.3.2 方法 2	错误！未定义书签。
10.4 系统定制（无法联网）	错误！未定义书签。
附录一 视频链接	169

注意事项与售后维修

一 注意事项

- ◆ 请注意和遵循标注在产品上的所有警示和指引信息；
- ◆ 请勿带电插拔核心板及外围模块；
- ◆ 使用产品之前，请仔细阅读本手册，并妥善保管，以备将来参考；
- ◆ 请使用配套电源适配器，以保证电压、电流的稳定；
- ◆ 请在凉爽、干燥、清洁的地方使用本产品；
- ◆ 请勿在冷热交替环境中使用本产品，避免结露损坏元器件；
- ◆ 请勿将任何液体泼溅在本产品上，如果不慎被任何液体泼溅或浸润，请立刻断电并充分晾干；
- ◆ 请勿使用有机溶剂或腐蚀性液体清洗本产品；
- ◆ 请勿在多尘、脏乱的环境中使用本产品，如果长期不使用，请包装好本产品；
- ◆ 请勿在震动过大的环境中使用，任何跌落、敲打或剧烈晃动都可能损坏线路及元器件；
- ◆ 请勿在通电情况下，插拔核心板及外围模块(特别是串口模块)；
- ◆ 请勿自行维修、拆解本产品，如产品出现故障应及时联系本公司进行维修；
- ◆ 请勿自行修改或使用未经授权的配件，由此造成的损坏将不予保修；

二 售后维修

凡是通过迅为直接购买或经迅为授权的正规代理商处购买的迅为产品，均可享受以下权益：

- 1、开发板本身 1 年免费保修服务（配件除外）；
- 2、保修期满后出现产品异常，迅为提供有偿维修服务，可与迅为取得联系，收费视具体情况而定。如遇损坏程度严重等其他不可控因素导致无法维修的，公司不再提供维修服务；
- 3、如您购买的产品需要维修或检测，请提前备份机器内的相关数据。迅为不对因数据丢失所造成的损失负责。

注：以下情况不属于免费维修范围，可提供有偿维修：

- 1、超出保修期的产品；
- 2、非保元件：CPU、内存芯片、Flash；
- 3、由于使用不当，出现诸如 PCB 烧毁、破裂等物理损伤的产品；
- 4、由于人为疏忽或错误使用、未按说明书规定使用而造成的产品损坏等；

- 5、拆装或更换组件、器件而造成无法复原的开发板；
- 6、在将故障件返回迅为技术服务部的过程中由于包装或运输操作不当造成损坏的产品。

维修周期：收到返修产品后，我们将即日安排工程师进行检测，我们将在短时间内维修或更换并寄回。一般的故障维修周期为 5 个工作日（自我司收到物品之日起，不计运输过程时间），由于特殊故障导致无法短期内维修的产品，我们会与用户另行沟通并确认维修周期。

维修费用：在免费保修期内的产品，由于产品质量问题引起的故障，不收任何维修费用；不属于免费保修范围内的故障或损坏，在检测确认问题后，我们将与客户沟通并确认维修费用，我们仅收取元器件材料费，不收取维修服务费；超过保修期限的产品，根据实际损坏的程度来确定收取的元器件材料费和维修服务费。

运输费用：产品正常保修时，用户寄回的运费由用户承担，维修后寄回给用户的费用由我司承担。非正常保修产品来回运费均有用户承担。

联系方式：

地 址：北京市海淀区永翔北路 9 号中国航发大厦三层北京迅为电子有限公司

联系人：迅为开发板售后服务部 北京迅为电子有限公司

邮 编：100094

电 话：010-58957586

技术支持与开发定制

1、技术支持范围

- (1) 了解产品的软、硬件资源提供情况咨询；
- (2) 产品的软、硬件手册使用过程中遇到的问题；
- (3) 下载和烧写更新系统过程中遇到的问题；
- (4) 产品用户的资料丢失、更新后重新获取；
- (5) 产品的故障判断及售后维修服务。

2、技术讨论范围

由于嵌入式系统知识范围广泛，涉猎种类繁多，我们无法保证对各种问题都能一一解答，以下内容无法供技术支持，只能提供建议。

- (1) 源码如何理解和修改，电路板的自行设计制作或修改；
- (2) 如何编译和移植操作系统；
- (3) 用户在自行修改以及开发中遇到的软硬件问题。

3、技术支持方式

论坛：<http://bbs.topeetboard.com/forum.php>

电话：0312-6796610

邮箱：support@topeetboard.com

4、技术支持服务时间

上午 9:00--12:00, 下午 13:30--17:30 (周一至周六)

QQ 群主动技术支持：

上午 10:00--11:00, 下午 15:00--16:00 (周一至周六)

5、定制开发服务

本公司提供嵌入式操作系统底层驱动、硬件板卡的有偿定制开发服务，以缩短您的产品开发周期。请将需求：

发送邮件到：support@topeetboard.com

联系电话: 0312-6796610

淘宝店铺 1:

<https://arm-board.taobao.com/?spm=a1z10.1-c-s.0.0.7bf93dd3q2C808>

淘宝店铺 2:

https://shop459378556.taobao.com/shop/view_shop.htm?spm=a230r.1.14.48.25b84a81oPy9v1&user_number_id=2207480684984

资料获取与后续更新

一. 资料的获取

(1) 百度网盘下载

网盘的链接在购买开发板后可以在迅为电子技术支持 QQ 群下载。如果链接有更新，会在群里贴通告

二 后续更新

后续文档、视频等资料的更新，为了确保您的资料是最新状态，请密切关注我们的动态，我们将会通过微信公众号和 QQ 群推送。关注“迅为电子”微信公众号，不定期分享教程、资料 and 行业干货及产品一线资料。



迅为电子
让学习更容易，让开发更简单

第 1 章 编译 Android11 源码包

编译环境说明：

本手册使用的是迅为提供的编译环境 ubuntu18.04，在网盘资料“**iTOP-3568 开发板\02_【iTOP-RK3568 开发板】开发资料\02_iTOP-3568 开发板搭建好编译环境的 Ubuntu18.04 虚拟机镜像**”路径下。这里的 ubuntu 默认开发环境已经搭建好。

因 Ubuntu 版本众多，无法将所有版本 Ubuntu 全部测试通过，如使用其他开发环境，在开发过程中遇到问题，需自行研究解决。

在编译源码之前，首先要阅读并且操作《**【北京迅为】itop-3568 开发板快速使用编译环境 Ubuntu18.04 .doc**》手册，才可进行编译工作。

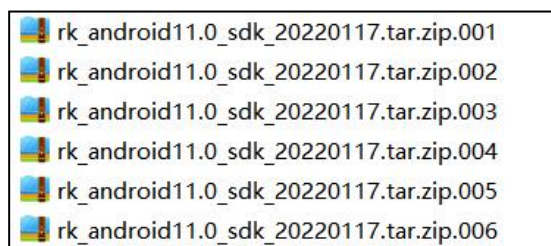
1.1 获取 Android11 源码

Android11 源码网盘下载路径：“**iTOP-3568 开发板\01_【iTOP-RK3568 开发板】基础资料【V1.7 版本】\03_iTOP-RK3568 开发板 Android11 源码**”

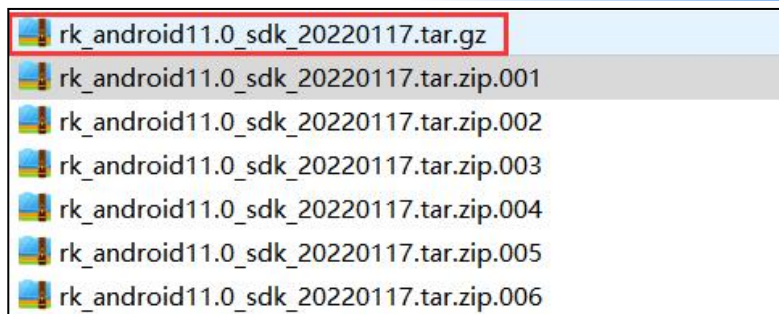
源码是分卷压缩包，需要全部下载下来放在同一级文件夹。所以要从网盘将 Android11 源码包 rk_android11.0_sdk_20220117.tar.zip.001 至 rk_android11.0_sdk_20220117.tar.zip.00X（X 代表 1, 2, 3.....）等分卷压缩包全部下载到 windows 电脑上，并放在同一级目录下。

源码名称说明：源码会逐渐更新，发布的源码日期会有所不同，具体以网盘中实际名称为准。

下载的源码，如下图所示：



右键点击第一个分卷压缩包 rk_android11.0_sdk_20220117.tar.zip.001，选择“解压到 rk_android11.0_sdk_20220117XXX”，（XXX 为省略），解压时间会比较长，耐心等待解压完以后会生成一个 rk_android11.0_sdk_20220117.tar.gz 名字的压缩包，如下图所示：



使用 SSH 软件拷贝 rk_android11.0_sdk_20220117.tar.gz 文件到 Ubuntu 系统，如下图所示：

```
topeet@ubuntu:~/Android11$  
topeet@ubuntu:~/Android11$ ls  
lost+found rk_android11.0_sdk_20220117.tar.gz  
topeet@ubuntu:~/Android11$  
topeet@ubuntu:~/Android11$
```

输入以下命令解压压缩包

```
sudo tar -vxf rk_android11.0_sdk_20220117.tar.gz
```

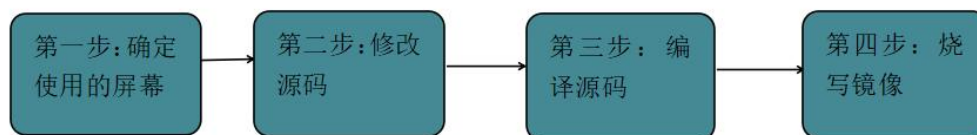
```
topeet@ubuntu:~/Android11$  
topeet@ubuntu:~/Android11$ sudo tar -vxf rk_android11.0_sdk_20220117.tar.gz
```

解压完毕生成 rk_android11.0_sdk 文件夹，如下图所示：

```
rk_android11.0_sdk/tools/doc_generation/.git/refs  
rk_android11.0_sdk/tools/doc_generation/.git/svn  
rk_android11.0_sdk/build.sh  
topeet@ubuntu:~/Android11$ ls rk_android11.0_sdk  
Android.bp  build  developers  hardware  makeall.sh  packages  rkbin  sdk  u-boot  
art  build.sh  development  javaenv.sh  Makefile  pdk  RKDocs  system  vendor  
bionic  compatibility  device  kernel  mkcombinedroot  platform_testing  rkst  test  
bootable  cts  external  libcore  mkimage_ab.sh  prebuilt  RKTools  toolchain  
bootstrap.bash  dalvik  frameworks  libnativehelper  mkimage.sh  restore_patches.sh  rockdev  tools  
topeet@ubuntu:~/Android11$  
topeet@ubuntu:~/Android11$
```

1.2 设置屏幕配置

在源码编译之前首先要确定自己想要使用的屏幕并修改源码，在编译镜像，烧写镜像。如下图所示：



第一步：确定要使用的屏幕种类，屏幕种类选择如下所示：

iTOP-3568 开发板支持以下种类屏幕：

- 迅为 LVDS 7 寸屏幕
- 迅为 LVDS 10.1 寸 1024*600 屏幕
- 迅为 LVDS 10.1 寸 1280*800 屏幕

- 迅为 MIPI 7 寸屏幕
- 标准 HDMI 屏幕（通过 HDMI 线连接）
- 标准 HDMI 屏幕（通过 VGA 线连接）

第二步：修改源码的设备树

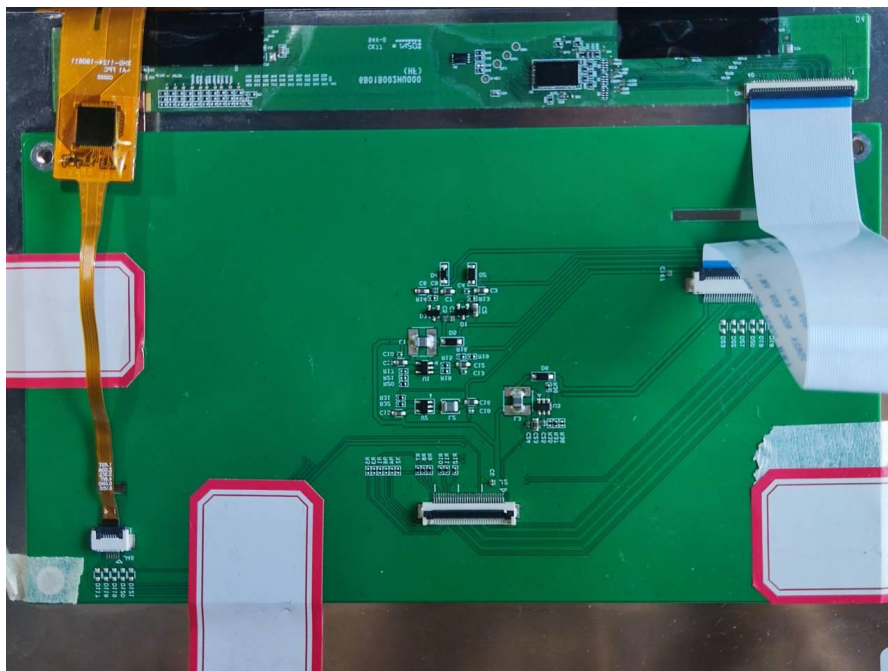
修改 rk_android11.0_sdk/kernel/arch/arm64/boot/dts/rockchip 目录下的 topeet_screen_choose.dtsi 文件。打开如下图所示：

```
/******单屏显示*****/
#define LCD_TYPE_MIPI //in vp 1
//#define LCD_TYPE_LVDS_10_1_1024X600 //in vp 2
//#define LCD_TYPE_LVDS_10_1_1280X800_gt911 //in vp 2
//#define LCD_TYPE_LVDS_10_1_1280X800_gt9271 //in vp 2
//#define LCD_TYPE_LVDS_7_0 //in vp 2
//#define LCD_TYPE_EDP_VGA //in vp 0
//#define LCD_TYPE_HDMI_VP0 //hdmi in vp 0
//#define LCD_TYPE_HDMI_VP1 //hdmi in vp 1
```

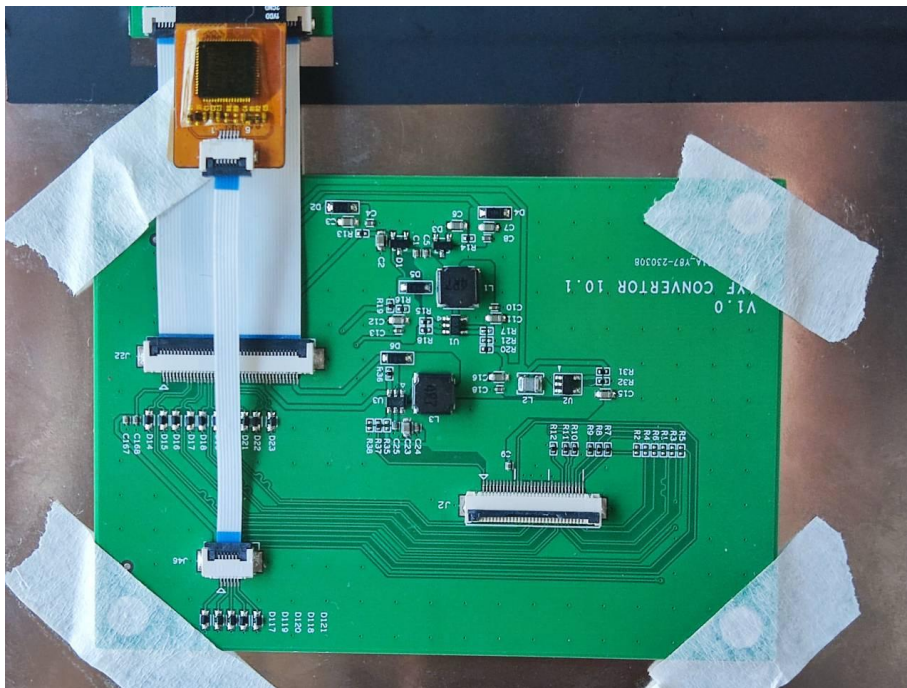
如上图所示，迅为的 Android11 源码支持 MIPI7 寸屏幕，LVDS 10.1 寸 1024*600 屏幕，LVDS 10.1 寸 1280*800 屏幕，LVDS 7 寸屏幕，VGA 接口连接显示器，HDMI 接口连接显示。

注：LVDS 10.1 寸 1280*800 屏幕分为#define LCD_TYPE_LVDS_10_1_1280X800_gt911 和#define LCD_TYPE_LVDS_10_1_1280X800_gt9271 两种，可根据屏幕背板区分，如下图所示：

#define LCD_TYPE_LVDS_10_1_1280X800_gt911 对应屏幕如下图所示：



#define LCD_TYPE_LVDS_10_1_1280X800_gt9271 对应屏幕如下图所示：



若选择 define LCD_TYPE_LVDS_10_1_1280X800_gt9271 宏定义则需修改默认配置文件 kernel/arch/arm64/configs/rockchip_defconfig，修改完成如下图所示：

```
2246 # CONFIG_TOUCHSCREEN_GSL3673_800X1280 is not set
2247 # CONFIG_TOUCHSCREEN_GSL3676 is not set
2248 # CONFIG_TOUCHSCREEN_GT9XX=y
2249 CONFIG_TOUCHSCREEN_GT9271=y
2250 # CONFIG_TOUCHSCREEN_HIDEAP is not set
2251 # CONFIG_TOUCHSCREEN_HYN_CST2XX is not set
2252 # CONFIG_TOUCHSCREEN_ILI210X is not set
2253 # CONFIG_TOUCHSCREEN_S6SY761 is not set
2254 # CONFIG_TOUCHSCREEN_GUNZE is not set
2255 # CONFIG_TOUCHSCREEN_EKTF2127 is not set
```

若选择 define LCD_TYPE_LVDS_10_1_1280X800_gt911 宏定义则需修改默认配置文件 kernel/arch/arm64/configs/rockchip_defconfig，修改完成如下图所示：

```
2247 # CONFIG_TOUCHSCREEN_GSL3676 is not set
2248 CONFIG_TOUCHSCREEN_GT9XX=y
2249 #CONFIG_TOUCHSCREEN_GT9271=y|
2250 # CONFIG_TOUCHSCREEN_HIDEAP is not set
2251 # CONFIG_TOUCHSCREEN_HYN_CST2XX is not set
2252 # CONFIG_TOUCHSCREEN_ILI210X is not set
2253 # CONFIG_TOUCHSCREEN_S6SY761 is not set
2254 # CONFIG_TOUCHSCREEN_GUNZE is not set
2255 # CONFIG_TOUCHSCREEN_EKTF2127 is not set
2256 CONFIG_TOUCHSCREEN_ELAN=y
2257 # CONFIG_TOUCHSCREEN_ELO is not set
```

若没有使用屏幕，使能 LCD_TYPE_HDMI_VP0 宏定义即可。

我们如果想要连接某个屏幕，使能对应的屏幕宏定义即可。如果想要设置多屏显示，请参考《【北京迅为】itop-3568 开发板多屏显示手册 V1.0.doc》。

1.3 修改编译线程数

在编译 Android 系统时，可以通过调整线程数来控制并行编译的任务数量，使用多线程可以加快构建速度。**要注意根据 Ubuntu 的处理器内核总数以及内存大小来设置合适的线程数。**过高的线程数可能导致系统资源的竞争，过低的线程数则不能充分利用系统的并行处理能力。找到合适的线程数可以加快编译速度，提高效率。

可以在 build.sh 编译脚本中修改线程数，如下图所示：



```
source build/envsetup.sh >/dev/null
BUILD_UBOOT=false
BUILD_KERNEL_WITH_CLANG=false
BUILD_KERNEL=false
BUILD_ANDROID=false
BUILD_AB_IMAGE=false
BUILD_UPDATE_IMG=false
BUILD_OTA=false
BUILD_PACKING=false
BUILD_VARIANT='get_build_var TARGET_BUILD_VARIANT'
KERNEL_DTS=""
BUILD_VERSION=""
BUILD_JOBS=16
```

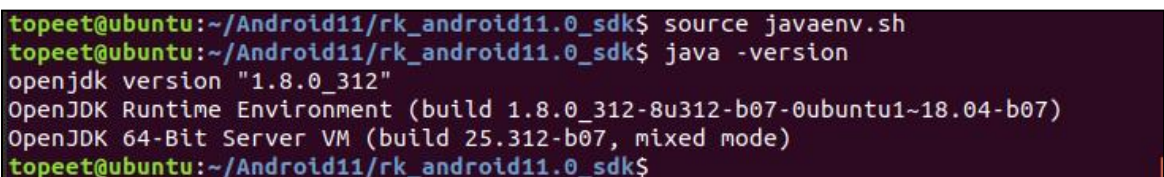
1.4 整体编译 Android 固件

注意：编译系统时一定要使用普通权限编译。

进入到 rk_android11.0_sdk 文件夹，输入以下命令设置 java 版本为 1.8 版本，确认 java 版本是 1.8 版本之后，才可以进行下一步编译，如下图所示：

```
source javaenv.sh
```

```
java -version
```



```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ source javaenv.sh
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ java -version
openjdk version "1.8.0_312"
OpenJDK Runtime Environment (build 1.8.0_312-8u312-b07-0ubuntu1~18.04-b07)
OpenJDK 64-Bit Server VM (build 25.312-b07, mixed mode)
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
```

输入以下命令使能编译环境：

```
source build/envsetup.sh
```

```
lunch rk3568_r-userdebug
```



```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ source build/envsetup.sh
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ lunch rk3568_r-userdebug

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=11
TARGET_PRODUCT=rk3568_r
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-94-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=RQ3A.210705.001
OUT_DIR=out
=====
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
```

最后输入以下命令开始整体编译

```
./build.sh -AUCKu
```

```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ ./build.sh -AUCKu
```

编译完成如下图所示：

```
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x2ba,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x6f1c0,userspace=0xdf
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x70800,size=0x282,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x71000,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x471000,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47d000,size=0x1ef9000,userspace=0x3df2
Add file: ./Image/dtbo.img
dtbo,Add file: ./Image/dtbo.img done,offset=0x2376000,size=0x26f,userspace=0x1
Add file: ./Image/vbmeta.img
vbmeta,Add file: ./Image/vbmeta.img done,offset=0x2376800,size=0x1000,userspace=0x2
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x2377800,size=0x46f6800,userspace=0x8ded
Add file: ./Image/baseparameter.img
baseparameter,Add file: ./Image/baseparameter.img done,offset=0x6a6e000,size=0x100000,userspace=0x200
Add file: ./Image/super.img
super,Add file: ./Image/super.img done,offset=0x6b6e000,size=0x5b2e1dc8,userspace=0xb65c4
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making update.img OK.
Make update image ok!
/home/topeet/Android11/rk_android11.0_sdk
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
```

编译生成的镜像统一放在 rockdev/Image-rk3568_r 目录下，

生成的镜像包括 MiniLoaderAll.bin、uboot.img、trust.img、resource.img、boot.img、misc.img、recovery.img、boot-debug.img、dtbo.img、pcba_small_misc.img、pcba_whole_misc.img、super.img、vbmeta.img 等 13 个镜像，但是在烧写系统时，这 13 个镜像并不需要全部使用。

```
topeet@ubuntu:~/Android11/rk_android11.0_sdk/rockdev/Image-rk3568_r$ ls
baseparameter.img boot.img dtbo.img misc.img pcba_small_misc.img recovery.img super.img update.img
boot-debug.img config.cfg MiniLoaderAll.bin parameter.txt pcba_whole_misc.img resource.img uboot.img vbmeta.img
topeet@ubuntu:~/Android11/rk_android11.0_sdk/rockdev/Image-rk3568_r$
topeet@ubuntu:~/Android11/rk_android11.0_sdk/rockdev/Image-rk3568_r$
topeet@ubuntu:~/Android11/rk_android11.0_sdk/rockdev/Image-rk3568_r$
```

镜像作用如下表所示：

镜像	含义
boot-debug.img	与 boot.img 的差别是 user 固件可以烧写这个 boot.img 进行 root 权限操作
config.cfg	烧写工具的配置文件，可以直接导入烧写工具显示需要烧写的选项。
MiniLoaderAll.bin	包含一级 loader。
misc.img	包含 recovery-wipe 开机标识信息，烧写后会进行 recovery。
parameter.txt	包含分区信息。
pcba_small_misc.img	包含 pcba 开机标识信息，烧写后会进入简易版 pcba 模式。

pcba_whole_misc.img	包含 pcba 开机标识信息，烧写后会进入完整版 pcba 模式
recovery.img	包含 recovery-ramdis、kernel、dtb
super.img	包含 odm、product、vendor、system、system_ext 分区内容。
trust.img	包含 BL31、BL32 RK3566/RK3568 没有生成这个固件，不需要烧写
uboot.img	包含 uboot 固件
vbmata.img	包含 avb 校验信息，用于 AVB 校验
update.img	包含以上需要烧写的 img 文件，可以用于工具直接烧写整个固件包。

1.5 单独编译 Android 固件

进入到 rk_android11.0_sdk 文件夹，输入以下命令设置 java 版本为 1.8 版本，如下图所示：

```
source javaenv.sh
```

```
java -version
```

```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ source javaenv.sh
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ java -version
openjdk version "1.8.0_312"
OpenJDK Runtime Environment (build 1.8.0_312-8u312-b07-0ubuntu1~18.04-b07)
OpenJDK 64-Bit Server VM (build 25.312-b07, mixed mode)
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
```

输入以下命令使能编译环境：

```
source build/envsetup.sh
```

```
lunch rk3568_r-userdebug
```

```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ source build/envsetup.sh
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ lunch rk3568_r-userdebug

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=11
TARGET_PRODUCT=rk3568_r
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-94-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=RQ3A.210705.001
OUT_DIR=out
=====
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
```

执行完上述命令，如果需要编译 uboot，则参考编译 uboot 小节，如果需要编译 Android 内核，则参考编译 Android 内核小节，如果编译 Android 文件系统，则参考编译 Android 文件系统小节。
注意！要用同一个窗口普通用户进行编译。

1.5.1 编译 uboot

方法一：

在 Android 源码目录下执行如下命令编译 uboot:

```
./build.sh -U
```



```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ ./build.sh -U
will build u-boot
-----KERNEL_DTS:rk3568-evb1-ddr4-v10

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=11
TARGET_PRODUCT=rk3568_r
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-94-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=RQ3A.210705.001
OUT_DIR=out
=====
start build uboot

#### build completed successfully ####

#### build completed successfully (1 seconds) ####

#### build completed successfully ####
```

编译完 uboot 如下图所示:

```
./resource.img with battery images is ready
/home/topeet/Android11/rk_android11.0_sdk
make and copy android images
TARGET_PRODUCT=rk3568_r
TARGET_BASE_PARAMETER_IMAGE=device/rockchip/common/baseparameter/v2.0/baseparameter.img
HIGH_RELIABLE_RECOVERY_OTA=
BOARD_AVB_ENABLE=false
system filesystem is ext4
create dtbo.img...
done.
create resource.img...
done.
create boot.img...
done.
create boot-debug.img...
done.
skip copy images: vendor_boot.img
skip copy images: vendor_boot-debug.img
create recovery.img...
done.
create super.img...
done.
create vbmeta.img...
BOARD_AVB_ENABLE is false, use default vbmeta.img
create misc.img... done.
create uboot.img...
u-boot/trust.img not found! Please make it from u-boot first!
create loader...
create config.cfg...
create baseparameter...done.
Make image ok!
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
```

编译完成后在 u-boot 目录会生成 rk356x_spl_loader_v1.10.111.bin 文件、uboot.img 文件、resource.img 三个镜像文件。


```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ cd u-boot/
topeet@ubuntu:~/Android11/rk_android11.0_sdk/u-boot$ ls
apl          board      drivers   Kconfig    PREUPLOAD.cfg  tee.bin      u-boot.dtb
arch         cmd        dts       lib         README         test         u-boot-dtb.bin
bl31_0x00040000.bin common    env       licenses   resource.img   tools        uboot.img
bl31_0x00060000.bin config.mk examples MAINTAINERS rk356x_spl_loader_v1.10.111.bin tpl          u-boot.lds
bl31_0x00068000.bin configs  fit       Makefile    scripts        u-boot      u-boot.map
bl31_0x0fdcc9000.bin disk     fs        make.sh     snapshot.commit u-boot.bin   u-boot-nodtb.bin
bl31_0x0fdcd0000.bin doc      include  net         spl            u-boot.cfg  u-boot.srec
bl31.elf     Documentation Kbuild    post        System.map     u-boot.cfg.configs u-boot.syn
```

同时编译成功后会将 `rk356x_spl_loader_v1.10.111.bin`、`uboot.img` 拷贝到 `rockdev/Image-rk3568_r` 目录下，`rk356x_spl_loader_v1.10.111` 会改名为 `MiniLoaderAll.bin`。
`MiniLoaderAll.bin` 也就是一级 loader

只需要将 `MiniLoaderAll.bin` 和 `uboot.img` 烧写到开发板即可更新 uboot。

方法二：

方法一使用的是整体编译脚本,也可以使用 u-boot 下的编译脚本。

如果只是调试 uboot 或者 rkbin，可以在 u-boot 目录下直接编译，这样编译速度更快，如下图所示，输入以下命令：

```
./make.sh rk3568
```

```
topeet@ubuntu:~/Android/rk_android11.0_sdk/u-boot$ ./make.sh rk3568
## make rk3568_defconfig -j8
#
# configuration written to .config
#
/home/topeet/Android/rk_android11.0_sdk/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-gcc
scripts/kconfig/conf --silentoldconfig Kconfig
CHK include/config.h
CFG u-boot.cfg
GEN include/autoconf.mk.dep
CFG spl/u-boot.cfg
CFG tpl/u-boot.cfg
GEN spl/include/autoconf.mk
GEN include/autoconf.mk
GEN tpl/include/autoconf.mk
CHK include/config/uboot.release
CHK include/generated/timestamp_autogenerated.h
UPD include/generated/timestamp_autogenerated.h
CHK include/generated/version_autogenerated.h
CHK include/generated/generic-asm-offsets.h
CHK include/generated/asm-offsets.h
HOSTCC tools/mkenvimage.o
HOSTCC tools/fit_image.o
HOSTCC tools/image-host.o
```

这样编译后会在 uboot 目录下生成 `uboot.img` 和 `rk356x_spl_loader_v1.10.111.bin`，但是不会拷贝到 `rockdev/Image-rk3568_r` 下面，所以需要烧写 uboot 目录下的 `uboot.img` 和 `rk356x_spl_loader_v1.10.111.bin`

烧写时将 `rk356x_spl_loader_v1.10.111.bin` 重命名为 `MiniLoaderAll.bin`，再烧写。

1.5.2 编译 Android 内核

方法一：

在 Android 源码目录下执行如下命令编译 Android 内核。

```
./build.sh -CKA
```

```
topeet@ubuntu:~/Android/rk_android11.0_sdk$ ./build.sh -CKA
will build kernel with Clang
will build kernel
will build android
-----KERNEL_DTS:rk3568-evb1-ddr4-v10

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=11
TARGET_PRODUCT=rk3568_r
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-109-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=RQ3A.210705.001
OUT_DIR=out
```

编译后会在 rockdev/Image-rk3588_r 目录下生成 boot.img，boot.img 为内核镜像。boot.img 镜像里面包含了设备树镜像。所以烧写 boot.img 即可更新内核镜像。

解释如下：

参数-C 表示用 clang 编译器编译，如果不带 C 则使用 gcc 编译，如果需要过 GMS 认证的 google 有要求要用 clang 编译。

因为 kernel 编译完后需要通过 Android 去打包成 boot.img，所以这里需要加上 A 参数，即编译 kernel 的时需要一起编译 Android 才能打包 boot.img。通过以上介绍可以知道单独编译 kernel 需要同时编译 Android，导致编译很耗时，针对这个问题，我们推荐使用方法二来单独编译内核。

方法二：

此方法常用于 kernel 的开发和调试，以下的方法既编译 kernel 部分时，同时打包成 boot.img，这样加快了我们开发的速度；

进入内核目录下，输入以下命令：

```
cd kernel
make ARCH=arm64 \
CC=./prebuilts/clang/host/linux-x86/clang-r383902b/bin/clang \
LD=./prebuilts/clang/host/linux-x86/clang-r383902b/bin/ld.lld rockchip_defconfig
android-11.config \
&& make ARCH=arm64 \
CC=./prebuilts/clang/host/linux-x86/clang-r383902b/bin/clang \
```

```
LD=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/ld.lld \  
BOOT_IMG=../rockdev/Image-rk3568_r/boot.img rk3568-evb1-ddr4-v10.img
```

```
topeet@ubuntu:~/Android/rk_android11.0_sdk$ cd kernel/  
topeet@ubuntu:~/Android/rk_android11.0_sdk/kernel$ make ARCH=arm64 CC=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/clang LD=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/ld.lld rockchip_defconfig android-11.config && make ARCH=arm64 CC=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/clang LD=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/ld.lld BOOT_IMG=../rockdev/Image-rk3568_r/boot.img rk3568-evb1-ddr4-v10.img
```

为方便使用，可以将上述命令写成脚本，在 kernel 目录下创建 makekernel.sh,在调试的过程中直接在 kernel 目录下执行该脚本，makekernel.sh 内容为：

```
#!/bin/sh  
make ARCH=arm64 CC=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/clang \  
LD=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/ld.lld rockchip_defconfig \  
android-11.config && make ARCH=arm64 \  
CC=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/clang \  
LD=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/ld.lld \  
BOOT_IMG=../rockdev/Image-rk3568_r/boot.img rk3568-evb1-ddr4-v10.img
```

我们来解释以上命令是什么意思！

BOOT_IMG 是指整编 Android 源码之后在 rockdev 目录下生成的 boot.img，所以在单独编译 Android 内核之前，一定要整编 Android 源码。整编 Android 源码生成的 rockdev 目录下的 boot.img 镜像里面不仅仅只有 kernel 和 resource，还有其他文件。当我们修改了内核源码并单独编译 Android 内核时，kernel 和 resource 发生了改变，这时候需要指定一个 boot.img，然后将新编译的 kernel 和 resource 覆盖进去。打包完成之后，在 kernel 目录下会有新 boot.img 镜像生成，就可以把这个 boot.img 镜像单独烧写到开发板中进行调试了

如果不指定 BOOT_IMG，会在下载镜像之后，系统跑进 recovery 模式，而不是进入正常的启动流程

rk3568-evb1-ddr4-v10.img 的意思是指定所使用的设备树 DTS；

打包完后，在 kernel 目录有 boot.img 镜像生成，就可以把这个 boot.img 镜像单独烧入到机器中进行调试了。

1.5.3 编译 Android 文件系统

方法一：

在源码根目录的 build.sh 脚本，找到 make installclean 命令，将 make installclean 注释掉。


```
176     cp $OUT/$INTERNAL_OTA_PACKAGE TARGET $IMAGE_PATH/  
177     cp $OUT/$INTERNAL_OTA_PACKAGE_OBJ_TARGET $IMAGE_PATH/  
178     else # regular build without OTA  
179         echo "start build android"  
180         #make installclean  
181         make -j$BUILD_JOBS  
182         # check the result of make  
183         if [ $? -eq 0 ]; then  
184             echo "Build android ok!"  
185         else  
186             echo "Build android failed!"  
187             exit 1  
188         fi
```

接着在 Android 源码目录下执行如下命令编译 Android 文件系统：

./build.sh -A

```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ ./build.sh -A  
will build android  
-----KERNEL_DTS:rk3568-evb1-ddr4-v10  
  
===== PLATFORM_VERSION_CODENAME=REL  
PLATFORM_VERSION=11  
TARGET_PRODUCT=rk3568_r  
TARGET_BUILD_VARIANT=userdebug  
TARGET_BUILD_TYPE=release  
TARGET_ARCH=arm64  
TARGET_ARCH_VARIANT=armv8-a  
TARGET_CPU_VARIANT=cortex-a55  
TARGET_2ND_ARCH=arm  
TARGET_2ND_ARCH_VARIANT=armv8-2a  
TARGET_2ND_CPU_VARIANT=cortex-a55  
HOST_ARCH=x86_64  
HOST_2ND_ARCH=x86  
HOST_OS=linux  
HOST_OS_EXTRA=Linux-5.4.0-94-generic-x86_64-Ubuntu-18.04.6-LTS  
HOST_CROSS_OS=windows  
HOST_CROSS_ARCH=x86  
HOST_CROSS_2ND_ARCH=x86_64  
HOST_BUILD_TYPE=release  
BUILD_ID=RQ3A.210705.001  
OUT_DIR=out
```

Android 编译过程如下图所示：

```
[100% 6979/6979] Target super fs image for debug: out/target/product/rk3568_r/super.img  
2022-01-19 22:15:15 - build_super_image.py - INFO : Building super image from info dict...  
2022-01-19 22:15:15 - sparse_img.py - INFO : Total of 239329 4096-byte output blocks in 18 input chunks.  
2022-01-19 22:15:15 - sparse_img.py - INFO : Total of 30169 4096-byte output blocks in 3 input chunks.  
2022-01-19 22:15:15 - sparse_img.py - INFO : Total of 55304 4096-byte output blocks in 5 input chunks.  
2022-01-19 22:15:15 - sparse_img.py - INFO : Total of 52087 4096-byte output blocks in 5 input chunks.  
2022-01-19 22:15:15 - sparse_img.py - INFO : Total of 153 4096-byte output blocks in 2 input chunks.  
2022-01-19 22:15:15 - common.py - INFO : Running: "lpmake --metadata-size 65536 --super-name super --metadata-slots 2 --device s  
uper:3263168512 --group rockchip_dynamic_partitions:3258974208 --partition system:readonly:980291584:rockchip_dynamic_partitions --im  
age system=out/target/product/rk3568_r/system.img --partition system_ext:readonly:123572224:rockchip_dynamic_partitions --image syste  
m_ext=out/target/product/rk3568_r/system_ext.img --partition vendor:readonly:226525184:rockchip_dynamic_partitions --image vendor=out  
/target/product/rk3568_r/vendor.img --partition product:readonly:213348352:rockchip_dynamic_partitions --image product=out/target/pro  
duct/rk3568_r/product.img --partition odm:readonly:626688:rockchip_dynamic_partitions --image odm=out/target/product/rk3568_r/odm.img  
--sparse --output out/target/product/rk3568_r/super.img"  
2022-01-19 22:15:49 - common.py - INFO : lpmake I 01-19 22:15:15 29041 29041 builder.cpp:1031] [liblp]Partition system will resize  
from 0 bytes to 980291584 bytes  
lpmake I 01-19 22:15:15 29041 29041 builder.cpp:1031] [liblp]Partition system_ext will resize from 0 bytes to 123572224 bytes  
lpmake I 01-19 22:15:15 29041 29041 builder.cpp:1031] [liblp]Partition vendor will resize from 0 bytes to 226525184 bytes  
lpmake I 01-19 22:15:15 29041 29041 builder.cpp:1031] [liblp]Partition product will resize from 0 bytes to 213348352 bytes  
lpmake I 01-19 22:15:15 29041 29041 builder.cpp:1031] [liblp]Partition odm will resize from 0 bytes to 626688 bytes  
2022-01-19 22:15:49 - build_super_image.py - INFO : Done writing image out/target/product/rk3568_r/super.img  
  
#### build completed successfully (10:07 (mm:ss)) ####  
  
Build android ok!  
make and copy android images
```

Android 系统编译完如下图所示:

```
#### build completed successfully (10:07 (mm:ss)) ####

Build android ok!
make and copy android images
TARGET_PRODUCT=rk3568_r
TARGET_BASE_PARAMETER_IMAGE==device/rockchip/common/baseparameter/v2.0/baseparameter.img
HIGH_RELIABLE_RECOVERY_OTA=
BOARD_AVB_ENABLE=false
system filesystem is ext4
create dtbo.img...
done.
create resource.img...
done.
create boot.img...
done.
create boot-debug.img...
done.
skip copy images: vendor_boot.img
skip copy images: vendor_boot-debug.img
create recovery.img...
done.
create super.img...
done.
create vbmeta.img...
BOARD_AVB_ENABLE is false, use default vbmeta.img
create misc.img... done.
create uboot.img...
u-boot/trust.img not found! Please make it from u-boot first!
create loader...
create config.cfg...
create baseparameter...done.
Make image ok!
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
```

编译完成后镜像文件会在 rockdev/Image-rk3568_r 目录下生成 super.img,单独烧写 super.img 即可。

```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ cd rockdev/Image-rk3568_r/
topeet@ubuntu:~/Android11/rk_android11.0_sdk/rockdev/Image-rk3568_r$ ls
baseparameter.img  boot.img  dtbo.img  misc.img  pcba_small_misc.img  recovery.img  super.img  vbmeta.img
boot-debug.img  config.cfg  MiniLoaderAll.bin  parameter.txt  pcba_whole_misc.img  resource.img  uboot.img
topeet@ubuntu:~/Android11/rk_android11.0_sdk/rockdev/Image-rk3568_r$
```

各种镜像解释如下:

boot.img: 包含 kernel.img 和 resource.img 镜像文件

boot-debug.img: 与 boot.img 的差别是 user 固件可以烧写这个 boot.img 进行 root 权限操作

config.cfg: 烧写工具的配置文件,可以直接导入烧写工具显示需要烧写的选项。

MiniLoaderAll.bin: 包含一级 loader。

misc.img: 包含 recovery-wipe 开机标识信息,烧写后会进行 recovery。

parameter.txt: 包含分区信息。

pcba_small_misc.img : 包含 pcba 开机标识信息,烧写后会进入简易版 pcba 模式。

pcba_whole_misc.img: 包含 pcba 开机标识信息,烧写后会进入完整版 pcba 模式

recovery.img: 包含 recovery-ramdis、kernel、dtb

super.img: 包含 odm、product、vendor、system、system_ext 分区内容。

trust.img: 包含 BL31、BL32 RK3566/RK3568 没有生成这个固件,不需要烧写

uboot.img: 包含 uboot 固件

vbmata.img: 包含 avb 校验信息, 用于 AVB 校验

update.img: 包含以上需要烧写的 img 文件, 可以用于工具直接烧写整个固件包。

方法二:

在 Android 源码编译的时候, 如果编译整个工程可能需要两三个小时, 即使二次编译也要二十分钟, 如果只想单独编译某个模块, 则可以使用 mm 命令, 编译出来的 so 库或者其他文件, 使用 adb 工具 push 到机器中进行替换, 重启系统验证, 不必重新烧写固件。

在使用 m/mm/mmm 命令之前, 首先要确保执行以下命令, 以下命令的作用是将 mm 等这些命令添加到环境变量当中去, 没执行的话, 会找不到命令。

```
source javaenv.sh
source build/envsetup.sh
lunch rk3568_r-userdebug
```

我们来了解下命令的含义:

m: 编译所有的模块;

mm: 编译当前目录下的模块, 当前目录下要有 Android.mk 文件;

mmm: 编译指定路径下的模块, 指定路径下要有 Android.mk 文件;

比如说, 修改了 HAL 层的音频文件: hardware/rockchip/audio/tinyalsa_hal/audio_hw.c, 就可以使用 mm 命令来进行编译, 首先要进入到 hardware/rockchip/audio/tinyalsa_hal/目录下, 注意要有 Android.mk 文件哦, 然后直接输入 mm 编译即可, 如下图所示:

```
topeet@ubuntu:~/Android/rk_android11.0_sdk$ cd hardware/rockchip/audio/tinyalsa_hal/
topeet@ubuntu:~/Android/rk_android11.0_sdk/hardware/rockchip/audio/tinyalsa_hal$ mm
22:19:36 *****
22:19:36 You are building on a machine with 15.6GB of RAM
22:19:36
22:19:36 The minimum required amount of free memory is around 16GB,
22:19:36 and even with that, some configurations may not work.
22:19:36
22:19:36 If you run into segfaults or other errors, try reducing your
22:19:36 -j value.
22:19:36 *****
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=11
TARGET_PRODUCT=rk3568_r
```

编译完成如下图所示:

```
22:20:33 Removed directory that is no longer installed: out/target/product/rk3568_r/recovery/root/pcba/lfb/modules
22:20:33 Removed directory that is no longer installed: out/target/product/rk3568_r/recovery/root/pcba/lfb
[ 98% 280/285] target C: antx <= hardware/rockchip/audio/tinyalsa_hal/antx.c
hardware/rockchip/audio/tinyalsa_hal/antx.c:66:44: warning: 'strncpy' call operates on objects of type 'char' while the size is based on a different type 'char *' [-Wsizeof-pointer-memaccess]
    if ((strncpy(argv[i], "-c", sizeof(argv[i])) == 0) ||
        ^~~~~~
hardware/rockchip/audio/tinyalsa_hal/antx.c:66:44: note: did you mean to provide an explicit length?
    if ((strncpy(argv[i], "-c", sizeof(argv[i])) == 0) ||
        ^~~~~~
hardware/rockchip/audio/tinyalsa_hal/antx.c:67:47: warning: 'strncpy' call operates on objects of type 'char' while the size is based on a different type 'char *' [-Wsizeof-pointer-memaccess]
    (strncpy(argv[i], "-card", sizeof(argv[i])) == 0) {
        ^~~~~~
hardware/rockchip/audio/tinyalsa_hal/antx.c:67:47: note: did you mean to provide an explicit length?
    (strncpy(argv[i], "-card", sizeof(argv[i])) == 0) {
        ^~~~~~
2 warnings generated.
[ 98% 281/285] target C: antx <= hardware/rockchip/audio/tinyalsa_hal/alsa_mixer.c
hardware/rockchip/audio/tinyalsa_hal/alsa_mixer.c:35:9: warning: '___bitwise' macro redefined [-Wmacro-redefined]
#define ___bitwise
        ^
bionic/libc/kernel/uapi/linux/types.h:25:9: note: previous definition is here
#define ___bitwise
        ^
1 warning generated.
[100% 285/285] Install: out/target/product/rk3568_r/vendor/bin/antx
### build completed successfully (01:03 (m:ss)) ###
topeet@ubuntu:~/Android/rk_android11.0_sdk/hardware/rockchip/audio/tinyalsa_hal$
```

在 out 目录就有新生成的 so 文件:

out\target\product\rk3568_r\vendor\lib\hw\audio.primary.rk30board.so 或者

out\target\product\rk3568_r\vendor\lib64\hw\audio.primary.rk30board.so

```
topeet@ubuntu:~/Android/rk_android11.0_sdk$ ls out/target/product/rk3568_r/vendor/lib/hw/audio.primary.rk30board.so
out/target/product/rk3568_r/vendor/lib64/hw/audio.primary.rk30board.so
topeet@ubuntu:~/Android/rk_android11.0_sdk$
```

分别把 lib 和 lib64 目录下的 audio.primary.rk30board.so 使用 adb 工具 push 到 Android 开发板的 /vendor/lib/hw 或者 /vendor/lib64/hw 目录，重启系统验证。

使用 mmm 命令和 mm 命令最大的区别是 mmm 命令后面需要带上全路径。编译结果和 mm 命令编译相同。

如果无法使用 mm 编译，可以使用方法一来编译 android 的整个部分。

1.6 打包 update.img

在 uboot kernel android 都编译完成的情况下，才可以打包 update.img，所以一般在完整编译的时候用。输入以下命令：

```
./build.sh -u
```

```
topeet@ubuntu:~/Android/rk_android11.0_sdk$ ./build.sh -u
will build update.img
-----KERNEL_DTS:rk3568-evb1-ddr4-v10

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=11
TARGET_PRODUCT=rk3568_r
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-110-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=RQ3A.210705.001
OUT_DIR=out
=====
```

打包完成会在 rockdev/Image-rk3568_r 目录下生成 update.img 镜像。

第 2 章 编译 Android11 问题总结

问题一：编译内核报错 warning: override: reassigning to symbol

```
topeet@ubuntu:~/Android/rk_android11.0_sdk$ cd kernel && make ARCH=arm64 CC=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/clang LD=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/ld.lld rockchip_defconfig android-11.config && make ARCH=arm64 CC=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/clang LD=../prebuilts/clang/host/linux-x86/clang-r383902b/bin/ld.lld BOOT_IMG=../rockdev/Image-rk3568_r/boot.img rk3568-evb1-ddr4-v10.img && cd -
arch/arm64/configs/rockchip_defconfig:2098:warning: override: reassigning to symbol TOUCHSCREEN_EDT_FT5X06
arch/arm64/configs/rockchip_defconfig:4428:warning: override: reassigning to symbol RTC_DRV_RX80D10
arch/arm64/configs/rockchip_defconfig:6052:warning: override: reassigning to symbol DEVMEM
#
# configuration written to .config
#
Using .config as base
```

解决方法：

warning 字面意思是“覆盖：重新分配给符号 XXX”，打开默认的设备树文件 kernel/arch/arm64/configs/

rockchip_defconfig 文件。打开 2098 行，如下图所示：

```
2089 # CONFIG_TOUCHSCREEN_MCS5000 is not set
2090 # CONFIG_TOUCHSCREEN_MMS114 is not set
2091 # CONFIG_TOUCHSCREEN_MELFAS_MIP4 is not set
2092 # CONFIG_TOUCHSCREEN_MTOUCH is not set
2093 # CONFIG_TOUCHSCREEN_IMX6UL_TSC is not set
2094 # CONFIG_TOUCHSCREEN_INEXIO is not set
2095 # CONFIG_TOUCHSCREEN_MK712 is not set
2096 # CONFIG_TOUCHSCREEN_PENMOUNT is not set
2097 # CONFIG_TOUCHSCREEN_EDT_FT5X06 is not set
2098 CONFIG_TOUCHSCREEN_EDT_FT5X06=v
2099 # CONFIG_TOUCHSCREEN_TOUCHRIGHT is not set
```

如上图所示，配置文件的 2098 行和 2097 行有点矛盾，一个说配置了，一个说没配置。只要把 2097 行矛盾的配置删掉，重新编译，发现警告消失了。

问题二：如何查询编译的帮助信息呢？

输入以下命令查看编译帮助信息，如下图所示：

./build.sh -h

```
topeet@ubuntu:~/Android11/rk_android11.0_sdk$ ./build.sh -h
./build.sh: illegal option -- h
USAGE: [-U] [-CK] [-A] [-p] [-o] [-u] [-v VERSION_NAME]
No ARGS means use default build option
WHERE: -U = build uboot
       -C = build kernel with Clang
       -K = build kernel
       -A = build android
       -p = will build packaging in IMAGE
       -o = build OTA package
       -u = build update.img
       -v = build android with 'user' or 'userdebug'
       -d = build kernel dts name
       -V = build version
       -J = build jobs
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
topeet@ubuntu:~/Android11/rk_android11.0_sdk$
```

“-U”表示编译 uboot

“-CK”表示编译内核

“-A”表示编译 Android 文件系统

“-u”表示所有的镜像编译完成后执行打包程序（生成 update.img 镜像）

“-J=”表示多线程编译（比如-j=20）

第 3 章 编译 Android12 源码包

编译环境说明：

本手册使用的是迅为提供的编译环境 ubuntu18.04，在网盘资料“**iTOP-3568 开发板\02_【iTOP-RK3568 开发板】开发资料\02_iTOP-3568 开发板搭建好编译环境的 Ubuntu18.04 虚拟机镜像**”路径下。这里的 ubuntu 默认开发环境已经搭建好。

因 Ubuntu 版本众多，无法将所有版本 Ubuntu 全部测试通过，如使用其他开发环境，在开发过程中遇到问题，需自行研究解决。

在编译源码之前，首先要阅读并且操作《**【北京迅为】itop-3568 开发板快速使用编译环境 Ubuntu18.04 v1.0.doc**》手册，才可进行编译工作。

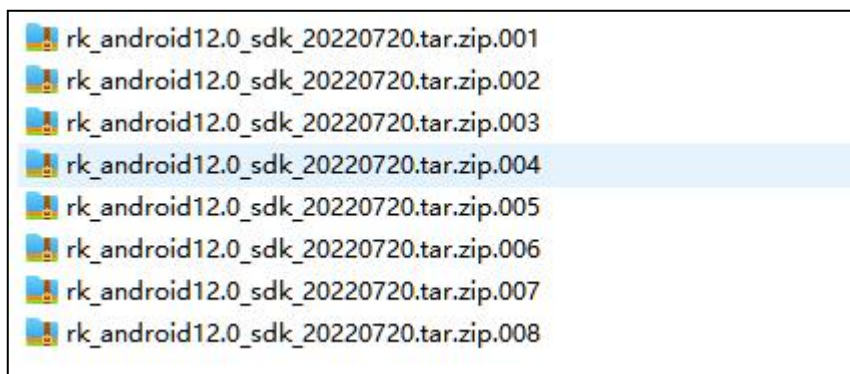
3.1 获取 Android12 源码

Android12 源码网盘下载路径：“**iTOP-3568 开发板\01_【iTOP-RK3568 开发板】基础资料【V1.7 版本】\010_iTOP-RK3568 开发板 Android12 源码**”

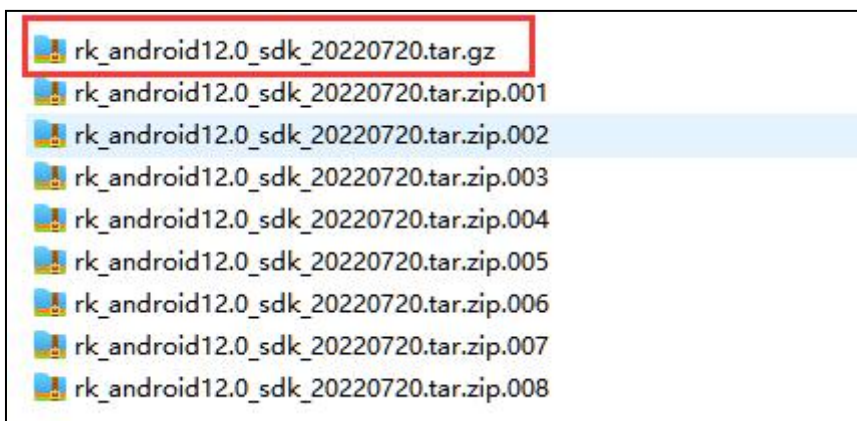
源码是分卷压缩包，需要全部下载下来放在同一级文件夹。所以要从网盘将 Android12 源码包 rk_android12.0_sdk_20220720.tar.zip.001 至 rk_android12.0_sdk_20220720.tar.zip.00X（X 代表 1, 2, 3.....）等分卷压缩包全部下载到 windows 电脑上，并放在同一级目录下。

源码名称说明：源码会逐渐更新，发布的源码日期会有所不同，具体以网盘中实际名称为准。

下载的源码，如下图所示：



右键点击第一个分卷压缩包 rk_android12.0_sdk_20220720.tar.zip.001，选择“解压到 rk_android12.0_sdk_20220720XXX”，（XXX 为省略），解压时间会比较长，耐心等待解压完以后会生成一个 rk_android12.0_sdk_20220720.tar.gz 名字的压缩包，如下图所示：



使用 SSH 软件拷贝 rk_android12.0_sdk_20220720.tar.gz 文件到 Ubuntu 系统，如下图所示：

```
root@ubuntu:/home/topeet/Android# ls
rk_android12.0_sdk rk_android12.0_sdk_20220720.tar.gz
root@ubuntu:/home/topeet/Android#
root@ubuntu:/home/topeet/Android#
```

输入以下命令解压压缩包

```
tar -vxf rk_android12.0_sdk_20220720.tar.gz
```

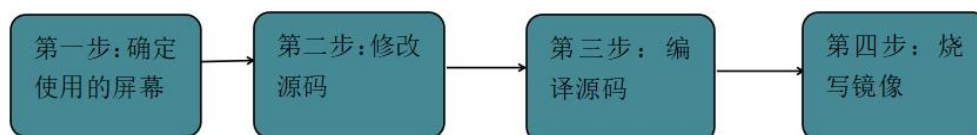
```
topeet@ubuntu:~/Android12$
topeet@ubuntu:~/Android12$ tar -vxf rk_android12.0_sdk_20230517.tar.gz
```

解压完毕生成 rk_android12.0_sdk 文件夹，如下图所示：

```
topeet@ubuntu:~/Android$ ls
Android.bp  bootstrap.bash  rk_android12.0_sdk  development  hardware  kernel-5.10  Makefile  out  packages  prebuilts  rkst  system  u-boot
art         build          cts                device       javaenv.sh  libcore      mkcombinedroot  pdk  platform_testing  restore_patches.sh  RKTools  test  vendor
bionic     BUILD         dalvik             external     kernel      libnativehelper  nkimage_ab.sh  platform_testing  rkbin  rockdev  toolchain  WORKSPACE
bootable   build.sh      developers         frameworks  kernel-4.19  libopenCL.so  nktmage.sh      RKDocs  sdk  tools
```

3.2 设置屏幕配置

在源码编译之前首先要确定自己想要使用的屏幕并修改源码，在编译镜像，烧写镜像。如下图所示：



第一步：确定要使用的屏幕种类，屏幕种类选择如下所示：

iTOP-3568 开发板支持以下种类屏幕：

- 迅为 LVDS 7 寸屏幕
- 迅为 LVDS 10.1 寸 1024*600 屏幕

- 迅为 LVDS 10.1 寸 1280*800 屏幕
- 迅为 MIPI 7 寸屏幕
- 标准 HDMI 屏幕（通过 HDMI 线连接）
- 标准 HDMI 屏幕（通过 VGA 线连接）

第二步：修改源码的设备树

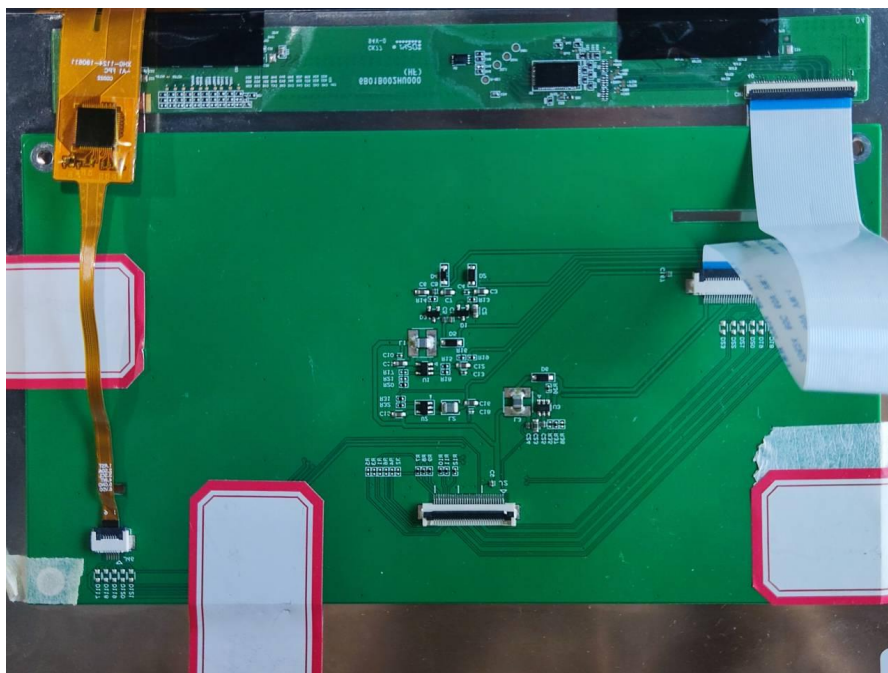
修改 rk_android12.0_sdk/kernel-4.19/arch/arm64/boot/dts/rockchip 目录下的 topeet_screen_choose.dtsi 文件。打开如下图所示：

```
/******单屏显示******/
#define LCD_TYPE_MIPI //in vp 1
//#define LCD_TYPE_LVDS_10_1_1024X600 //in vp 2
//#define LCD_TYPE_LVDS_10_1_1280X800_gt911 //in vp 2
//#define LCD_TYPE_LVDS_10_1_1280X800_gt9271 //in vp 2
//#define LCD_TYPE_LVDS_7_0 //in vp 2
//#define LCD_TYPE_EDP_VGA //in vp 0
//#define LCD_TYPE_HDMI_VP0 //hdmi in vp 0
//#define LCD_TYPE_HDMI_VP1 //hdmi in vp 1
```

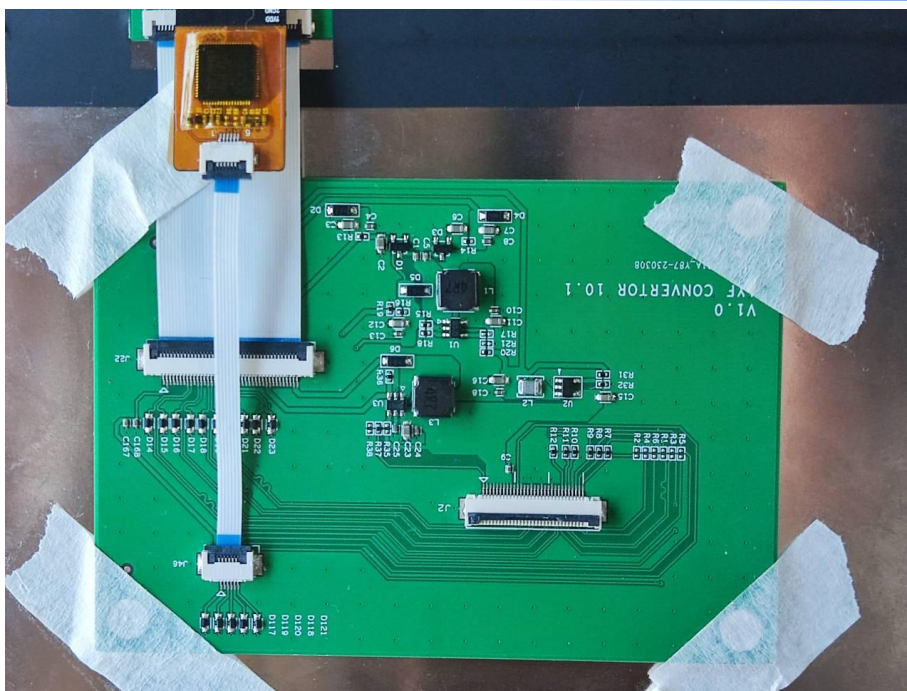
如上图所示，迅为的 Android12 源码支持 MIPI7 寸屏幕，LVDS 10.1 寸 1024*600 屏幕，LVDS 10.1 寸 1280*800 屏幕，LVDS 7 寸屏幕，VGA 接口连接显示器，HDMI 接口连接显示。

注：LVDS 10.1 寸 1280*800 屏幕分为#define LCD_TYPE_LVDS_10_1_1280X800_gt911 和 #define LCD_TYPE_LVDS_10_1_1280X800_gt9271 两种，可根据屏幕背板区分，如下图所示：

#define LCD_TYPE_LVDS_10_1_1280X800_gt911



#define LCD_TYPE_LVDS_10_1_1280X800_gt9271



若选择 define LCD_TYPE_LVDS_10_1_1280X800_gt9271 宏定义则需修改默认配置文件 kernel-4.19/arch/arm64/configs/rockchip_defconfig，修改完成如下图所示：

```
2122 CONFIG_TOUCHSCREEN_GSL3673_800X1280=y
2123 CONFIG_TOUCHSCREEN_GSL3676=v
2124 #CONFIG_TOUCHSCREEN_GT9XX=y
2125 CONFIG_TOUCHSCREEN_GT9271=y
2126 # CONFIG_TOUCHSCREEN_HIDEAP is not set
2127 CONFIG_TOUCHSCREEN_HYN_CST2XX=y
2128 # CONFIG_TOUCHSCREEN_ILI210X is not set
```

若没有使用屏幕，使能 LCD_TYPE_HDMI_VP0 宏定义即可。

我们如果想要连接某个屏幕，使能对应的屏幕宏定义即可。如果想要设置多屏显示，请参考《【北京迅为】itop-3568 开发板多屏显示手册 V1.0.doc》。

3.3 修改编译线程数

在编译 Android 系统时，可以通过调整线程数来控制并行编译的任务数量，使用多线程可以加快构建速度。要注意根据 Ubuntu 的处理器内核总数以及内存大小来设置合适的线程数。过高的线程数可能导致系统资源的竞争，过低的线程数则不能充分利用系统的并行处理能力。找到合适的线程数可以加快编译速度，提高效率。

可以在 build.sh 编译脚本中修改线程数，如下图所示：


```
source build/envsetup.sh >/dev/null
BUILD_UBOOT=false
BUILD_KERNEL_WITH_CLANG=false
BUILD_KERNEL=false
BUILD_ANDROID=false
BUILD_AB_IMAGE=false
BUILD_UPDATE_IMG=false
BUILD_OTA=false
BUILD_PACKING=false
BUILD_VARIANT='get_build_var TARGET_BUILD_VARIANT'
KERNEL_DTS=""
BUILD_VERSION=""
BUILD_JOBS=16
```

3.4 整体编译 Android 固件

注意：编译系统时一定要使用普通权限编译。

进入到 rk_android12.0_sdk 文件夹，输入以下命令设置 java 版本为 1.8 版本，确认 java 版本是 1.8 版本之后，才可以进行下一步编译，如下图所示：

```
source javaenv.sh
```

```
java -version
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ source javaenv.sh
topeet@ubuntu:~/Android/rk_android12.0_sdk$ java -version
openjdk version "1.8.0_312"
OpenJDK Runtime Environment (build 1.8.0_312-8u312-b07-0ubuntu1~18.04-b07)
OpenJDK 64-Bit Server VM (build 25.312-b07, mixed mode)
topeet@ubuntu:~/Android/rk_android12.0_sdk$
topeet@ubuntu:~/Android/rk_android12.0_sdk$
```

输入以下命令使能编译环境：

```
source build/envsetup.sh
```

```
lunch rk3568_s-userdebug
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ source build/envsetup.sh
topeet@ubuntu:~/Android/rk_android12.0_sdk$ lunch rk3568_s-userdebug
```

```
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=rk3568_s
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-122-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SP2A.220305.012
OUT_DIR=out
=====
topeet@ubuntu:~/Android/rk_android12.0_sdk$
```

最后输入以下命令开始整体编译

```
./build.sh -AUCKu
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$
topeet@ubuntu:~/Android/rk_android12.0_sdk$ ./build.sh -AUCKu
```

编译完成如下图所示:

```
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x29a,userspace=0x1
Add file: ./Image/MiniloaderAll.bin
bootloader,Add file: ./Image/MiniloaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x282,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/dtbo.img
dtbo,Add file: ./Image/dtbo.img done,offset=0x47f800,size=0x26f,userspace=0x1
Add file: ./Image/vbmeta.img
vbmeta,Add file: ./Image/vbmeta.img done,offset=0x480000,size=0x1000,userspace=0x2
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x481000,size=0x1e6c800,userspace=0x3cd9
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x22ed800,size=0x3d69800,userspace=0x7ad3
Add file: ./Image/baseparameter.img
baseparameter,Add file: ./Image/baseparameter.img done,offset=0x6057000,size=0x100000,userspace=0x200
Add file: ./Image/super.img
super,Add file: ./Image/super.img done,offset=0x6157000,size=0x64676574,userspace=0xc8ced
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making update.img OK.
Make update image ok!
/home/topeet/Android/rk_android12.0_sdk
topeet@ubuntu:~/Android/rk_android12.0_sdk$
```


编译生成的镜像统一放在 rockdev/Image-rk3568_s 目录下，

生成的镜像包括 MiniLoaderAll.bin、uboot.img、trust.img、resource.img、boot.img、misc.img、recovery.img、boot-debug.img、dtbo.img、pcba_small_misc.img、pcba_whole_misc.img、super.img、vbmeta.img 等 13 个镜像，但是在烧写系统时，这 13 个镜像并不需要全部使用。

```
topeet@ubuntu:~/Android/rk_android12.0_sdk/rockdev/Image-rk3568_s$  
topeet@ubuntu:~/Android/rk_android12.0_sdk/rockdev/Image-rk3568_s$ ls  
baseparameter.img boot.img dtbo.img misc.img pcba_small_misc.img recovery.img super.img update.img  
boot-debug.img config.cfg MiniLoaderAll.bin parameter.txt pcba_whole_misc.img resource.img uboot.img vbmeta.img  
topeet@ubuntu:~/Android/rk_android12.0_sdk/rockdev/Image-rk3568_s$  
topeet@ubuntu:~/Android/rk_android12.0_sdk/rockdev/Image-rk3568_s$
```

镜像作用如下表所示：

镜像	含义
boot-debug.img	与 boot.img 的差别是 user 固件可以烧写这个 boot.img 进行 root 权限操作
config.cfg	烧写工具的配置文件，可以直接导入烧写工具显示需要烧写的选项。
MiniLoaderAll.bin	包含一级 loader。
misc.img	包含 recovery-wipe 开机标识信息，烧写后会进行 recovery。
parameter.txt	包含分区信息。
pcba_small_misc.img	包含 pcba 开机标识信息，烧写后会进入简易版 pcba 模式。
pcba_whole_misc.img	包含 pcba 开机标识信息，烧写后会进入完整版 pcba 模式
recovery.img	包含 recovery-ramdis、kernel、dtb
super.img	包含 odm、product、vendor、system、system_ext 分区内容。
trust.img	包含 BL31、BL32 RK3566/RK3568 没有生成这个固件，不需要烧写
uboot.img	包含 uboot 固件
vbmeta.img	包含 avb 校验信息，用于 AVB 校验
update.img	包含以上需要烧写的 img 文件，可以用于工具直接烧写整个固件包。

3.5 单独编译 Android 固件

进入到 rk_android12.0_sdk 文件夹，输入以下命令设置 java 版本为 1.8 版本，如下图所示：

```
source javaenv.sh
```

```
java -version
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ source javaenv.sh  
topeet@ubuntu:~/Android/rk_android12.0_sdk$ java -version  
openjdk version "1.8.0_312"  
OpenJDK Runtime Environment (build 1.8.0_312-8u312-b07-0ubuntu1~18.04-b07)  
OpenJDK 64-Bit Server VM (build 25.312-b07, mixed mode)  
topeet@ubuntu:~/Android/rk_android12.0_sdk$  
topeet@ubuntu:~/Android/rk_android12.0_sdk$
```

输入以下命令使能编译环境：

```
source build/envsetup.sh
```

```
lunch rk3568_s-userdebug
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ source build/envsetup.sh
topeet@ubuntu:~/Android/rk_android12.0_sdk$ lunch rk3568_s-userdebug

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=rk3568_s
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-122-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SP2A.220305.012
OUT_DIR=out
=====
topeet@ubuntu:~/Android/rk_android12.0_sdk$
```

执行完上述命令，如果需要编译 uboot，则参考编译 uboot 小节，如果需要编译 Android 内核小节，如果编译 Android 文件系统，则参考编译 Android 文件系统小节。
注意！ 要用同一个窗口普通用户进行编译。

3.5.1 编译 uboot

方法一：

在 Android 源码目录下执行如下命令编译 uboot：

```
./build.sh -U
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ ./build.sh -U
will build u-boot
-----KERNEL_VERSION:4.19
-----KERNEL_DTS:rk3568-evb1-ddr4-v10

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=rk3568_s
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-122-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SP2A.220305.012
OUT_DIR=out
=====
start build uboot
```

编译完 uboot 如下图所示:

```
Packed resources:
rk-kernel.dtb battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp battery_5.bmp battery_fail.bmp logo.bmp logo_kernel.bmp battery_0.bmp 10

./resource.img with battery images is ready
/home/topeet/Android/rk_android12.0_sdk
Repacking header 2 boot...
make and copy android images
TARGET_PRODUCT=rk3568_s
TARGET_BASE_PARAMETER_IMAGE==device/rockchip/common/baseparameter/v2.0/baseparameter.img
HIGH_RELIABLE_RECOVERY_OTA=
BOARD_AVB_ENABLE=false
system filesystem is ext4
create dtbo.img...
done.
create rockdev/Image-rk3568_s/resource.img...
done.
create rockdev/Image-rk3568_s/boot.img...
done.
create rockdev/Image-rk3568_s/boot-debug.img...
done.
skip copy images: /home/topeet/Android/rk_android12.0_sdk/out/target/product/rk3568_s/vendor_boot.img
skip copy images: /home/topeet/Android/rk_android12.0_sdk/out/target/product/rk3568_s/vendor_boot-debug.img
create rockdev/Image-rk3568_s/recovery.img...
done.
create rockdev/Image-rk3568_s/super.img...
done.
skip copy images: /home/topeet/Android/rk_android12.0_sdk/out/target/product/rk3568_s/userdata.img
create vbmeta.img...
BOARD_AVB_ENABLE is false, use default vbmeta.img
create misc.img.... done.
create uboot.img...
u-boot/trust.img not found! Please make it from u-boot first!
create loader...
create config.cfg...
create baseparameter...done.
Make image ok!
topeet@ubuntu:~/Android/rk_android12.0_sdk$
```

编译完成后在 u-boot 目录会生成 **rk356x_spl_loader_v1.13.112.bin** 文件、**uboot.img** 文件、**resource.img** 三个镜像文件。

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ cd u-boot/
topeet@ubuntu:~/Android/rk_android12.0_sdk/u-boot$ ls
api          bl31_0xfdc9000.bin  common  Documentation  fit      lib          net          rk356x_spl_loader_v1.13.112.bin  tee.bin  u-boot.bin  u-boot.cfg  u-boot.img  u-boot.ldf  u-boot.syn
arch         bl31_0xfdc9000.bin  config.mk  drivers        fs       licenses    post         snapshot.commit  test     u-boot.cfg.configs  u-boot.map  u-boot.nodtb.bin  u-boot.srec
bl31_0x00040000.bin  bl31.elf            configs  dts            include  MAINTAINERS  PREUPLOAD.cfg  spl          tools     u-boot.dtb  u-boot.dtb.bin  u-boot.srec
bl31_0x00060000.bin  board               disk     env            Kbuild  Makefile     README         System.map    tpl       u-boot.dtb
bl31_0x00080000.bin  cmd                 doc      examples       Kconfig  make.sh      resource.img    u-boot       u-boot
```


同时编译成功后会将 `rk356x_spl_loader_v1.13.112.bin`、`uboot.img` 拷贝到 `rockdev/Image-rk3568_s` 目录下，`rk356x_spl_loader_v1.13.112` 会改名为 `MiniLoaderAll.bin`。`MiniLoaderAll.bin` 也就是一级 loader,只需要将 `MiniLoaderAll.bin` 和 `uboot.img` 烧写到开发板即可更新 `uboot`。

方法二：

方法一使用的是整体编译脚本,也可以使用 `u-boot` 下的编译脚本。

如果只是调试 `uboot` 或者 `rkbin`，可以在 `u-boot` 目录下直接编译，这样编译速度更快，如下图所示，输入以下命令：

```
cd u-boot/  
./make.sh rk3568
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ cd u-boot/  
topeet@ubuntu:~/Android/rk_android12.0_sdk/u-boot$ ./make.sh rk3568  
## make rk3568_defconfig -j8  
#  
# configuration written to .config  
#  
scripts/kconfig/conf --silentoldconfig Kconfig  
CHK include/config.h  
CFG u-boot.cfg  
GEN include/autoconf.mk.dep  
CFG spl/u-boot.cfg  
CFG tpl/u-boot.cfg  
GEN include/autoconf.mk  
GEN tpl/include/autoconf.mk  
GEN spl/include/autoconf.mk  
CHK include/config/uboot.release  
CHK include/generated/timestamp_autogenerated.h  
UPD include/generated/timestamp_autogenerated.h  
CHK include/generated/version_autogenerated.h  
CHK include/generated/generic-asm-offsets.h  
CHK include/generated/asm-offsets.h  
HOSTCC tools/mkenvimage.o  
HOSTCC tools/fit_image.o  
HOSTCC tools/image-host.o  
HOSTCC tools/dumpimage.o  
HOSTCC tools/mkimage.o  
HOSTCC tools/rockchip/boot_merger.o  
HOSTCC tools/rockchip/loaderimage.o  
CHK include/config.h  
HOSTLD tools/mkenvimage
```

这样编译后会在 `uboot` 目录下生成 `uboot.img` 和 `rk356x_spl_loader_v1.13.112.bin`，但是不会拷贝到 `rockdev/Image-rk3568_s` 下面，所以需要烧写 `uboot` 目录下的 `uboot.img` 和 `rk356x_spl_loader_v1.13.112.bin`，烧写时将 `rk356x_spl_loader_v1.13.112.bin` 重命名为 `MiniLoaderAll.bin`，再烧写。

3.5.2 编译 Android 内核

方法一：

在 Android 源码目录下执行如下命令编译 Android 内核。

```
./build.sh -CKA
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ ./build.sh -CKA
will build kernel with clang
will build kernel
will build android
-----KERNEL_VERSION:4.19
-----KERNEL_DTS:rk3568-evb1-ddr4-v10
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=rk3568_s
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-122-generic-x86_64-Ubuntu-18.04.6-LTS
```

编译后会在 rockdev/Image-rk3568_s 目录下生成 boot.img，boot.img 为内核镜像。boot.img 镜像里面包含了设备树镜像。所以烧写 boot.img 即可更新内核镜像。

解释如下：

参数-C 表示用 clang 编译器编译，如果不带 C 则使用 gcc 编译，如果需要过 GMS 认证的 google 有要求要用 clang 编译。

因为 kernel 编译完后需要通过 Android 去打包成 boot.img，所以这里需要加上 A 参数，即编译 kernel 的时需要一起编译 Android 才能打包 boot.img。通过以上介绍可以知道单独编译 kernel 需要同时编译 Android，导致编译很耗时，针对这个问题，我们推荐使用方法二来单独编译内核。

方法二：

此方法常用于 kernel 的开发和调试，以下的方法既编译 kernel 部分时，同时打包成 boot.img，这样加快了我们开发的速度；

进入内核目录下，输入以下命令：

```
cd kernel
```

```
make ARCH=arm64 CC=./prebuilts/clang/host/linux-x86/clang-r416183b/bin/clang
```

```
LD=./prebuilts/clang/host/linux-x86/clang-r416183b/bin/ld.lld rockchip_defconfig
```

```
android-11.config && make ARCH=arm64
CC=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/clang
LD=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/ld.lld
BOOT_IMG=../rockdev/Image-rk3568_s/boot.img rk3568-evb1-ddr4-v10.img
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk/kernel-4.19$ make ARCH=arm64 CC=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/clang LD=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/ld.lld rockchip_defconfig android-11.config && make ARCH=arm64 CC=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/clang LD=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/ld.lld BOOT_IMG=../rockdev/Image-rk3568_s/boot.img rk3568-evb1-ddr4-v10.img
```

为方便使用，可以将上述命令写成脚本，在 kernel 目录下创建 makekernel.sh,在调试的过程中直接在 kernel 目录下执行该脚本，makekernel.sh 内容为：

```
#!/bin/sh
make ARCH=arm64 CC=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/clang
LD=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/ld.lld rockchip_defconfig
android-11.config && make ARCH=arm64
CC=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/clang
LD=../prebuilts/clang/host/linux-x86/clang-r416183b/bin/ld.lld
BOOT_IMG=../rockdev/Image-rk3568_s/boot.img rk3568-evb1-ddr4-v10.img
```

我们来解释以上命令是什么意思！

BOOT_IMG 是指整编 Android 源码之后在 rockdev 目录下生成的 boot.img，所以在单独编译 Android 内核之前，一定要整编 Android 源码。整编 Android 源码生成的 rockdev 目录下的 boot.img 镜像里面不仅仅只有 kernel 和 resource，还有其他文件。当我们修改了内核源码并单独编译 Android 内核时，kernel 和 resource 发生了改变，这时候需要指定一个 boot.img，然后将新编译的 kernel 和 resource 覆盖进去。打包完成之后，在 kernel 目录下会有新 boot.img 镜像生成，就可以把这个 boot.img 镜像单独烧写到开发板中进行调试了

如果不指定 BOOT_IMG，会在下载镜像之后，系统跑进 recovery 模式，而不是进入正常的启动流程

rk3568-evb1-ddr4-v10.img 的意思是指定所使用的设备树 DTS；

打包完后，在 kernel 目录有 boot.img 镜像生成，就可以把这个 boot.img 镜像单独烧入到机器中进行调试了。

3.5.3 编译 Android 文件系统

方法一：

在源码根目录的 build.sh 脚本，找到 make installclean 命令，将 make installclean 注释掉。

```
176 cp $OUT/$INTERNAL_OTA_PACKAGE TARGET $IMAGE_PATH/  
177 cp $OUT/$INTERNAL_OTA_PACKAGE_OBJ_TARGET $IMAGE_PATH/  
178 else # regular build without OTA  
179     echo "start build android"  
180     #make installclean  
181     make -j$BUILD_JOBS  
182     # check the result of make  
183     if [ $? -eq 0 ]; then  
184         echo "Build android ok!"  
185     else  
186         echo "Build android failed!"  
187         exit 1  
188     fi
```

接着在 Android 源码目录下执行如下命令编译 Android 文件系统：

```
./build.sh -A
```

```
toopeet@ubuntu:~/Android/rk_android12.0_sdk$ ./build.sh -A  
will build android  
-----KERNEL_VERSION:4.19  
-----KERNEL_DTS:rk3568-evb1-ddr4-v10  
  
=====
```

PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=rk3568_s
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-122-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SP2A.220305.012
OUT_DIR=out

Android 编译过程如下图所示：

```
HOST_OS_EXTRA=Linux-5.4.0-122-generic-x86_64-Ubuntu-18.04.6-LTS  
HOST_CROSS_OS=windows  
HOST_CROSS_ARCH=x86  
HOST_CROSS_2ND_ARCH=x86_64  
HOST_BUILD_TYPE=release  
BUILD_ID=SP2A.220305.012  
OUT_DIR=out  
-----
```

[71% 0762/12283] build out/target/product/rk3568_s/system/etc/event-log-tags
packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:6: warning: tag "lock_screen_type" (90200) duplicated in packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:10
packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:9: warning: tag "exp_det_device_admin_activated_by_user" (90201) duplicated in packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:12
packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:12: warning: tag "exp_det_device_admin_declined_by_user" (90202) duplicated in packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:15
packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:15: warning: tag "exp_det_device_admin_uninstalled_by_user" (90203) duplicated in packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:18
packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:18: warning: tag "settings_latency" (90204) duplicated in packages/apps/Settings/src/com/android/settings/EventLogTags.logtags:19
[83% 10241/12283] Install: out/target/product/rk3568_s/vendor/lib/egl/libGLES_mali.so
/home/toopeet/Android/rk_android12.0_sdk
/home/toopeet/Android/rk_android12.0_sdk
[87% 10701/12283] Install: out/target/product/rk3568_s/vendor/lib64/egl/libGLES_mali.so
/home/toopeet/Android/rk_android12.0_sdk
/home/toopeet/Android/rk_android12.0_sdk
[88% 10854/12283] Building dtbo.img file out/target/product/rk3568_s/obj/FAKE/rockchip_dtbo_intermediates/rebuild-dtbo.img.
Create image file: out/target/product/rk3568_s/obj/FAKE/rockchip_dtbo_intermediates/rebuild-dtbo.img...
Total 1 entries.
[93% 11450/12283] depmod out/target/product/rk3568_s/obj/PACKAGING/depmod_VENDOR_intermediates
depmod: WARNING: could not open modules.order at /home/toopeet/Android/rk_android12.0_sdk/out/target/product/rk3568_s/obj/PACKAGING/depmod_VENDOR_intermediates/lib/modules/0.0: No such file or directory
depmod: WARNING: could not open modules.built-in at /home/toopeet/Android/rk_android12.0_sdk/out/target/product/rk3568_s/obj/PACKAGING/depmod_VENDOR_intermediates/lib/modules/0.0: No such file or directory
[98% 12049/12283] Target empty super fs image: out/target/product/rk3568_s/super_empty.img
2022-07-18 22:22:06 - build_super_image.py - INFO : Building super image from info dict...
2022-07-18 22:22:06 - build_super_image.py - INFO : Done writing image out/target/product/rk3568_s/super_empty.img
[98% 12087/12283] build out/target/product/rk3568_s/obj/FAKE/treble_sepolicy_tests_31.0_intermediates/31.0_compat
0:31 build out/target/product/rk3568_s/obj/ETC/sepolicy_intermediates/sepolicy
0:15 build out/target/product/rk3568_s/obj/ETC/precompiled_sepolicy_intermediates/precompiled_sepolicy
0:11 build out/target/product/rk3568_s/obj/ETC/recovery_recovery.img

Android 系统编译完如下图所示：


```
Build android ok!
make and copy android images
TARGET_PRODUCT=rk3568_s
TARGET_BASE_PARAMETER_IMAGE==device/rockchip/common/baseparameter/v2.0/baseparameter.img
HIGH_RELIABLE_RECOVERY_OTA=
BOARD_AVB_ENABLE=false
system filesystem is ext4
create dtbo.img...
done.
create rockdev/Image-rk3568_s/resource.img...
done.
create rockdev/Image-rk3568_s/boot.img...
done.
create rockdev/Image-rk3568_s/boot-debug.img...
done.
skip copy images: /home/topeet/Android/rk_android12.0_sdk/out/target/product/rk3568_s/vendor_boot.img
skip copy images: /home/topeet/Android/rk_android12.0_sdk/out/target/product/rk3568_s/vendor_boot-debug.img
create rockdev/Image-rk3568_s/recovery.img...
done.
create rockdev/Image-rk3568_s/super.img...
done.
skip copy images: /home/topeet/Android/rk_android12.0_sdk/out/target/product/rk3568_s/userdata.img
create vbmeta.img...
BOARD_AVB_ENABLE is false, use default vbmeta.img
create misc.img.... done.
create uboot.img...
u-boot/trust.img not found! Please make it from u-boot first!
create loader...
create config.cfg...
create baseparameter...done.
Make image ok!
topeet@ubuntu:~/Android/rk_android12.0_sdk$
```

编译完成后镜像文件会在 rockdev/Image-rk3568_s 目录下生成 super.img, 单独烧写 super.img 即可。

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ cd rockdev/Image-rk3568_s/
topeet@ubuntu:~/Android/rk_android12.0_sdk/rockdev/Image-rk3568_s$ ls
baseparameter.img  boot.img  dtbo.img  misc.img  pcba_small_misc.img  recovery.img  super.img  vbmeta.img
boot-debug.img  config.cfg  MiniLoaderAll.bin  parameter.txt  pcba_whole_misc.img  resource.img  uboot.img
topeet@ubuntu:~/Android/rk_android12.0_sdk/rockdev/Image-rk3568_s$
```

各种镜像解释如下：

boot.img: 包含 kernel.img 和 resource.img 镜像文件

boot-debug.img: 与 boot.img 的差别是 user 固件可以烧写这个 boot.img 进行 root 权限操作

config.cfg: 烧写工具的配置文件，可以直接导入烧写工具显示需要烧写的选项。

MiniLoaderAll.bin: 包含一级 loader。

misc.img: 包含 recovery-wipe 开机标识信息，烧写后会进行 recovery。

parameter.txt: 包含分区信息。

pcba_small_misc.img : 包含 pcba 开机标识信息，烧写后会进入简易版 pcba 模式。

pcba_whole_misc.img: 包含 pcba 开机标识信息，烧写后会进入完整版 pcba 模式

recovery.img: 包含 recovery-ramdis、kernel、dtb

super.img: 包含 odm、product、vendor、system、system_ext 分区内容。

trust.img: 包含 BL31、BL32 RK3566/RK3568 没有生成这个固件，不需要烧写

uboot.img: 包含 uboot 固件

vbmeta.img: 包含 avb 校验信息，用于 AVB 校验

update.img: 包含以上需要烧写的 img 文件，可以用于工具直接烧写整个固件包。

方法二：

在 Android 源码编译的时候，如果编译整个工程可能需要两三个小时，即使二次编译也要二十分钟，如果只想单独编译某个模块，则可以使用 `mm` 命令，编译出来的 `so` 库或者其他文件，使用 `adb` 工具 `push` 到机器中进行替换，重启系统验证，不必重新烧写固件。

在使用 `m/mm/mmm` 命令之前，首先要确保执行以下命令，以下命令的作用是将 `mm` 等这些命令添加到环境变量当中去，没执行的话，会找不到命令。

```
source javaenv.sh
source build/envsetup.sh
lunch rk3568_s-userdebug
```

我们来了解下命令的含义：

`m`：编译所有的模块；

`mm`：编译当前目录下的模块，当前目录下要有 `Android.mk` 文件；

`mmm`：编译指定路径下的模块，指定路径下要有 `Android.mk` 文件；

比如说，修改了 HAL 层的音频文件：`hardware/rockchip/audio/tinyalsa_hal/audio_hw.c`，就可以使用 `mm` 命令来进行编译，首先要进入到 `hardware/rockchip/audio/tinyalsa_hal/` 目录下，注意要有 `Android.mk` 文件哦，然后直接输入 `mm` 编译即可，如下图所示：

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ cd hardware/rockchip/audio/tinyalsa_hal/
topeet@ubuntu:~/Android/rk_android12.0_sdk/hardware/rockchip/audio/tinyalsa_hal$ mm
22:27:33 *****
22:27:33 You are building on a machine with 15.6GB of RAM
22:27:33
22:27:33 The minimum required amount of free memory is around 16GB,
22:27:33 and even with that, some configurations may not work.
22:27:33
22:27:33 If you run into segfaults or other errors, try reducing your
22:27:33 -j value.
22:27:33 *****
build/make/core/soong_config.mk:195: warning: BOARD_PLAT_PUBLIC_SEPOLICY_DIR has been deprecated. Use SYSTEM_EXT_PUBLIC_SEPOLICY_DIRS instead.
build/make/core/soong_config.mk:196: warning: BOARD_PLAT_PRIVATE_SEPOLICY_DIR has been deprecated. Use SYSTEM_EXT_PRIVATE_SEPOLICY_DIRS instead.
=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=rk3568_s
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-122-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
```

编译完成如下图所示：

```
#####
[ 44% 4/9] target C: amix <- hardware/rockchip/audio/tinyalsa_hal/amix.c
hardware/rockchip/audio/tinyalsa_hal/amix.c:66:44: warning: 'strncpy' call operates on objects of type 'char' while the size is based on a different type 'char *' [-Wsizeof-pointer-memaccess]
    if ((strncpy(argv[i], "-c", sizeof(argv[i])) == 0) ||
        ^~~~~~
hardware/rockchip/audio/tinyalsa_hal/amix.c:67:47: note: did you mean to provide an explicit length?
    if ((strncpy(argv[i], "-c", sizeof(argv[i])) == 0) ||
        ^~~~~~
hardware/rockchip/audio/tinyalsa_hal/amix.c:67:47: warning: 'strncpy' call operates on objects of type 'char' while the size is based on a different type 'char *' [-Wsizeof-pointer-memaccess]
    (strncpy(argv[i], "-card", sizeof(argv[i])) == 0) {
    ^~~~~~
hardware/rockchip/audio/tinyalsa_hal/amix.c:67:47: note: did you mean to provide an explicit length?
    (strncpy(argv[i], "-card", sizeof(argv[i])) == 0) {
    ^~~~~~
2 warnings generated.
[ 55% 5/9] target C: amix <- hardware/rockchip/audio/tinyalsa_hal/alsa_mixer.c
hardware/rockchip/audio/tinyalsa_hal/alsa_mixer.c:35:9: warning: '__bitwise' macro redefined [-Wmacro-redefined]
#define __bitwise
        ^
bionic/libc/kernel/uapi/linux/types.h:25:9: note: previous definition is here
#define __bitwise__
        ^
1 warning generated.
[100% 9/9] Install: out/target/product/rk3568_s/vendor/bin/amix
### build completed successfully (22 seconds) ###
topeet@ubuntu:~/Android/rk_android12.0_sdk/hardware/rockchip/audio/tinyalsa_hal$
```

在 out 目录就有新生成的 so 文件：

out/target/product/rk3568_s/vendor/lib/hw/audio.primary.rk30board.so 或者
out/target/product/rk3568_s/vendor/lib64/hw/audio.primary.rk30board.so

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ ls out/target/product/rk3568_s/vendor/lib64/hw/audio.primary.rk30board.so
out/target/product/rk3568_s/vendor/lib64/hw/audio.primary.rk30board.so
topeet@ubuntu:~/Android/rk_android12.0_sdk$
```

分别把 lib 和 lib64 目录下的 audio.primary.rk30board.so 使用 adb 工具 push 到 Android 开发板的/vendor/lib/hw 或者/vendor/lib64/hw 目录，重启系统验证。

使用 mmm 命令和 mm 命令最大的区别是 mmm 命令后面需要带上全路径。编译结果和 mm 命令编译相同。

如果无法使用 mm 编译，可以使用方法一来编译 android 的整个部分。

3.6 打包 update.img

在 uboot kernel android 都编译完成的情况下，才可以打包 update.img，所以一般在完整编译的时候用。输入以下命令：

```
./build.sh -u
```

```
topeet@ubuntu:~/Android/rk_android12.0_sdk$ ./build.sh -u
will build update.img
-----KERNEL_VERSION:4.19
-----KERNEL_DTS:rk3568-evb1-ddr4-v10

=====
PLATFORM_VERSION_CODENAME=REL
PLATFORM_VERSION=12
TARGET_PRODUCT=rk3568_s
TARGET_BUILD_VARIANT=userdebug
TARGET_BUILD_TYPE=release
TARGET_ARCH=arm64
TARGET_ARCH_VARIANT=armv8-a
TARGET_CPU_VARIANT=cortex-a55
TARGET_2ND_ARCH=arm
TARGET_2ND_ARCH_VARIANT=armv8-2a
TARGET_2ND_CPU_VARIANT=cortex-a55
HOST_ARCH=x86_64
HOST_2ND_ARCH=x86
HOST_OS=linux
HOST_OS_EXTRA=Linux-5.4.0-122-generic-x86_64-Ubuntu-18.04.6-LTS
HOST_CROSS_OS=windows
HOST_CROSS_ARCH=x86
HOST_CROSS_2ND_ARCH=x86_64
HOST_BUILD_TYPE=release
BUILD_ID=SP2A.220305.012
OUT_DIR=out
=====
```

打包完成会在 rockdev/Image-rk3568_s 目录下生成 update.img 镜像。

第 4 章 编译 Linux 源码包

4.1 获取 Linux 源码包

编译环境说明：

本手册使用的是迅为提供的编译环境 ubuntu18.04，在网盘资料“**iTOP-3568 开发板\02_【iTOP-RK3568 开发板】开发资料\02_iTOP-3568 开发板搭建好编译环境的 Ubuntu18.04 虚拟机镜像**”路径下。这里的 ubuntu 默认开发环境已经搭建好。

因 Ubuntu 版本众多，无法将所有版本 Ubuntu 全部测试通过，如使用其他开发环境，在开发过程中遇到问题，需自行研究解决。

在编译源码之前，首先要阅读并且操作《**【北京迅为】itop-3568 开发板快速使用编译环境 Ubuntu18.04 v1.0.doc**》手册，才可进行编译工作。

Linux 源码网盘下载路径：“**iTOP-3568 开发板\01_【iTOP-RK3568 开发板】基础资料【V1.7 版本】\05_iTOP-RK3568 开发板 Linux 源码\01_Linux 系统源码**”

源码名称说明：源码会逐渐更新，发布的源码日期会有所不同，具体以网盘中实际名称为准。

1 从网盘中下载 Linux 源码包，如下图所示：

 01_Linux系统源码	2022/6/20 12:18	文件夹
--	-----------------	-----

2 首先下载 **01_Linux 系统源码** 到 windows 电脑上，如下所示：

 rk356x_linux_20220621.tar.zip.001	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.002	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.003	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.004	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.005	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.006	2022/6/21 23:18	360压缩 分卷文件	4,098,047...

下载后得到的源码如果是分卷压缩包，大家要完整全部下载下来，然后点击第一个分卷压缩包，鼠标点击右键，选择点击“解压到 XXXXXXXX”（XX...为名称的简写），然后等待解压完毕。解压完，会生成一个 XXXXXXXX.tar.gz（XX...为名称的简写）名字的压缩包。然后将 XXXXXXXX.tar.gz（XX...为名称的简写）拷贝到 ubuntu 中解压。

3 在迅为提供的 Ubuntu18.04 中，我们将 rk356x_linux_XXXXXXX.tar.gz（XX...为名称的简写）拷贝到 **/home/topeet/Linux**（若要编译 yocto 则 **Linux 源码必须为此路径，其他系统无要求**）文件夹，使用命令“**tar -vxf rk356x_linux_XXXXXXX.tar.gz（XX...为名称的简写）**”解压缩

缩包。解压后会生成一个 **rk356x_linux** 文件夹，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ls  
app      debian  envsetup.sh  Makefile      repo      rockdev  yocto  
buildroot device  external     mkfirmware.sh rkbin      tools  
build.sh docs    kernel       prebuilts     rkflash.sh u-boot  
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.2 SDK 包说明

4.2.1 SDK 包源码简要

进入源码目录有下图的文件目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ls  
app      debian  envsetup.sh  Makefile      repo      rockdev  yocto  
buildroot device  external     mkfirmware.sh rkbin      tools  
build.sh docs    kernel       prebuilts     rkflash.sh u-boot  
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$
```

从上图可以看出有很多文件夹，我们讲解一下每个目录或者文件的作用：

app：存放上层应用程序的目录，主要是 `qcamera/qfm/qplayer/settings` 等一些应用程序。

buildroot：基于 `buildroot (2018.02-rc3)` 开发的根文件系统。

build.sh：编译用的脚本，使用方法后面会教。

debian：基于 `debian 10` 开发的根文件系统。

device/rockchip：存放每个平台的一些编译和打包固件的脚本和预备文件。

docs：存放 RK 开发指导文件、平台支持列表、工具使用文档、Linux 开发指南等。

envsetup.sh：要修改文件系统时候要设置的环境脚本。

external：存放相关的库，包括音频，视频等。

kernel：kernel 源码。

Makefile：整个 SDK 包编译的 `Makefile`。

mkfirmware.sh：固件打包使用的脚本，默认在当前路径下的 `rockdev` 目录。

prebuilts：存放交叉编译工具链。

rkbin：存放固件和工具。

rkflash.sh：linux 下的系统烧录脚本。

rockdev：存放编译输出固件的目录(整个 SDK 包编译完成后就会创建)。

tools：存放固件和工具的目录。

u-boot：U-boot 源码目录。

yocto：基于 `yocto gatesgarth 3.2` 开发的根文件系统。

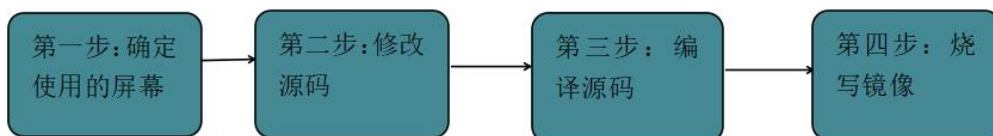
4.2.2 模块代码目录相关说明

什么叫做模块代码？一个完整的 SDK 包除了 kernel、u-boot、buildroot、Debian 和 yocto 之外，还需要提供上层的第三方库和 APP，第三方库和 APP 合起来叫做模块代码。比如：可以做监控摄像头可以参考 common_algorithm 代码。一个 SDK 包包含了很多产品的应用代码。

部分模块代码目录路径	含义
external/linux-rga	Raster Graphic Acceleration(RGA2)
external/recovery	recovery 和 Rockchip 升级代码
external/rkwifibt	Wi-Fi 和 BT
external/rk_pcba_test	PCBA 测试代码
external/isp2-ipc	图像信号处理服务端
external/mpp	编解码代码
external/camera_engine_rkaiq	图像处理算法模块
external/rknpu2	npu 驱动
external/common_algorithm	音视频通用算法库
external/rknn-toolkit2	模型转换、推理和性能评估的开发套件
external/CallFunIpc	应用进程间通信代码
app/multivideoplayer	Buildroot 桌面多路视频播放器应用源码
app/qcamera	Buildroot 桌面相机应用源码
app/qfm	Buildroot 桌面文件管理应用源码
app/qsetting	Buildroot 桌面设置应用源码
app/qplayer	Buildroot 桌面多功能播放器源码

4.3 设置屏幕配置

在源码编译之前首先要确定自己想要使用的屏幕，然后修改源码，编译镜像，烧写镜像。如下图所示：



第一步：确定要使用的屏幕种类，屏幕种类选择如下所示：

iTOP-3568 开发板支持以下种类屏幕：

- 迅为 LVDS 7 寸屏幕
- 迅为 LVDS 10.1 寸 1024*600 屏幕
- 迅为 LVDS 10.1 寸 1280*800 屏幕
- 迅为 MIPI 7 寸屏幕
- 标准 HDMI 屏幕（通过 HDMI 线连接）
- 标准 HDMI 屏幕（通过 VGA 线连接）

第二步：修改源码的设备树

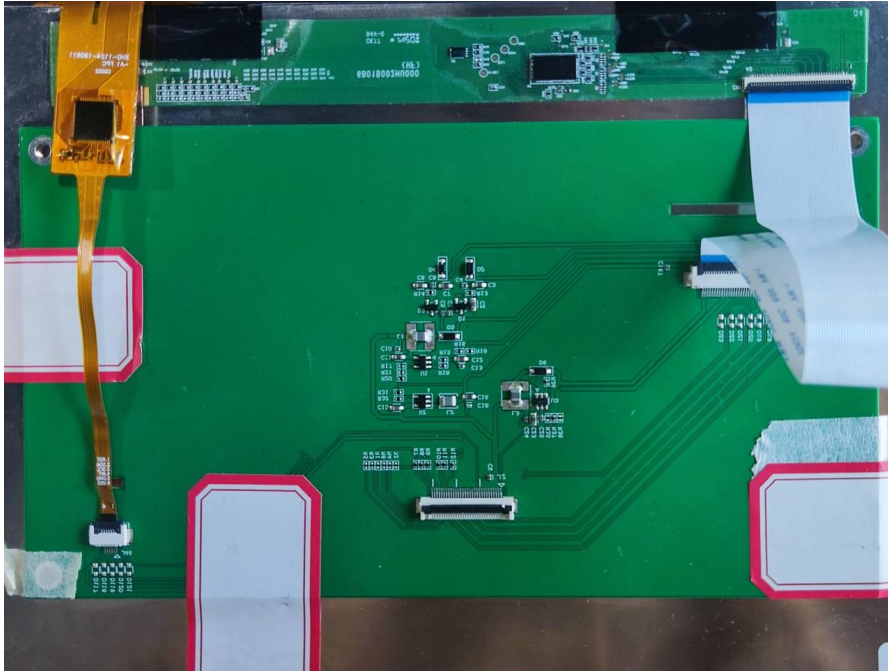
修改 Linux/rk356x_linux/kernel/arch/arm64/boot/dts/rockchip 目录下的
topeet_screen_choose.dtsi 文件。打开如下图所示：

```
/******单屏显示*****/  
#define LCD_TYPE_MIPI //in vp 1  
//#define LCD_TYPE_LVDS_10_1_1024X600 //in vp 2  
//#define LCD_TYPE_LVDS_10_1_1280X800_gt911 //in vp 2  
//#define LCD_TYPE_LVDS_10_1_1280X800_gt9271 //in vp 2  
//#define LCD_TYPE_LVDS_7_0 //in vp 2  
//#define LCD_TYPE_EDP_VGA //in vp 0  
//#define LCD_TYPE_HDMI_VP0 //hdmi in vp 0  
//#define LCD_TYPE_HDMI_VP1 //hdmi in vp 1
```

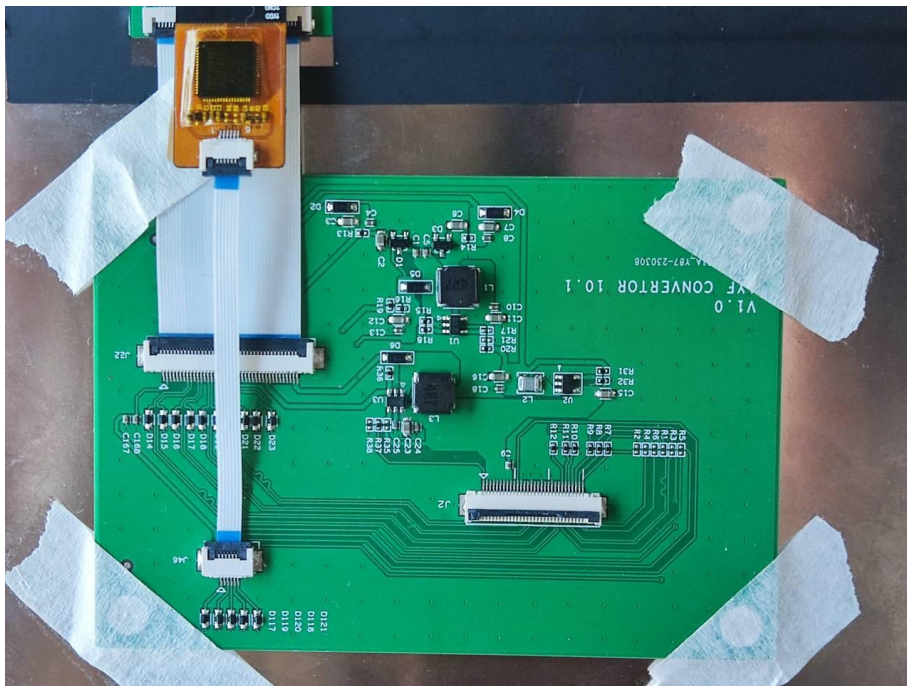
如上图所示，迅为的 Linux 源码支持 MIPI7 寸屏幕，LVDS 10.1 寸 1024*600 屏幕，LVDS 10.1 寸 1280*800 屏幕，LVDS 7 寸屏幕，VGA 接口连接显示器，HDMI 接口连接显示。

注：LVDS 10.1 寸 1280*800 屏幕分为#define LCD_TYPE_LVDS_10_1_1280X800_gt911 和
#define LCD_TYPE_LVDS_10_1_1280X800_gt9271 两种，可根据屏幕背板区分，如下图所示：

#define LCD_TYPE_LVDS_10_1_1280X800_gt911



#define LCD_TYPE_LVDS_10_1_1280X800_gt9271



若选择 define LCD_TYPE_LVDS_10_1_1280X800_gt9271 宏定义则需修改默认配置文件 kernel/arch/arm64/configs/rockchip_linux_defconfig，修改完成如下图所示：

```
2246 # CONFIG_TOUCHSCREEN_GSL3673_800X1280 is not set
2247 # CONFIG_TOUCHSCREEN_GSL3676 is not set
2248 #CONFIG_TOUCHSCREEN_GT9XX=y
2249 CONFIG_TOUCHSCREEN_GT9271=y
2250 # CONFIG_TOUCHSCREEN_HIDEAP is not set
2251 # CONFIG_TOUCHSCREEN_HYN_CST2XX is not set
2252 # CONFIG_TOUCHSCREEN_ILI210X is not set
```

若没有使用屏幕，使能 LCD_TYPE_HDMI_VP0 宏定义即可。

我们如果想要连接某个屏幕，使能对应的屏幕宏定义即可。如果想要设置多屏显示，请参考《【北京迅为】itop-3568 开发板多屏显示手册 V1.0.doc》。

4.4 编译 Buildroot

注意：编译系统时一定要使用普通权限编译。

Buildroot 是一款集成的编译集合包，解决了以前交叉编译麻烦的问题，本小节将介绍 buildroot 镜像的编译流程，分为单独编译和完整全自动编译。

4.4.1 单独编译

4.4.1.1 图形化界面

本小节单独编译镜像的顺序如下所示：

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 buildroot

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 buildroot 镜像 update.img。

第一步：编译 uboot

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh

iTOP-RK3568 building script, http://www.topeet.com

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

光标默认就在 uboot，所以直接回车即可开始 uboot 的编译，编译过程如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
# configuration written to .config
#
```

Uboot 编译完成如下图所示：


```

*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIALL.ini
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.ini

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-
-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 21:17:38 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

编译完成后生成 uboot.img 到 u-boot 目录，如下图所示：

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api                config.mk          Kbuild            scripts            u-boot.dtb
arch               configs            Kconfig           snapshot.commit    u-boot-dtb.bin
bl31_0x00040000.bin disk              lib               spl                uboot.img
bl31_0x00069000.bin doc               Licenses          System.map         u-boot.lds
bl31_0x0006b000.bin Documentation    MAINTAINERS       tee.bin           u-boot.map
bl31_0xfdcc1000.bin drivers          Makefile          test              u-boot-nodtb.bin
bl31_0xfdcce000.bin dts              make.sh           tools             u-boot.srec
bl31_0xfdc00000.bin env              net               tpl               u-boot.sym
bl31.elf           examples         post              u-boot            u-boot.bin
board              fit              PREUPLOAD.cfg    u-boot.bin        u-boot.cfg
cmd                fs               README            u-boot.cfg        u-boot.cfg.configs
common             include          rk356x_spl_loader_v1.13.112.bin
topeet@ubuntu:~/Linux/rk356x_linux$

```

第二步：编译 kernel

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```

topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh

```

iTOP-RK3568 building script, <http://www.topeet.com>

```

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

```

然后将光标移动到第二个 kernel，点击回车即可开始 kernel 内核的编译，编译过程如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
kernel
processing option: kernel
=====Start building kernel=====
TARGET_KERNEL_ARCH =
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
TARGET_KERNEL_DTS =rk3568-evb1-ddr4-v10-linux
TARGET_KERNEL_CONFIG_FRAGMENT =
=====
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
#
# configuration written to .config
#
scripts/kconfig/conf --syncconfig Kconfig
```

内核编译完如下图所示:

```
Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android          build.config.gki_kasan.x86_64  include          net
ar               build.config.gki_kprobes       ini              OWNERS
arch            build.config.gki_kprobes.aarch64  init            README
block          build.config.gki_kprobes.x86_64  ipc             resource.img
boot.img       build.config.gki.x86_64         Kbuild          samples
boot.its       build.config.x86_64            Kconfig         scripts
build.config.aarch64  certs                          kern            securit
build.config.allmodconfig  COPYING                       kernel          security
build.config.allmodconfig.aarch64  CREDITS                     kernel.img      sound
build.config.allmodconfig.arm      crypto                       lib            System.map
build.config.allmodconfig.x86_64    d                            LICENSES       tools
build.config.arm                Documentation                 logo.bmp       usr
build.config.common              dri                          logo_kernel.bmp  virt
build.config.gki                 drive                       MAINTAINERS     vmlinux
build.config.gki.aarch64         drivers                     Makefile        vmlinux.o
build.config.gki-debug.aarch64   firmware                   mm              zboot.img
build.config.gki-debug.x86_64    fs
build.config.gki_kasan
build.config.gki_kasan.aarch64
topeet@ubuntu:~/Linux/rk356x_linux$
```

第三步：编译 Recovery

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第四个 recovery，点击回车即可开始 recovery 的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====

#TARGET_BOARD=rk356x
#OUTPUT_DIR=output/rockchip_rk356x_recovery
#CONFIG=rockchip_rk356x_recovery_defconfig
```

编译完成如下图所示:

```
Image 2 (ramdisk)
Description: unavailable
Created: Mon Jan 1 22:52:09 2024
Type: RAMDisk Image
Compression: uncompressed
Data Size: 7479307 Bytes = 7304.01 KiB = 7.13 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff02
Entry Point: unavailable
Hash algo: sha256
Hash value: 22e0ed02b9a9c1f7573c07c449f4a094788b1b8de8ab4dafc8521abc71cd244c
Image 3 (resource)
Description: unavailable
Created: Mon Jan 1 22:52:09 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
Init Ramdisk: ramdisk
FDT: fdt
done.
you take 0:06.68 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步：编译 buildroot

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```



```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第三个 rootfs，点击回车会进入到文件系统镜像选择界面，如下图所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22

buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto
```

这里总共有五种镜像供编译选择，由于本小节要编译的是 buildroot，所以将光标移动到 buildroot 按下回车即可开始 buildroot 镜像的构建，构建过程如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
buildroot
processing option: buildroot
=====Start building rootfs(buildroot) to .rootfs=====
=====Start building buildroot rootfs =====
TARGET_BUILDROOT_CONFIG=rockchip_rk3568
=====
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
=====
```

构建完成如下所示：

```
make: Leaving directory '/home/topeet/Linux/rk356x_linux/buildroot'
2024-01-01T22:55:13 >>> Finalizing target directory
2024-01-01T22:55:14 >>> Sanitizing RPATH in target tree
2024-01-01T22:55:24 >>> Copying overlay board/rockchip/common/base
2024-01-01T22:55:24 >>> Copying overlay board/rockchip/rk356x/fs-overlay/
2024-01-01T22:55:24 >>> Copying overlay board/rockchip/common/wifi
2024-01-01T22:55:24 >>> Copying overlay board/rockchip/rk356x/fs-overlay-system/
2024-01-01T22:55:25 >>> Executing post-build script build/post.sh
2024-01-01T22:55:25 >>> Generating root filesystem image rootfs.cpio
2024-01-01T22:56:15 >>> Generating root filesystem image rootfs.ext2
2024-01-01T22:56:18 >>> Generating root filesystem image rootfs.squashfs
2024-01-01T22:56:22 >>> Generating root filesystem image rootfs.tar
Done in 1min 14s
log saved on /home/topeet/Linux/rk356x_linux/br.log. pack buildroot image at: /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/images/rootfs.ext4
you take 1:14.90 to build buildroot
Running build_buildroot succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后,将打包要用到的所有固件移动到 rockdev 目录下,然后打包为完整的 update.img 镜像。

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面,进入之后如下所示:

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第六个 firmware, 点击回车就会运行相应的脚本将打包要用到的所有固件移动到 rockdev 目录下, 具体打印如下所示:

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
firmware
processing option: firmware
/usr/bin/fakeroot
Source buildroot/build/envsetup.sh
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
=====
make: Entering directory '/home/topeet/Linux/rk356x_linux/buildroot'
GEN      /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/Makefile
/home/topeet/Linux/rk356x_linux/buildroot/build/defconfig_hook.py -m /home/topeet/Linux/rk356x_linux/buildroot/configs/rockchip_rk3568_defconfig /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/.rockchipconfig
```

打包成 update.img 所需要的镜像就已经拷贝到了 rockdev 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    userdata.img
MiniloaderAll.bin  oem.img      recovery.img    uboot.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

然后进行最后的打包步骤，在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第七个 updateimg，点击回车 rockdev 目录下全部固件打包成为一个整体的 update.img 镜像，具体打印如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```



```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ff9e00,userspace=0x3fff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

命令运行完成之后，在 rockdev 目录下就生成了打包而来的 update.img 镜像，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img          misc.img  parameter.txt  rootfs.img  update.img
MiniLoaderAll.bin oem.img   recovery.img   uboot.img   userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.4.1.2 命令行

本小节单独编译镜像的顺序如下所示：

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 buildroot

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 buildroot 镜像 update.img。

第一步：编译 uboot

在 linux 源码目录下输入以下命令编译 U-Boot

```
./build.sh uboot
```



```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh uboot
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
```

Uboot 编译完成如下图所示:

```
*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIAL.INI
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.INI

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 23:23:55 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录, 如下图所示:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api          config.mk    Kbuild      scripts      u-boot.dtb
arch         configs      Kconfig     snapshot.commit  u-boot-dtb.bin
bl31_0x00040000.bin  disk        lib         spl          uboot.img
bl31_0x00069000.bin  doc         Licenses   System.map   u-boot.lds
bl31_0x0006b000.bin  Documentation MAINTAINERS tee.bin      u-boot.map
bl31_0xfdcc1000.bin  drivers     Makefile   test         u-boot-nodtb.bin
bl31_0xfdcce000.bin  dts         make.sh    tools        u-boot.srec
bl31_0xfdcdd000.bin  env         net        tpl          u-boot.sym
bl31.elf       examples    post       u-boot      u-boot
board         fit         PREUPLOAD.cfg  u-boot.bin  u-boot.cfg
cmd          fs          README     u-boot.cfg  u-boot.cfg.configs
common       include     rk356x_spl_loader_v1.13.112.bin
```

第二步: 编译 kernel

在 linux 源码目录下输入以下命令编译 Kernel

```
./build.sh kernel
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh kernel
kernel
processing option: kernel
=====Start building kernel=====
TARGET_KERNEL_ARCH =
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
TARGET_KERNEL_DTS   =rk3568-evb1-ddr4-v10-linux
TARGET_KERNEL_CONFIG_FRAGMENT =
=====
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
#
# configuration written to .config
#
```

内核编译完如下图所示:

```

Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0fd3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar           build.config.gki_kprobes         ini          OWNERS
arch        build.config.gki_kprobes.aarch64  init        README
block      build.config.gki_kprobes.x86_64   ipc         resource.img
boot.img    build.config.gki.x86_64          Kbuild      samples
boot.its    build.config.x86_64              Kconfig     scripts
build.config.aarch64    built-in.a                      kern        securit
build.config.allmodconfig  certs                          kernel      security
build.config.allmodconfig.aarch64  COPYING                      kernel.img  sound
build.config.allmodconfig.arm      CREDITS                      lib         System.map
build.config.allmodconfig.x86_64   crypto                       LICENSES    tools
build.config.arm                d                             logo.bmp    usr
build.config.common             Documentation                 logo_kernel.bmp  virt
build.config.gki                dr                             MAINTAINERS  vmlinux
build.config.gki.aarch64        dri                           Makefile     vmlinux.o
build.config.gki-debug.aarch64  drive                        mm           zboot.img
build.config.gki-debug.x86_64   drivers
build.config.gki_kasan          firmware
build.config.gki_kasan.aarch64  fs
topeet@ubuntu:~/Linux/rk356x_linux$

```

第三步：编译 Recovery

在 linux 源码目录下输入以下命令编译 Recovery

```
./build.sh recovery
```

```

topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh recovery
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET BOARD=rk356x

```


编译完成如下图所示:

```
Image 3 (resource)
Description: unavailable
Created:      Mon Jan 1 23:25:39 2024
Type:        Multi-File Image
Compression: uncompressed
Data Size:   176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo:   sha256
Hash value:  e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description:  unavailable
Kernel:      kernel
Init Ramdisk: ramdisk
FDT:         fdt
done.
you take 0:06.85 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步: 编译 buildroot

进入源码根目录执行以下命令自动完成 Rootfs 的编译及打包:

```
./build.sh buildroot
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh buildroot
buildroot
processing option: buildroot
=====Start building rootfs(buildroot) to .rootfs=====
=====Start building buildroot rootfs =====
TARGET_BUILDROOT_CONFIG=rockchip_rk3568
=====
Top of tree: /home/topeet/Linux/rk356x_linux
=====

#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
```

编译完成如下图所示:

```
make: Leaving directory '/home/topeet/Linux/rk356x_linux/buildroot'
2024-01-01T23:26:23 >>> Finalizing target directory
2024-01-01T23:26:24 >>> Sanitizing RPATH in target tree
2024-01-01T23:26:34 >>> Copying overlay board/rockchip/common/base
2024-01-01T23:26:34 >>> Copying overlay board/rockchip/rk356x/fs-overlay/
2024-01-01T23:26:34 >>> Copying overlay board/rockchip/common/wifi
2024-01-01T23:26:34 >>> Copying overlay board/rockchip/rk356x/fs-overlay-system/
2024-01-01T23:26:35 >>> Executing post-build script build/post.sh
2024-01-01T23:26:35 >>> Generating root filesystem image rootfs.cpio
2024-01-01T23:27:25 >>> Generating root filesystem image rootfs.ext2
2024-01-01T23:27:28 >>> Generating root filesystem image rootfs.squashfs
2024-01-01T23:27:32 >>> Generating root filesystem image rootfs.tar
Done in 1min 14s
log saved on /home/topeet/Linux/rk356x_linux/br.log. pack buildroot image at: /home/topeet/Linux/rk356x_linux
/buildroot/output/rockchip_rk3568/images/rootfs.ext4
you take 1:14.51 to build buildroot
Running build_buildroot succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第五步: 打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后, 进入工程目录根目录执行以下命令自动完成所有固件打包到 rockdev 目录下。

输入以下命令打包所有启动系统需要的镜像。进入 rockdev 目录下可以看到打包的单独镜像。

```
./build.sh firmware
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh firmware
firmware
processing option: firmware
/usr/bin/fakeroot
Source buildroot/build/envsetup.sh
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
```

输入以下命令，打包 update.img 镜像。

```
./build.sh updateimg
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh updateimg
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ffa000,userspace=0x3ff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img  update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img   userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```


4.4.2 全自动编译

4.4.2.1 图形化界面

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后选择第五个 all，就会进入到文件系统类型选择页面，如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile update.img image | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22  
  
buildroot  
debian10  
debian11  
ubuntu20  
ubuntu22  
yocto
```

由于本小节全自动编译的是 buildroot 系统，所以这里选择 buildroot，然后回车，等待编译完成即可，脚本会自动编译 uboot kernel recovery buildroot 并打包为一个完整的 update.img 镜像，编译完成之后如下所示：

```
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
Running build_buildroot_all succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

在 rockdev 目录下就生成了打包完成的 update.img 镜像如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.4.2.2 命令行

进入源码根目录执行以下命令自动完成所有的编译。如果想自动全编译 buildroot 文件系统，输入以下命令：

```
./build.sh buildroot_update
```

输入上面的命令后，会自动编译 uboot kernel recovery buildroot 打包。打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.5 编译 Ubuntu

本小节将介绍 ubuntu 镜像的编译流程，分为单独编译和完整全自动编译。

4.5.1 单独编译

4.5.1.1 图形化界面

本小节单独编译镜像的顺序如下所示：

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 ubuntu

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 ubuntu 镜像 update.img。

第一步：编译 uboot

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

```

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

```

光标默认就在 uboot，所以直接回车即可开始 uboot 的编译，编译过程如下所示：

```

topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
# configuration written to .config
#

```

Uboot 编译完成如下图所示：

```

*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIALL.ini
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.ini

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 21:17:38 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

编译完成后生成 uboot.img 到 u-boot 目录，如下图所示：

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api                config.mk          Kbuild             scripts            u-boot.dtb
arch               configs            Kconfig            snapshot.commit   u-boot-dtb.bin
bl31_0x00040000.bin disk              lib                 spl                uboot.img
bl31_0x00069000.bin doc               licenses            System.map         u-boot.lds
bl31_0x0006b000.bin Documentation    MAINTAINERS         tee.bin            u-boot.map
bl31_0xfdcc1000.bin drivers          Makefile            test               u-boot-nodtb.bin
bl31_0xfdcce000.bin dts              make.sh             tools              u-boot.srec
bl31_0xfdcdd000.bin env              net                 tpl                u-boot.sym
bl31.elf           examples          post                u-boot            u-boot.bin
board              fit               PREUPLOAD.cfg      u-boot.bin        u-boot.cfg
cmd                fs                README              u-boot.cfg        u-boot.cfg.configs
common             include           rk356x_spl_loader_v1.13.112.bin
topeet@ubuntu:~/Linux/rk356x_linux$

```

第二步：编译 kernel

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```

topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh

```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
Choose an option
```

```
uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第二个 kernel，点击回车即可开始 kernel 内核的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
Choose an option
```

```
uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
kernel
```

```
processing option: kernel
```

```
=====Start building kernel=====
```

```
TARGET_KERNEL_ARCH =
```

```
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
```

```
TARGET_KERNEL_DTS =rk3568-evb1-ddr4-v10-linux
```

```
TARGET_KERNEL_CONFIG_FRAGMENT =
```

```
=====
```

```
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.
```

```
1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
```

```
#
```

```
# configuration written to .config
```

```
#
```

```
scripts/kconfig/conf --syncconfig Kconfig
```

内核编译完如下图所示：


```

Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdcce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar            build.config.gki_kprobes         ini          OWNERS
arch          build.config.gki_kprobes.aarch64 init         README
block         build.config.gki_kprobes.x86_64  ipc          resource.img
boot.img      build.config.gki.x86_64          Kbuild      samples
boot.its      build.config.x86_64             Kconfig     scripts
build.config.aarch64    built-in.a                      kern         securit
build.config.allmodconfig  certs                           kernel      security
build.config.allmodconfig.aarch64  COPYING                       kernel.img  sound
build.config.allmodconfig.arm      CREDITS                       lib         System.map
build.config.allmodconfig.x86_64    crypto                        LICENSES    tools
build.config.arm              d                             logo.bmp    usr
build.config.common            Documentation                 logo_kernel.bmp  virt
build.config.gki              dr                             MAINTAINERS     vmlinux
build.config.gki.aarch64      dri                           Makefile        vmlinux.o
build.config.gki-debug.aarch64  drive                         mm             zboot.img
build.config.gki-debug.x86_64   drivers
build.config.gki_kasan         firmware
build.config.gki_kasan.aarch64  fs
topeet@ubuntu:~/Linux/rk356x_linux$

```

第三步：编译 Recovery

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```

topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh

```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第四个 recovery，点击回车即可开始 recovery 的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk356x
#OUTPUT_DIR=output/rockchip_rk356x_recovery
#CONFIG=rockchip_rk356x_recovery_defconfig
```

编译完成如下图所示：

```
Entry Point: unavailable
Hash algo: sha256
Hash value: 22e0ed02b9a9c1f7573c07c449f4a094788b1b8de8ab4dafc8521abc71cd244c
Image 3 (resource)
Description: unavailable
Created: Mon Jan 1 22:52:09 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
Init Ramdisk: ramdisk
FDT: fdt
done.
you take 0:06.68 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步：构建 Ubuntu

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后将光标移动到第三个 rootfs，点击回车会进入到文件系统镜像选择界面，如下图所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22  
  
buildroot  
debian10  
debian11  
ubuntu20  
ubuntu22  
yocto
```

这里总共有六种镜像供编译选择，由于本小节要编译的是 ubuntu，所以将光标移动到第 4 个 ubuntu20 或者第 5 个 ubuntu22，大家根据自己的需求选择合适的镜像即可。按下回车即可开始 ubuntu 镜像的构建，构建过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22  
  
buildroot  
debian10  
debian11  
ubuntu20  
ubuntu22  
yocto
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
ubuntu20
processing option: ubuntu20
=====Start building rootfs(ubuntu20) to .rootfs=====
=====Start building ubuntu20=====
Building for ubuntu20
Extract image
[sudo] password for topeet:
```

需要注意，由于 ubuntu 镜像的构建中的解压步骤需要 root 权限，所以需要输入一次 root 用户的密码，构建完成如下所示：

```
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Copying files into the device: done
Writing superblocks and filesystem accounting information: done

Rootfs Image: rootfs.img
Running build_ubuntu20 succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

会在 ubuntu 目录下生成名为 rootfs.img 的文件系统烧写镜像，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls ubuntu/
binary      mk-rootfs.sh  rootfs.img      ubuntu-jammy-arm64.tar.xz
mk-image.sh post-build.sh ubuntu-focal-arm64.tar.xz
topeet@ubuntu:~/Linux/rk356x_linux$
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，将打包要用到的所有固件移动到 rockdev 目录下，然后打包为完整的 update.img 镜像。

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```


然后将光标移动到第六个 firmware，点击回车就会运行相应的脚本将打包要用到的所有固件移动到 rockdev 目录下，具体打印如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
firmware
processing option: firmware
/usr/bin/fakeroot
Source buildroot/build/envsetup.sh
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
=====
make: Entering directory '/home/topeet/Linux/rk356x_linux/buildroot'
GEN /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/Makefile
/home/topeet/Linux/rk356x_linux/buildroot/build/defconfig_hook.py -m /home/topeet/Linux/rk356x_linux/buildroot/configs/rockchip_rk3568_defconfig /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/.rockchipconfig
```

打包成 update.img 所需要的镜像就已经拷贝到了 rockdev 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img          misc.img  parameter.txt  rootfs.img  userdata.img
MiniLoaderAll.bin oem.img  recovery.img   uboot.img
```

然后进行最后的打包步骤，在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第七个 updateimg, 点击回车 rockdev 目录下全部固件打包成为一个整体的 update.img 镜像, 具体打印如下所示:

```
iTOP-RK3568 building script, http://www.topeet.com

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ff9e00,userspace=0x3ff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

命令运行完成之后, 在 rockdev 目录下就生成了打包而来的 update.img 镜像, 如下图所示:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.5.1.2 命令行

本小节单独编译镜像的顺序如下所示:

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 ubuntu

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 ubuntu 镜像 update.img。

第一步：编译 uboot

在 linux 源码目录下输入以下命令编译 U-Boot

```
./build.sh uboot
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh uboot
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
```

Uboot 编译完成如下图所示：

```
*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIALL.ini
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.ini

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 23:23:55 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api                config.mk          Kbuild            scripts            u-boot.dtb
arch               configs            Kconfig           snapshot.commit   u-boot-dtb.bin
bl31_0x00040000.bin disk              lib               spl                uboot.img
bl31_0x00069000.bin doc               Licenses          System.map         u-boot.lds
bl31_0x0006b000.bin Documentation    MAINTAINERS       tee.bin            u-boot.map
bl31_0xfdcc1000.bin drivers          Makefile          test               u-boot-nodtb.bin
bl31_0xfdcce000.bin dts              make.sh           tools              u-boot.srec
bl31_0xfdcdd000.bin env              net               tpl                u-boot.sym
bl31.elf           examples          post              u-boot            u-boot.bin
board             fit              PREUPLOAD.cfg    u-boot.bin        u-boot.cfg
cmd              fs               README            u-boot.cfg        u-boot.cfg.configs
common           include          rk356x_spl_loader_v1.13.112.bin
```

第二步：编译 kernel

在 linux 源码目录下输入以下命令编译 Kernel

```
./build.sh kernel
```



```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh kernel
kernel
processing option: kernel
=====Start building kernel=====
TARGET_KERNEL_ARCH =
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
TARGET_KERNEL_DTS =rk3568-evb1-ddr4-v10-linux
TARGET_KERNEL_CONFIG_FRAGMENT =
=====
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
#
# configuration written to .config
#
```

内核编译完如下图所示：

```
Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdcce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar           build.config.gki_kprobes         ini          OWNERS
arch         build.config.gki_kprobes.aarch64  init        README
block       build.config.gki_kprobes.x86_64   ipc         resource.img
boot.img     build.config.gki.x86_64          Kbuild      samples
boot.its     build.config.x86_64              Kconfig     scripts
build.config.aarch64    built-in.a                      kern        security
build.config.allmodconfig  certs                          kernel      sound
build.config.allmodconfig.aarch64  COPYING                      kernel.img  System.map
build.config.allmodconfig.arm      CREDITS                      lib         tools
build.config.allmodconfig.x86_64   crypto                       LICENSES   usr
build.config.arm              d                             logo.bmp   virt
build.config.common            Documentation                 logo_kernel.bmp  vmlinux
build.config.gki              dr                             MAINTAINERS  vmlinux.o
build.config.gki.aarch64      dri                           Makefile     zboot.img
build.config.gki-debug.aarch64  drive                        modules.builtin
build.config.gki-debug.x86_64   drivers                      modules.order
build.config.gki_kasan         firmware                     Module.symvers
build.config.gki_kasan.aarch64   fs
```

第三步：编译 Recovery

在 linux 源码目录下输入以下命令编译 Recovery


```
./build.sh recovery
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh recovery
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk356x
```

编译完成如下图所示:

```
Image 3 (resource)
Description: unavailable
Created: Mon Jan 1 23:25:39 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
Init Ramdisk: ramdisk
FDT: fdt
done.
you take 0:06.85 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步: 编译 ubuntu

由于 ubuntu 有 ubuntu20 和 ubuntu22 两个版本, 所以提供了两个命令, 分别对应 ubuntu20 和 ubuntu22:

```
./build.sh ubuntu20
```

```
./build.sh ubuntu22
```

这里以 ubuntu20 为例进行编译, 编译过程如下所示:

```
topeet@topeet:~/Linux/3588s-linux$ ./build.sh ubuntu20
ubuntu20
processing option: ubuntu20
=====Start building rootfs(ubuntu20) to .rootfs=====
=====Start building ubuntu20=====
Building for ubuntu20
Deleted binary directory.
Extract image
```

需要注意, 由于 ubuntu 镜像的构建中的解压步骤需要 root 权限, 所以需要输入一次 root 用户的密码, 编译完成如下图所示:

```
Adding build-info to /etc/os-release...
Fixing up /etc/fstab...
Fixing up rootfs type: auto
Adding dirs and links...
0+0 records in
0+0 records out
0 bytes copied, 2.762e-05 s, 0.0 kB/s
mke2fs 1.46.5 (30-Dec-2021)
Discarding device blocks: done
Creating filesystem with 1298432 4k blocks and 325120 inodes
Filesystem UUID: 67dfd0bb-5224-42a8-8964-c568bd810def
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Copying files into the device: done
Writing superblocks and filesystem accounting information: done

Rootfs Image: rootfs.img
Running build_ubuntu20 succeeded.
Running build_rootfs succeeded.
topeet@topeet:~/Linux/3588s-linux$
```

会在 ubuntu 目录下生成名为 rootfs.img 的文件系统烧写镜像，如下图所示：

```
topeet@topeet:~/Linux/3588s-linux$ ls ubuntu/
binary      mk-rootfs.sh  rootfs.img      ubuntu-jammy-arm64.tar.xz
mk-image.sh post-build.sh ubuntu-focal-arm64.tar.xz
topeet@topeet:~/Linux/3588s-linux$
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，进入工程目录根目录执行以下命令自动完成所有固件打包到 rockdev 目录下。

输入以下命令打包所有启动系统需要的镜像。进入 rockdev 目录下可以看到打包的单独镜像。

```
./build.sh firmware
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh firmware
firmware
processing option: firmware
/usr/bin/fakeroot
Source buildroot/build/envsetup.sh
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
```

输入以下命令，打包 update.img 镜像。

```
./build.sh updateimg
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh updateimg
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ffa000,userspace=0x3ff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.5.2 全自动编译

4.5.2.1 图形化界面

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

Choose an option

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

然后选择第五个 all，就会进入到文件系统类型选择页面，如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

Choose an option

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

iTOP-RK3568 building script, <http://www.topeet.com>

Compile update.img image | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22

Choose an option

buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto

由于本小节全自动编译的是 ubuntu 系统，所以这里可以根据大家想编译的 ubuntu 版本选择第四个 ubuntu20 或者 ubuntu22，然后回车，等待编译完成即可，脚本会自动编译 uboot kernel recovery ubuntu 并打包为一个完整的 update.img 镜像，编译完成之后如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

Compile update.img image | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22

Choose an option

buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto


```
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
Running build_buildroot_all succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

在 rockdev 目录下就生成了打包完成的 update.img 镜像如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.5.2.2 命令行

进入源码根目录执行以下命令自动完成所有的编译。如果想自动全编译 ubuntu 文件系统，输入以下命令：

```
./build.sh ubuntu20_update
./build.sh ubuntu22_update
```

上面两个命令分别对应 ubuntu20 版本和 ubuntu22 版本，输入上面的命令后，会自动编译 uboot kernel recovery ubuntu 并打包。打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.6 编译 Debian

本小节将介绍 Debian 镜像的编译流程，分为单独编译和完整全自动编译。

4.6.1 单独编译

4.6.1.1 图形化界面

本小节单独编译镜像的顺序如下所示：

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 debian

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 debian 镜像 update.img。

第一步：编译 uboot

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

光标默认就在 uboot，所以直接回车即可开始 uboot 的编译，编译过程如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh  
uboot  
processing option: uboot  
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-  
=====Start building uboot=====  
TARGET_UBOOT_CONFIG=rk3568  
=====make rk3568_defconfig -j40  
#  
# configuration written to .config  
#
```

Uboot 编译完成如下图所示：

```
*****boot_merger ver 1.2*****  
Info:Pack loader ok.  
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIAL.INI  
/home/topeet/Linux/rk356x_linux/u-boot  
  
Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready  
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready  
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.INI  
  
Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)  
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-  
Mon Jan 1 21:17:38 PST 2024  
Running build_uboot succeeded.  
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/  
api                config.mk          Kbuild             scripts            u-boot.dtb  
arch               configs            Kconfig            snapshot.commit    u-boot-dtb.bin  
bl31_0x00040000.bin disk              lib                spl                u-boot.img  
bl31_0x00069000.bin doc               licenses           System.map         u-boot.lds  
bl31_0x0006b000.bin Documentation    MAINTAINERS        tee.bin           u-boot.map  
bl31_0xfdcc1000.bin drivers          Makefile           test              u-boot-nodtb.bin  
bl31_0xfdcc0000.bin dts              make.sh            tools             u-boot.srec  
bl31_0xfdc00000.bin env              net                tpl               u-boot.sym  
bl31.elf            examples          post               u-boot            u-boot.bin  
board               fit              PREUPLOADER.cfg    u-boot.cfg        u-boot.cfg.configs  
cmd                 fs               README             u-boot.cfg.configs  
common              include          rk356x_spl_loader_v1.13.112.bin  
topeet@ubuntu:~/Linux/rk356x_linux$
```

第二步：编译 kernel

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后将光标移动到第二个 kernel，点击回车即可开始 kernel 内核的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh  
kernel  
processing option: kernel  
=====Start building kernel=====  
TARGET_KERNEL_ARCH =  
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig  
TARGET_KERNEL_DTS =rk3568-evb1-ddr4-v10-linux  
TARGET_KERNEL_CONFIG_FRAGMENT =  
=====Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-#  
# configuration written to .config  
#  
scripts/kconfig/conf --syncconfig Kconfig
```

内核编译完如下图所示：

```

Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdcce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar            build.config.gki_kprobes         ini          OWNERS
arch         build.config.gki_kprobes.aarch64 init         README
block        build.config.gki_kprobes.x86_64 ipc          resource.img
boot.img     build.config.gki.x86_64          Kbuild      samples
boot.its     build.config.x86_64             Kconfig     scripts
build.config.aarch64    built-in.a    kern         securit
build.config.allmodconfig  certs        kernel       security
build.config.allmodconfig.aarch64  COPYING     kernel.img   sound
build.config.allmodconfig.arm      CREDITS     lib          System.map
build.config.allmodconfig.x86_64   crypto      LICENSES     tools
build.config.arm              d           logo.bmp     usr
build.config.common            Documentation logo_kernel.bmp virt
build.config.gki              dr           MAINTAINERS  vmlinux
build.config.gki.aarch64      dri          Makefile     vmlinux.o
build.config.gki-debug.aarch64  drive        mm           zboot.img
build.config.gki-debug.x86_64   drivers
build.config.gki_kasan         firmware
build.config.gki_kasan.aarch64  fs
topeet@ubuntu:~/Linux/rk356x_linux$

```

第三步：编译 Recovery

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```

topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh

```



```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第四个 recovery，点击回车即可开始 recovery 的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk356x
#OUTPUT_DIR=output/rockchip_rk356x_recovery
#CONFIG=rockchip_rk356x_recovery_defconfig
```

编译完成如下图所示：

```
Image 2 (ramdisk)
Description: unavailable
Created:    Mon Jan 1 22:52:09 2024
Type:      RAMDisk Image
Compression: uncompressed
Data Size: 7479307 Bytes = 7304.01 KiB = 7.13 MiB
Architecture: AArch64
OS:        Linux
Load Address: 0xffffffff02
Entry Point: unavailable
Hash algo:  sha256
Hash value:  22e0ed02b9a9c1f7573c07c449f4a094788b1b8de8ab4dafc8521abc71cd244c
Image 3 (resource)
Description: unavailable
Created:    Mon Jan 1 22:52:09 2024
Type:      Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo:  sha256
Hash value:  e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel:     kernel
Init Ramdisk: ramdisk
FDT:        fdt
done.
you take 0:06.68 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步：构建 Debian

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第三个 rootfs，点击回车会进入到文件系统镜像选择界面，如下图所示：

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

iTOP-RK3568 building script, <http://www.topeet.com>

```

Choose an option
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22

buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto

```

这里总共有五种镜像供编译选择，由于本小节要编译的是 debian，所以将光标移动到第 2 个 debian10 或者第 3 个 debian11，大家根据自己的需求选择合适的镜像即可。按下回车即可开始 ubuntu 镜像的构建，构建过程如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

```

Choose an option
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22

buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto

```

```

topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
debian10
processing option: debian10
=====Start building rootfs(debian10) to .rootfs=====
=====Start building debian10=====
Building for debian10
Extract image

```

需要注意，由于 debian 镜像的构建中的解压步骤需要 root 权限，所以需要输入一次 root 用户的密码，构建完成如下所示：

```

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Copying files into the device: done
Writing superblocks and filesystem accounting information: done

Rootfs Image: rootfs.img
Running build_debian10 succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

会在 debian 目录下生成名为 rootfs.img 的文件系统烧写镜像，如下图所示：

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls debian/
binary          linaro-buster-arm64.tar.xz  mk-rootfs.sh  rootfs.img
linaro-bullseye-arm64.tar.xz  mk-image.sh      post-build.sh
topeet@ubuntu:~/Linux/rk356x_linux$

```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，将打包要用到的所有固件移动到 rockdev 目录下，然后打包为完整的 update.img 镜像。

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后将光标移动到第六个 firmware，点击回车就会运行相应的脚本将打包要用到的所有固件移动到 rockdev 目录下，具体打印如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
firmware  
processing option: firmware  
/usr/bin/fakeroot  
Source buildroot/build/envsetup.sh  
Top of tree: /home/topeet/Linux/rk356x_linux  
=====  
#TARGET_BOARD=rk3568  
#OUTPUT_DIR=output/rockchip_rk3568  
#CONFIG=rockchip_rk3568_defconfig  
=====  
make: Entering directory '/home/topeet/Linux/rk356x_linux/buildroot'  
GEN /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/Makefile  
/home/topeet/Linux/rk356x_linux/buildroot/build/defconfig_hook.py -m /home/topeet/Linux/rk356x_linux/buildroo  
t/configs/rockchip_rk3568_defconfig /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/.rockchi  
pconfig
```

打包成 update.img 所需要的镜像就已经拷贝到了 rockdev 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/  
boot.img      misc.img      parameter.txt  rootfs.img    userdata.img  
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img  
topeet@ubuntu:~/Linux/rk356x_linux$
```

然后进行最后的打包步骤，在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```


iTOP-RK3568 building script, <http://www.topeet.com>

Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

Choose an option

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

然后将光标移动到第七个 updateimg，点击回车 rockdev 目录下全部固件打包成为一个整体的 update.img 镜像，具体打印如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

Choose an option

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ff9e00,userspace=0x3fff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

命令运行完成之后，在 rockdev 目录下就生成了打包而来的 update.img 镜像，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.6.1.2 命令行

本小节单独编译镜像的顺序如下所示：

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 debian

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 debian 镜像 update.img。

第一步：编译 uboot

在 linux 源码目录下输入以下命令编译 U-Boot

```
./build.sh uboot
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh uboot
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
```

Uboot 编译完成如下图所示:

```
*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIAL.INI
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.INI

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 23:23:55 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录, 如下图所示:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api                config.mk          Kbuild            scripts           u-boot.dtb
arch               configs            Kconfig           snapshot.commit  u-boot-dtb.bin
bl31_0x00040000.bin disk              lib               spl              uboot.img
bl31_0x00069000.bin doc               Licenses          System.map        u-boot.lds
bl31_0x0006b000.bin Documentation    MAINTAINERS       tee.bin          u-boot.map
bl31_0xfdcc1000.bin drivers          Makefile          test             u-boot-nodtb.bin
bl31_0xfdcce000.bin dts              make.sh           tools            u-boot.srec
bl31_0xfdcdd000.bin env              net               tpl              u-boot.sym
bl31.elf           examples          post              u-boot          u-boot.bin
board              fit               PREUPLOAD.cfg     u-boot.cfg       u-boot.cfg.configs
cmd                fs                README
common             include           rk356x_spl_loader_v1.13.112.bin
```

第二步: 编译 kernel

在 linux 源码目录下输入以下命令编译 Kernel

```
./build.sh kernel
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh kernel
kernel
processing option: kernel
=====Start building kernel=====
TARGET_KERNEL_ARCH =
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
TARGET_KERNEL_DTS   =rk3568-evb1-ddr4-v10-linux
TARGET_KERNEL_CONFIG_FRAGMENT =
=====
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
#
# configuration written to .config
#
```

内核编译完如下图所示:


```

Hash value: 7e03bdc8b85b1dc49d49d5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar            build.config.gki_kprobes         ini          OWNERS
arch          build.config.gki_kprobes.aarch64 init         README
block         build.config.gki_kprobes.x86_64  ipc          resource.img
boot.img      build.config.gki.x86_64          Kbuild      samples
boot.its      build.config.x86_64             Kconfig     scripts
build.config.aarch64    built-in.a                      kern         securit
build.config.allmodconfig  certs                          kernel      security
build.config.allmodconfig.aarch64  COPYING                      kernel.img  sound
build.config.allmodconfig.arm      CREDITS                      lib         System.map
build.config.allmodconfig.x86_64    crypto                       LICENSES    tools
build.config.arm              d                             logo.bmp    usr
build.config.common           Documentation                 logo_kernel.bmp  virt
build.config.gki              dr                             MAINTAINERS    vmlinux
build.config.gki.aarch64      dri                           Makefile       vmlinux.o
build.config.gki-debug.aarch64  drive                        mm            zboot.img
build.config.gki-debug.x86_64   drivers
build.config.gki_kasan         firmware
build.config.gki_kasan.aarch64  fs
topeet@ubuntu:~/Linux/rk356x_linux$

```

第三步：编译 Recovery

在 linux 源码目录下输入以下命令编译 Recovery

```
./build.sh recovery
```

```

topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh recovery
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk356x

```

编译完成如下图所示：


```
Image 3 (resource)
Description: unavailable
Created:    Mon Jan 1 23:25:39 2024
Type:      Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel:     kernel
Init Ramdisk: ramdisk
FDT:        fdt
done.
you take 0:06.85 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步：编译 debian

由于 ubuntu 有 debian10 和 debian11 两个版本，所以提供了两个命令，分别对应 debian10 和 debian11：

```
./build.sh debian10
```

```
./build.sh debian11
```

这里以 debian10 为例进行编译，编译过程如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh debian10
debian10
processing option: debian10
=====Start building rootfs(debian10) to .rootfs=====
=====Start building debian10=====
Deleted binary directory.
Building for debian10
Extract image
```

需要注意，由于 debian 镜像的构建中的解压步骤需要 root 权限，所以需要输入一次 root 用户的密码，编译完成如下图所示：

```
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Copying files into the device: done
Writing superblocks and filesystem accounting information: done

Rootfs Image: rootfs.img
Running build_debian10 succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

会在 ubuntu 目录下生成名为 rootfs.img 的文件系统烧写镜像，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls debian/
binary          linaro-buster-arm64.tar.xz  mk-rootfs.sh  rootfs.img
linaro-bullseye-arm64.tar.xz  mk-image.sh               post-build.sh
topeet@ubuntu:~/Linux/rk356x_linux$
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后,进入工程目录根目录执行以下命令自动完成所有固件打包到 rockdev 目录下。

输入以下命令打包所有启动系统需要的镜像。进入 rockdev 目录下可以看到打包的单独镜像。

```
./build.sh firmware
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh firmware
firmware
processing option: firmware
/usr/bin/fakeroot
Source buildroot/build/envsetup.sh
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
```

输入以下命令,打包 update.img 镜像。

```
./build.sh updateimg
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh updateimg
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniloaderAll.bin
bootloader,Add file: ./Image/MiniloaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/u-boot.img
u-boot,Add file: ./Image/u-boot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ffa000,userspace=0x3ff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

打包完成会在 rockdev 目录下生成 update.img, 如下图所示:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/  
boot.img      misc.img  parameter.txt  rootfs.img  update.img  
MiniLoaderAll.bin  oem.img  recovery.img  uboot.img  userdata.img  
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.6.2 全自动编译

4.6.2.1 图形化界面

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

然后选择第五个 all，就会进入到文件系统类型选择页面，如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

iTOP-RK3568 building script, <http://www.topeet.com>

Choose an option
Compile update.img image | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22

buildroot
debian10
debian11
ubuntu20
ubuntu22
vocto

由于本小节全自动编译的是 debian 系统，所以这里可以根据大家想编译的 ubuntu 版本选择第四个 debian10 或者 debian11，然后回车，等待编译完成即可，脚本会自动编译 uboot kernel recovery debian 并打包为一个完整的 update.img 镜像，编译完成之后如下所示：

```
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
Running build_buildroot_all succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

在 rockdev 目录下就生成了打包完成的 update.img 镜像如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.6.2.2 命令行

进入源码根目录执行以下命令自动完成所有的编译。如果想自动全编译 ubuntu 文件系统，输入以下命令：

```
./build.sh debian10_update
./build.sh debian11_update
```

上面两个命令分别对应 debian10 版本和 debian11 版本，输入上面的命令后，会自动编译 uboot kernel recovery ubuntu 并打包。打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

4.7 编译 Yocto

注意：编译系统时一定要使用普通权限编译。

由于编译 yocto 非常耗费时间，下载软件包可能需要连接外网进行下载，为了方便大家使用，迅为提供了编译好的 yocto 源码，yocto 源码在网盘资料“iTOP-3568 开发板\01_【iTOP-RK3568 开发板】基础资料【V1.7 版本】\05_iTOP-RK3568 开发板 Linux 源码\02_Yocto 系统源码”目录下。下载 yocto 压缩包，解压后替换 Linux 源码包里面的 yocto 文件夹。如果使用迅为提供的 yocto 源码，Linux 源码编译路径必须为/home/topeet/Linux。

4.7.1 全自动编译

我们进入源码根目录执行以下命令自动完成所有的编译。如果想自动全编译 yocto 文件系

统，输入以下命令：

```
export RK_ROOTFS_SYSTEM=yocto
./build.sh all
./build.sh firmware
./build.sh updateimg
```

输入上面的命令后，会自动编译 uboot kernel recovery yocto 打包。打包完成会在 rockdev 目录下生成 update.img。

4.7.2 单独编译

本小节我们以

单独编译 [uboot](#) ->单独编译 [kernel](#) ->单独编译 [recovery](#) ->单独编译 [yocto](#)，最后各个镜像编译完成，最终打包镜像 update.img 的方式来编译 yocto 镜像 update.img。

第一步：编译 uboot

在 linux 源码目录下输入以下命令编译 U-Boot

```
./build.sh uboot
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh uboot
processing option: uboot
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
grep: .config: No such file or directory
## make rk3568_defconfig -j24
  HOSTCC  scripts/basic/fixdep
  HOSTCC  scripts/kconfig/conf.o
  SHIPPED scripts/kconfig/zconf.tab.c
  SHIPPED scripts/kconfig/zconf.lex.c
  SHIPPED scripts/kconfig/zconf.hash.c
  HOSTCC  scripts/kconfig/zconf.tab.o
In file included from scripts/kconfig/zconf.tab.c:2468:
scripts/kconfig/confdata.c: In function 'conf_write':
```

Uboot 编译完成如下图所示：

```
Hash value: fcfda359c09a8b315de67f0f6a4db29122f97fadfa185d0d4a9aee69488cc76d
Default Configuration: 'conf'
Configuration 0 (conf)
Description: rk3568-evb
Kernel: unavailable
Firmware: atf-1
FDT: fdt
Loadables: uboot
            atf-2
            atf-3
            atf-4
            atf-5
            optee
*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIALL.ini
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.ini

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j24)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Wed Jul 19 17:01:17 CST 2023
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api          bl31_0xfdc00000.bin  configs      env          Kconfig      net          snapshot.commit  tpl          u-boot-dtb.bin  u-boot.syn
arch         bl31.elf             disk         examples    lib          post         spl             u-boot        u-boot-dtb.bin  u-boot.img
bl31_0x00040000.bin  board               doc          fit          licenses    PREUPLOAD.cfg  System.map      u-boot.bin     u-boot.lds     u-boot.map
bl31_0x00060000.bin  cnd                 Documentation  fs           MAINTAINERS  README         tee.bin        u-boot.cfg     u-boot.map     u-boot-nodtb.bin
bl31_0xfdc00000.bin  common              drivers      include      Makefile     rk356x_spl_loader_v1.13.112.bin  test          u-boot.cfg.configs  u-boot-nodtb.bin  u-boot.srec
bl31_0xfdc00000.bta  config.mk           dts          Kbuild      make.sh      scripts        tools          u-boot.dtb      u-boot.srec
```

第二步：编译 kernel

在 linux 源码目录下输入以下命令编译 Kernel

./build.sh kernel

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh kernel
processing option: kernel
=====Start building kernel=====
TARGET_ARCH          =arm64
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
TARGET_KERNEL_DTS     =rk3568-evb1-ddr4-v10-linux
TARGET_KERNEL_CONFIG_FRAGMENT =
=====
HOSTCC  scripts/basic/fixdep
HOSTCC  scripts/kconfig/conf.o
YACC    scripts/kconfig/zconf.tab.c
LEX     scripts/kconfig/zconf.lex.c
HOSTCC  scripts/kconfig/zconf.tab.o
HOSTLD  scripts/kconfig/conf
^[@#
# configuration written to .config
#
UPD     include/generated/uapi/linux/version.h
UPD     include/config/kernel.release
WRAP    arch/arm64/include/generated/uapi/asm/errno.h
WRAP    arch/arm64/include/generated/uapi/asm/ioctl.h
WRAP    arch/arm64/include/generated/uapi/asm/ioctls.h
```

内核编译完如下图所示：

```
Description: unavailable
Created: Wed Jul 19 17:10:30 2023
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: 2b6678a2eced4018ef9547552de04923bba3c4de664ca7f89f6c4081bb71a8d0
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
grep: exceeded PCRE's backtracking limit
grep: exceeded PCRE's backtracking limit
grep: exceeded PCRE's backtracking limit
grep: exceeded PCRE's backtracking limit
grep: exceeded PCRE's backtracking limit
grep: exceeded PCRE's backtracking limit
grep: exceeded PCRE's backtracking limit
grep: exceeded PCRE's backtracking limit
PLEASE CHECK BOARD GPIO POWER DOMAIN CONFIGURATION [!!!]
<<< ESPECIALLY at-Pi/Flash/Ethernet IO power domain >>> [!!!]
Check Node {pmu to domains} in the file: /home/topeet/Linux/rk356x_linux/kernel/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10-linux.dts
请再次确认板级的电源域配置! [!!!]
<<< 特别是MI-PI, FLASH, 以太网这几路IO电源的配置 >>> [!!!]
检查内核文件 /home/topeet/Linux/rk356x_linux/kernel/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10-linux.dts 的节点 {pmu to domains}
Running build kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后映像文件 kernel.img、resource.img 会打包成 boot.img 放到 kernel 目录。如下图所示

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki      certs                  ini                    Makefile              sound
ar            build.config.gki.aarch64  COPYING               init                  mm                    System.map
arch          build.config.gki-debug.aarch64  CREDITS               ipc                    modules.builtin      tools
block         build.config.gki-debug.x86_64  crypto                Kbuild                modules.order         usr
boot.img      build.config.gki_kasan.aarch64  d                      Kconfig               Module.symvers        vrt
boot.tts      build.config.gki_kasan.x86_64  Documentation         kern                    net                    vmlinux
build.config.aarch64  build.config.gki_kprobes      dr                      kernel                 OWNERS                vmlinux.o
build.config.allmodconfig  build.config.gki_kprobes.aarch64  dri                     kernel.img             README                 zboot.img
build.config.allmodconfig.aarch64  build.config.gki_kprobes.x86_64  drive                    LICENSES               samples
build.config.allmodconfig.arm  build.config.gki_kprobes.x86_64  drivers                  logo.bmp               scripts
build.config.allmodconfig.x86_64  build.config.gki.x86_64          firmware                 logo_kernel.bmp        securit
build.config.arm  build.config.x86_64           fs                       MAINTAINERS            security
build.config.common  built-in.a                     include                  logo_kernel.bmp        security
```

第三步：编译 Recovery

在 linux 源码目录下输入以下命令编译 Recovery

./build.sh recovery

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk356x
#OUTPUT_DIR=output/rockchip_rk356x_recovery
#CONFIG=rockchip_rk356x_recovery_defconfig
=====
make: Entering directory '/home/topeet/Linux/rk356x_linux/buildroot'
mkdir -p /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk356x_recovery/build/buildroot-config/ldialog
PKG_CONFIG_PATH="" make CC="/usr/bin/gcc" HOSTCC="/usr/bin/gcc" \
obj=/home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk356x_recovery/build/buildroot-config -C support/kconfig -f Makefile.
br conf
/usr/bin/gcc -D DEFAULT_SOURCE -D XOPEN_SOURCE=600 -DCURSES_LOC="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk356x_recovery/build/buildroot-config -DCONFIG_="" -MM *.c > /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk356x_recovery/build/buildroot-config/.depend 2>/dev/null || :
```

编译完成如下图所示：


```
Load Address: 0xffffffff02
Entry Point:  unavailable
Hash algo:    sha256
Hash value:   0dd7eba57d24065c12f6b6f3a36caac113443973cb5ec20c445eb55133fab47c
Image 3 (resource)
Description:  unavailable
Created:      Fri Jan 21 21:16:26 2022
Type:         Multi-File Image
Compression:  uncompressed
Data Size:    173056 Bytes = 169.00 KiB = 0.17 MiB
Hash algo:    sha256
Hash value:   f01b9bdd8e282c8008055487ddd8a2c4f858fb1ea15b25048d168e69e917e78d
Default Configuration: 'conf'
Configuration 0 (conf)
Description:  unavailable
Kernel:      kernel
Init Ramdisk: ramdisk
FDT:         fdt
done.
you take 28:26.08 to build recovery
Running build_recovery succeeded.
```

第四步：编译 yocto

选择 RK3568 搭配 DDR4 开发板的板级配置，输入以下命令：

```
./build.sh BoardConfig-rk3568-evb1-ddr4-v10.mk
```

进入源码根目录执行以下命令自动完成 yocto 的编译及打包：

```
./build.sh yocto
```

```
root@ubuntu:/home/topeet/Linux/rk356x_linux# ./build.sh yocto
processing option: yocto
=====Start building ramboot=====
TARGET_MACHINE=rockchip-rk3568-evb
=====

### Shell environment set up for builds. ###

You can now run 'bitbake <target>'

Common targets are:
  core-image-minimal
  core-image-sato
  meta-toolchain
  meta-ide-support

You can also run generated qemu images with a command like 'runqemu qemux86'

Other commonly useful commands are:
- 'devtool' and 'recipetool' handle common recipe tasks
- 'bitbake-layers' handles common layer tasks
- 'oe-pkgdata-util' handles common target package tasks
Loading cache: 100% | ETA: --:--:--
Loaded 0 entries from dependency cache.
Parsing recipes: 39% |##### | ETA: 0:00:47
```

Yocto 编译完如下图所示：


```
Build Configuration:
BB_VERSION           = "1.48.0"
BUILD_SYS            = "x86_64-linux"
NATIVELSDSTRING      = "universal"
TARGET_SYS           = "aarch64-poky-linux"
MACHINE              = "rockchip-rk3568-evb"
DISTRO               = "poky"
DISTRO_VERSION        = "3.2"
TUNE_FEATURES        = "aarch64 armv8a crc"
TARGET_FPU           = ""
meta-oe
meta-python
meta-qt5
meta-rust
meta-clang
meta-python2
meta-rockchip
meta
meta-poky
meta-yocto-bsp
meta-browser
= "HEAD:b9dcf17708014dec679307dc1247882dcd439ca"
= "HEAD:8d5672cc6ca327576a814d35dfb5d59ab24043cb"
= "HEAD:f56fd4ec6835d68c4e9f8a9630963acd34d172e5"
= "HEAD:1dfc3a905ad0fed7a3b300a51153365f9950fe7"
= "HEAD:27d2aebd478a608798a4f611d9bcbfcb4e6a35b"
= "HEAD:1f1a1a4e62b9b23d9c945009f547ca8e6a902d"
= "HEAD:5232b03e22a49c368fbf4d79e05519ad5e48db4a"
= "HEAD:42363319c7e4b60e1fd1338a2a21a42e70733e"

Initiating tasks: 100% [#####] Time: 0:00:0
State summary: Wanted 7 Found 0 Missed 7 Current 2111 (0% match, 99% complete)
NOTE: Executing Tasks
WARNING: chromium-ozone-wayland-06.0.4240.111-r0 do_package_qa: QA Issue: package contains desktop file with key 'MimeType' but does not inherit mime-xdg: chromium-ozone-wayland path '/work/armv8a-poky-lt
ux/chromium-ozone-wayland/06.0.4240.111-r0/packages-split/chromium-ozone-wayland/usr/share/applications/chromium.desktop' [mime-xdg]
NOTE: Tasks Summary: Attempted 5606 tasks of which 5586 didn't need to be rerun and all succeeded.

Summary: There was 1 WARNING message shown.
Running build_yocto succeeded.
Running build_rootfs succeeded.
root@ubuntu:/home/topeet/Linux/rk3568_linux#
root@ubuntu:/home/topeet/Linux/rk3568_linux#
```

编译之后会在 yocto 目录 build/latest 下生成 rootfs.img。如下图所示:

```
root@ubuntu:/home/topeet/Linux/rk3568_linux# cd yocto/build/latest
root@ubuntu:/home/topeet/Linux/rk3568_linux/yocto/build/latest# ls
boot.img          kernel.img
boot.img--4.19+git0+d47e486b19-r0-rockchip-rk3568-evb-20220124135545.bin  kernel.img-rockchip-rk3568-evb.bin
boot.img-rockchip-rk3568-evb.bin      modules--4.19+git0+d47e486b19-r0-rockchip-rk3568-evb-20220124135545.tgz
core-image-minimal.env                resource.img
core-image-minimal-generic-gptdisk.wks  resource.img-rockchip-rk3568-evb.bin
core-image-minimal-rockchip-rk3568-evb-20220124135545.rootfs.ext4      rk3568-evb1-ddr4-v10-linux--4.19+git0+d47e486b19-r0-rockchip-rk3568-evb-20220124135545.dtb
core-image-minimal-rockchip-rk3568-evb-20220124135545.rootfs.manifest  rk3568-evb1-ddr4-v10-linux-rockchip-rk3568-evb.dtb
core-image-minimal-rockchip-rk3568-evb-20220124135545.testdata.json    rk3568-evb1-ddr4-v10-linux-rockchip-rk3568-evb.dtb
core-image-minimal-rockchip-rk3568-evb-20220124135545.manifest         rootfs.img
core-image-minimal-rockchip-rk3568-evb-20220124135545.testdata.json    zboot.img
core-image-minimal-rockchip-rk3568-evb.wic                             zboot.img--4.19+git0+d47e486b19-r0-rockchip-rk3568-evb-20220124135545.bin
Image                             zboot.img-rockchip-rk3568-evb.bin
Image--4.19+git0+d47e486b19-r0-rockchip-rk3568-evb-20220124135545.bin
Image-rockchip-rk3568-evb.bin
root@ubuntu:/home/topeet/Linux/rk3568_linux/yocto/build/latest#
```

第五步：打包镜像

然后输入以下命令打包镜像

```
./build.sh firmware
```

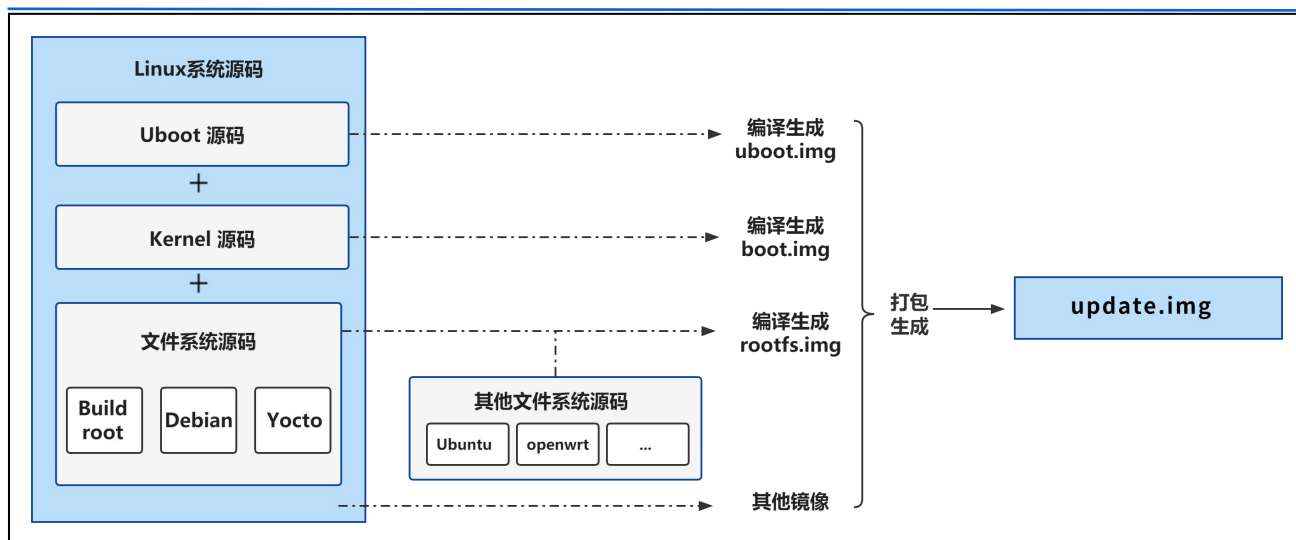
```
./build.sh updateimg
```

打包完成会在 rockdev 目录下生成镜像，如下图所示:

```
root@ubuntu:/home/topeet/Linux/rk3568_linux# cd rockdev/
root@ubuntu:/home/topeet/Linux/rk3568_linux/rockdev# ls
boot.img          misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin oem.img      recovery.img   uboot.img     userdata.img
root@ubuntu:/home/topeet/Linux/rk3568_linux/rockdev#
```

4.8 编译其他系统

因为瑞芯微官方的 SDK 包里只包含了 buildroot 系统、debian 系统和 yocto 系统源码，瑞芯微官方 SDK 包的 build.sh 编译脚本只能编译此三种系统。根据上面三小节内容可知除了文件系统之外的内核和 uboot 等源码使用的是相同的，所以想要编译其他 Linux 系统，如 Ubuntu，openwrt 等其他系统（可以参考《【北京迅为】itop-3568 开发板文件系统构建手册》手册进行系统构建），内核和 uboot 等的编译可以参考 buildroot 系统的单独编译章节进行编译。由此可以总结为下图内容，如下图所示:



若想在迅为提供的 Ubuntu 系统基础上进行修改可参考本手册 9.1 章节进行自己 Ubuntu 系统的定制。

其他 Linux 系统定制构建完，最后可根据本手册 9.2 小节打包生成整体 update.img 固件包。

第 5 章 编译 Linux-5.10 内核源码包

5.1 获取 Linux 源码包

编译环境说明：

本手册使用的是迅为提供的编译环境 ubuntu20.04，在网盘资料“**iTOP-3568 开发板\02_【iTOP-RK3568 开发板】开发资料\02_iTOP-3568 开发板搭建好编译环境的 Ubuntu 虚拟机镜像**”路径下。这里的 ubuntu 默认开发环境已经搭建好。

因 Ubuntu 版本众多，无法将所有版本 Ubuntu 全部测试通过，如使用其他开发环境，在开发过程中遇到问题，需自行研究解决。

在编译源码之前，首先要阅读并且操作《**【北京迅为】itop-3568 开发板快速使用编译环境 Ubuntu18.04 v1.0.doc**》手册，才可进行编译工作。

Linux 源码网盘下载路径：“**iTOP-3568 开发板\01_【iTOP-RK3568 开发板】基础资料\09_iTOP-RK3568 开发板 Linux-5.10 源码\01_Linux 系统源码**”

源码名称说明：源码会逐渐更新，发布的源码日期会有所不同，具体以网盘中实际名称为准。

1 从网盘中下载 Linux 源码包，如下图所示：

 01_Linux系统源码	2022/6/20 12:18	文件夹
--	-----------------	-----

2 首先下载 **01_Linux 系统源码** 到 windows 电脑上，如下所示：

 rk356x_linux_20220621.tar.zip.001	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.002	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.003	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.004	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.005	2022/6/21 23:18	360压缩 分卷文件	4,194,304...
 rk356x_linux_20220621.tar.zip.006	2022/6/21 23:18	360压缩 分卷文件	4,098,047...

下载后得到的源码如果是分卷压缩包，大家要完整全部下载下来，然后点击第一个分卷压缩包，鼠标点击右键，选择点击“解压到 XXXXXXXX”（XX...为名称的简写），然后等待解压完毕。解压完，会生成一个 XXXXXXXX.tar.gz（XX...为名称的简写）名字的压缩包。然后将 XXXXXXXX.tar.gz（XX...为名称的简写）拷贝到 ubuntu 中解压。

3 我们将 rk356x_linux_XXXXXXX.tar.gz（XX...为名称的简写）拷贝到迅为提供的 Ubuntu18.04 中，使用命令“tar -vxf rk356x_linux_XXXXXXX.tar.gz（XX...为名称的简写）”解压压缩包。解压后会生成一个 **rk3568_linux_5.10** 文件夹，如下图所示：

```
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$ ls  
app          build.sh  device  external  Makefile  prebuilts  rkbin      rockdev  u-boot  
buildroot    debian   docs    kernel    output    README.md  rkflash.sh tools    yocto  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$
```

5.2 SDK 包说明

5.2.1 SDK 包源码简要

进入源码目录有下图的文件目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$ ls  
app          build.sh  device  external  Makefile  prebuilts  rkbin      rockdev  u-boot  
buildroot    debian   docs    kernel    output    README.md  rkflash.sh tools    yocto  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$  
topeet@ubuntu:~/Linux/rk3568_linux_5.10$
```

从上图可以看出有很多文件夹，我们讲解一下每个目录或者文件的作用：

app：存放上层应用程序的目录，主要是 qcamera/qfm/qplayer/settings 等一些应用程序。

buildroot：基于 buildroot (2018.02-rc3) 开发的根文件系统。

build.sh：编译用的脚本，使用方法后面会教。

debian：基于 debian 10 开发的根文件系统。

device/rockchip：存放每个平台的一些编译和打包固件的脚本和预备文件。

docs：存放 RK 开发指导文件、平台支持列表、工具使用文档、Linux 开发指南等。

envsetup.sh：要修改文件系统时候要设置的环境脚本。

external：存放相关的库，包括音频，视频等。

kernel：kernel 源码。

Makefile：整个 SDK 包编译的 Makefile。

output：存放编译输出固件。

prebuilts：存放交叉编译工具链。

rkbin：存放固件和工具。

rkflash.sh：linux 下的系统烧录脚本。

rockdev：存放编译输出固件的目录(整个 SDK 包编译完成后就会创建)。

tools：存放固件和工具的目录。

u-boot：U-boot 源码目录。

yocto：基于 yocto gatesgarth 3.2 开发的根文件系统。

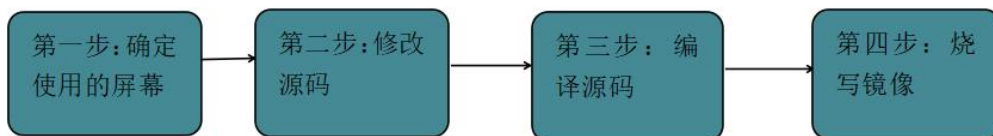
5.2.2 模块代码目录相关说明

什么叫做模块代码？一个完整的 SDK 包除了 kernel、u-boot、buildroot、Debian 和 yocto 之外，还需要提供上层的第三方库和 APP，第三方库和 APP 合起来叫做模块代码。比如：可以做监控摄像头可以参考 common_algorithm 代码。一个 SDK 包包含了很多产品的应用代码。

部分模块代码目录路径	含义
external/linux-rga	Raster Graphic Acceleration(RGA2)
external/recovery	recovery 和 Rockchip 升级代码
external/rkwifibt	Wi-Fi 和 BT
external/rk_pcba_test	PCBA 测试代码
external/mpp	编解码代码
external/camera_engine_rkaiq	图像处理算法模块
external/rknpu2	npu 驱动
external/common_algorithm	音视频通用算法库
external/rknn-toolkit2	模型转换、推理和性能评估的开发套件
app/qcamera	Buildroot 桌面相机应用源码
app/qfm	Buildroot 桌面文件管理应用源码
app/qsetting	Buildroot 桌面设置应用源码
app/qplayer	Buildroot 桌面多功能播放器源码

5.3 设置屏幕配置

在源码编译之前首先要确定自己想要使用的屏幕，然后修改源码，编译镜像，烧写镜像。如下图所示：



第一步：确定要使用的屏幕种类，屏幕种类选择如下所示：

iTOP-3568 开发板支持以下种类屏幕：

- 迅为 LVDS 7 寸屏幕
- 迅为 LVDS 10.1 寸 1024*600 屏幕

- 迅为 LVDS 10.1 寸 1280*800 屏幕
- 迅为 MIPI 7 寸屏幕
- 标准 HDMI 屏幕（通过 HDMI 线连接）

第二步：修改源码的设备树

修改 Linux/rk3568_linux_5.10/kernel/arch/arm64/boot/dts/rockchip 目录下的 topeet-screen-lcds.dts 文件。打开如下图所示：

```
//#define LCD_TYPE_MIPI //in vp 1
//#define LCD_TYPE_LVDS_10_1_1024X600 //in vp 2
//#define LCD_TYPE_LVDS_10_1_1280X800_gt9271 //in vp 2
#define LCD_TYPE_LVDS_7_0 //in vp 2
//#define LCD_TYPE_EDP_VGA //in vp 0
//#define LCD_TYPE_HDMI_VP0 //hdmi in vp 0
//#define LCD_TYPE_HDMI_VP1 //hdmi in vp 1
```

如上图所示，迅为的 Linux 源码支持 MIPI7 寸屏幕，LVDS 10.1 寸 1024*600 屏幕，LVDS 10.1 寸 1280*800 屏幕，LVDS 7 寸屏幕，HDMI 接口连接显示。

若选择 define LCD_TYPE_LVDS_10_1_1280X800_gt9271 宏定义则需修改默认配置文件 kernel/arch/arm64/configs/rockchip_linux_defconfig，修改完成如下图所示：

```
2431 # CONFIG_TOUCHSCREEN_GSL3673_800X1280 is not set
2432 # CONFIG_TOUCHSCREEN_GSLX680_PAD is not set
2433 #CONFIG_TOUCHSCREEN_GT9XX=y
2434 #CONFIG_TOUCHSCREEN_GT9271=y
2435 # CONFIG_TOUCHSCREEN_HIDEEP is not set
```

若选择 define LCD_TYPE_LVDS_10_1_1024X600_gt911 宏定义则需修改默认配置文件 kernel/arch/arm64/configs/rockchip_linux_defconfig，修改完成如下图所示：

```
2432 # CONFIG_TOUCHSCREEN_GSLX680_PAD is not set
2433 #CONFIG_TOUCHSCREEN_GT9XX=y
2434 #CONFIG_TOUCHSCREEN_GT9271=y
2435 # CONFIG_TOUCHSCREEN_HIDEEP is not set
```

我们如果想要连接某个屏幕，使能对应的屏幕宏定义即可。如果想要设置多屏显示，请参考《【北京迅为】itop-3568 开发板多屏显示手册 V1.0.doc》。

5.4 编译 Buildroot

注意：编译系统时一定要使用普通权限编译。

Buildroot 是一款集成的编译集合包，解决了以前交叉编译麻烦的问题，本小节将介绍 buildroot 镜像的编译流程，分为单独编译和完整全自动编译。

5.4.1 单独编译

5.4.1.1 图形化界面

本小节单独编译镜像的顺序如下所示：

单独编译 uboot ->单独编译 kernel ->单独编译 recovery ->单独编译 buildroot

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 buildroot 镜像 update.img。

第一步：编译 uboot

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

光标默认就在 uboot，所以直接回车即可开始 uboot 的编译，编译过程如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh  
uboot  
processing option: uboot  
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-  
=====Start building uboot=====  
TARGET_UBOOT_CONFIG=rk3568  
#####  
## make rk3568_defconfig -j40  
#  
# configuration written to .config  
#
```

Uboot 编译完成如下图所示：

```
*****boot_merger ver 1.2*****  
Info:Pack loader ok.  
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIALL.ini  
/home/topeet/Linux/rk356x_linux/u-boot  
  
Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready  
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready  
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.ini  
  
Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)  
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-  
Mon Jan 1 21:17:38 PST 2024  
Running build_uboot succeeded.  
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api                config.mk          Kbuild            scripts            u-boot.dtb
arch              configs           Kconfig           snapshot.commit   u-boot-dtb.bin
bl31_0x00040000.bin disk             lib               spl               u-boot.img
bl31_0x00069000.bin doc              Licenses          System.map        u-boot.lds
bl31_0x0006b000.bin Documentation  MAINTAINERS       tee.bin           u-boot.map
bl31_0xfdcc1000.bin drivers         Makefile          test              u-boot-nodtb.bin
bl31_0xfdcce000.bin dts             make.sh           tools             u-boot.srec
bl31_0xfdcdd000.bin env             net               tpl               u-boot.sym
bl31.elf           examples         post              u-boot           u-boot
board              fit              PREUPLOADER.cfg  u-boot.bin        u-boot
cmd                fs               README            u-boot.cfg         u-boot
common             include          rk356x_spl_loader_v1.13.112.bin u-boot.cfg.configs
topeet@ubuntu:~/Linux/rk356x_linux$
```

第二步：编译 kernel

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第二个 kernel，点击回车即可开始 kernel 内核的编译，编译过程如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```



```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
kernel
processing option: kernel
=====Start building kernel=====
TARGET_KERNEL_ARCH =
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
TARGET_KERNEL_DTS =rk3568-evb1-ddr4-v10-linux
TARGET_KERNEL_CONFIG_FRAGMENT =
=====
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
#
# configuration written to .config
#
scripts/kconfig/conf --syncconfig Kconfig
```

内核编译完如下图所示：

```
Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdcce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar           build.config.gki_kprobes         ini          OWNERS
arch        build.config.gki_kprobes.aarch64  init        README
block      build.config.gki_kprobes.x86_64   ipc         resource.img
boot.img    build.config.gki.x86_64          Kbuild      samples
boot.its    build.config.x86_64              Kconfig     scripts
build.config.aarch64    built-in.a                      kern        security
build.config.allmodconfig  certs                          kernel      sound
build.config.allmodconfig.aarch64  COPYING                      kernel.img  System.map
build.config.allmodconfig.arm      CREDITS                      lib         tools
build.config.allmodconfig.x86_64    crypto                       LICENSES   usr
build.config.arm                d                             logo.bmp   virt
build.config.common             Documentation                 logo_kernel.bmp  vmlinux
build.config.gki                dr                             MAINTAINERS  vmlinux.o
build.config.gki.aarch64        dri                           Makefile     zboot.img
build.config.gki-debug.aarch64  drive                         modules.builtin
build.config.gki-debug.x86_64   drivers                      modules.order
build.config.gki_kasan          firmware                     Module.symvers
build.config.gki_kasan.aarch64    fs
```

第三步：编译 Recovery

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
Choose an option
```

```
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后将光标移动到第四个 recovery，点击回车即可开始 recovery 的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
Choose an option
```

```
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh  
recovery  
processing option: recovery  
=====Start building recovery=====  
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery  
=====  
config is rockchip_rk356x_recovery  
found kernel image  
Top of tree: /home/topeet/Linux/rk356x_linux  
=====  
#TARGET_BOARD=rk356x  
#OUTPUT_DIR=output/rockchip_rk356x_recovery  
#CONFIG=rockchip_rk356x_recovery_defconfig
```

编译完成如下图所示：

```
Image 2 (ramdisk)
Description: unavailable
Created: Mon Jan 1 22:52:09 2024
Type: RAMDisk Image
Compression: uncompressed
Data Size: 7479307 Bytes = 7304.01 KiB = 7.13 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff02
Entry Point: unavailable
Hash algo: sha256
Hash value: 22e0ed02b9a9c1f7573c07c449f4a094788b1b8de8ab4dafc8521abc71cd244c
Image 3 (resource)
Description: unavailable
Created: Mon Jan 1 22:52:09 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0fd3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
Init Ramdisk: ramdisk
FDT: fdt
done.
you take 0:06.68 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步：编译 buildroot

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第三个 rootfs，点击回车会进入到文件系统镜像选择界面，如下图所示：

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22

buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto
```

这里总共有五种镜像供编译选择，由于本小节要编译的是 buildroot，所以将光标移动到 buildroot 按下回车即可开始 buildroot 镜像的构建，构建过程如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
buildroot
processing option: buildroot
=====Start building rootfs(buildroot) to .rootfs=====
=====Start building buildroot rootfs =====
TARGET_BUILDROOT_CONFIG=rockchip_rk3568
=====
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
=====
```

构建完成如下所示：

```
make: Leaving directory '/home/topeet/Linux/rk356x_linux/buildroot'
2024-01-01T22:55:13 >>> Finalizing target directory
2024-01-01T22:55:14 >>> Sanitizing RPATH in target tree
2024-01-01T22:55:24 >>> Copying overlay board/rockchip/common/base
2024-01-01T22:55:24 >>> Copying overlay board/rockchip/rk356x/fs-overlay/
2024-01-01T22:55:24 >>> Copying overlay board/rockchip/common/wifi
2024-01-01T22:55:24 >>> Copying overlay board/rockchip/rk356x/fs-overlay-system/
2024-01-01T22:55:25 >>> Executing post-build script build/post.sh
2024-01-01T22:55:25 >>> Generating root filesystem image rootfs.cpio
2024-01-01T22:56:15 >>> Generating root filesystem image rootfs.ext2
2024-01-01T22:56:18 >>> Generating root filesystem image rootfs.squashfs
2024-01-01T22:56:22 >>> Generating root filesystem image rootfs.tar
Done in 14s
log saved on /home/topeet/Linux/rk356x_linux/br.log. pack buildroot image at: /home/topeet/Linux/rk356x_linux
/buildroot/output/rockchip_rk3568/images/rootfs.ext4
you take 1:14.90 to build buildroot
Running build_buildroot succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后,将打包要用到的所有固件移动到 rockdev 目录下,然后打包为完整的 update.img 镜像。

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面,进入之后如下所示:

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后将光标移动到第六个 firmware,点击回车就会运行相应的脚本将打包要用到的所有固件移动到 rockdev 目录下,具体打印如下所示:

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
firmware
```

```
processing option: firmware
```

```
/usr/bin/fakeroot
```

```
Source buildroot/build/envsetup.sh
```

```
Top of tree: /home/topeet/Linux/rk356x_linux
```

```
#TARGET_BOARD=rk3568
```

```
#OUTPUT_DIR=output/rockchip_rk3568
```

```
#CONFIG=rockchip_rk3568_defconfig
```

```
make: Entering directory '/home/topeet/Linux/rk356x_linux/buildroot'
```

```
GEN /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/Makefile
```

```
/home/topeet/Linux/rk356x_linux/buildroot/build/defconfig_hook.py -m /home/topeet/Linux/rk356x_linux/buildroot/configs/rockchip_rk3568_defconfig /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/.rockchipconfig
```

打包成 update.img 所需要的镜像就已经拷贝到了 rockdev 目录,如下图所示:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/  
boot.img          misc.img  parameter.txt  rootfs.img  userdata.img  
MiniLoaderAll.bin oem.img  recovery.img   uboot.img  
topeet@ubuntu:~/Linux/rk356x_linux$
```

然后进行最后的打包步骤，在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

Choose an option

```
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后将光标移动到第七个 updateimg，点击回车 rockdev 目录下全部固件打包成为一个整体的 update.img 镜像，具体打印如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

Choose an option

```
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ff9e00,userspace=0x3fff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

命令运行完成之后，在 rockdev 目录下就生成了打包而来的 update.img 镜像，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img          misc.img  parameter.txt  rootfs.img  update.img
MiniLoaderAll.bin oem.img   recovery.img   uboot.img   userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.4.1.2 命令行

本小节单独编译镜像的顺序如下所示：

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 buildroot

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 buildroot 镜像 update.img。

第一步：编译 uboot

在 linux 源码目录下输入以下命令编译 U-Boot

```
./build.sh uboot
```



```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh uboot
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
```

Uboot 编译完成如下图所示:

```
*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIAL.INI
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.INI

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 23:23:55 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录, 如下图所示:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api                config.mk          Kbuild            scripts            u-boot.dtb
arch               configs            Kconfig           snapshot.commit    u-boot-dtb.bin
bl31_0x00040000.bin disk              lib                spl                u-boot.img
bl31_0x00069000.bin doc               Licenses           System.map          u-boot.lds
bl31_0x0006b000.bin Documentation    MAINTAINERS        tee.bin             u-boot.map
bl31_0xfdcc1000.bin drivers          Makefile           test                u-boot-nodtb.bin
bl31_0xfdcce000.bin dts              make.sh            tools               u-boot.srec
bl31_0xfdcdd000.bin env              net                 tpl                 u-boot.sym
bl31.elf            examples          post               u-boot             u-boot.bin
board               fit               PREUPLOAD.cfg     u-boot.cfg         u-boot.cfg.configs
cmd                 fs                README             rk356x_spl_loader_v1.13.112.bin
common              include           rk356x_spl_loader_v1.13.112.bin
```

第二步: 编译 kernel

在 linux 源码目录下输入以下命令编译 Kernel

```
./build.sh kernel
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh kernel
kernel
processing option: kernel
=====Start building kernel=====
TARGET_KERNEL_ARCH =
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
TARGET_KERNEL_DTS   =rk3568-evb1-ddr4-v10-linux
TARGET_KERNEL_CONFIG_FRAGMENT =
=====
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
#
# configuration written to .config
#
```

内核编译完如下图所示:


```

Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0fd3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android          build.config.gki_kasan.x86_64    include          net
ar               build.config.gki_kprobes         ini              OWNERS
arch             build.config.gki_kprobes.aarch64 init             README
block           build.config.gki_kprobes.x86_64  ipc              resource.img
boot.img        build.config.gki.x86_64          Kbuild          samples
boot.its       build.config.x86_64              Kconfig         scripts
build.config.aarch64  built-in.a                       kern            securit
build.config.allmodconfig  certs                            kernel          security
build.config.allmodconfig.aarch64  COPYING                       kernel.img      sound
build.config.allmodconfig.arm      CREDITS                       lib             System.map
build.config.allmodconfig.x86_64  crypto                         LICENSES        tools
build.config.arm              d                               logo.bmp        usr
build.config.common           Documentation                  logo_kernel.bmp virt
build.config.gki              dr                               MAINTAINERS    vmlinux
build.config.gki.aarch64      dri                             Makefile        vmlinux.o
build.config.gki-debug.aarch64  drive                           mm              zboot.img
build.config.gki-debug.x86_64  drivers
build.config.gki_kasan        firmware
build.config.gki_kasan.aarch64  fs
topeet@ubuntu:~/Linux/rk356x_linux$

```

第三步：编译 Recovery

在 linux 源码目录下输入以下命令编译 Recovery

```
./build.sh recovery
```

```

topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh recovery
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET BOARD=rk356x

```

编译完成如下图所示：

```
Image 3 (resource)
Description: unavailable
Created:      Mon Jan  1 23:25:39 2024
Type:        Multi-File Image
Compression: uncompressed
Data Size:   176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo:   sha256
Hash value:  e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description:  unavailable
Kernel:      kernel
Init Ramdisk: ramdisk
FDT:         fdt
done.
you take 0:06.85 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步：编译 buildroot

进入源码根目录执行以下命令自动完成 Rootfs 的编译及打包：

```
./build.sh buildroot
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh buildroot
buildroot
processing option: buildroot
=====Start building rootfs(buildroot) to .rootfs=====
=====Start building buildroot rootfs =====
TARGET_BUILDROOT_CONFIG=rockchip_rk3568
=====
Top of tree: /home/topeet/Linux/rk356x_linux
=====

#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
```

编译完成如下图所示：

```
make: Leaving directory '/home/topeet/Linux/rk356x_linux/buildroot'
2024-01-01T23:26:23 >>> Finalizing target directory
2024-01-01T23:26:24 >>> Sanitizing RPATH in target tree
2024-01-01T23:26:34 >>> Copying overlay board/rockchip/common/base
2024-01-01T23:26:34 >>> Copying overlay board/rockchip/rk356x/fs-overlay/
2024-01-01T23:26:34 >>> Copying overlay board/rockchip/common/wifi
2024-01-01T23:26:34 >>> Copying overlay board/rockchip/rk356x/fs-overlay-system/
2024-01-01T23:26:35 >>> Executing post-build script build/post.sh
2024-01-01T23:26:35 >>> Generating root filesystem image rootfs.cpio
2024-01-01T23:27:25 >>> Generating root filesystem image rootfs.ext2
2024-01-01T23:27:28 >>> Generating root filesystem image rootfs.squashfs
2024-01-01T23:27:32 >>> Generating root filesystem image rootfs.tar
Done in 1min 14s
log saved on /home/topeet/Linux/rk356x_linux/br.log. pack buildroot image at: /home/topeet/Linux/rk356x_linux
/buildroot/output/rockchip_rk3568/images/rootfs.ext4
you take 1:14.51 to build buildroot
Running build_buildroot succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，进入工程目录根目录执行以下命令自动完成所有固件打包到 rockdev 目录下。

输入以下命令打包所有启动系统需要的镜像。进入 rockdev 目录下可以看到打包的单独镜像。

```
./build.sh firmware
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh firmware
firmware
processing option: firmware
/usr/bin/fakeroot
Source buildroot/build/envsetup.sh
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
```

输入以下命令，打包 update.img 镜像。

```
./build.sh updateimg
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh updateimg
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ffa000,userspace=0x3ff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img  update.img
MiniLoaderAll.bin  oem.img      recovery.img  uboot.img   userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.4.2 全自动编译

5.4.2.1 图形化界面

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后选择第五个 all，就会进入到文件系统类型选择页面，如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile update.img image | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22  
  
buildroot  
debian10  
debian11  
ubuntu20  
ubuntu22  
yocto
```

由于本小节全自动编译的是 buildroot 系统，所以这里选择 buildroot，然后回车，等待编译完成即可，脚本会自动编译 uboot kernel recovery buildroot 并打包为一个完整的 update.img 镜像，编译完成之后如下所示：


```
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
Running build_buildroot_all succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

在 rockdev 目录下就生成了打包完成的 update.img 镜像如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img  update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img   userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.4.2.2 命令行

进入源码根目录执行以下命令自动完成所有的编译。如果想自动全编译 buildroot 文件系统，输入以下命令：

```
./build.sh buildroot_update
```

输入上面的命令后，会自动编译 uboot kernel recovery buildroot 打包。打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img  update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img   userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.5 编译 Ubuntu

本小节将介绍 ubuntu 镜像的编译流程，分为单独编译和完整全自动编译。

5.5.1 单独编译

5.5.1.1 图形化界面

本小节单独编译镜像的顺序如下所示：

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 ubuntu

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 ubuntu 镜像 update.img。

第一步：编译 uboot

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

光标默认就在 uboot，所以直接回车即可开始 uboot 的编译，编译过程如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
# configuration written to .config
#
```

Uboot 编译完成如下图所示：

```
*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIALL.ini
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.ini

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 21:17:38 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api                config.mk          Kbuild             scripts            u-boot.dtb
arch               configs            Kconfig            snapshot.commit   u-boot-dtb.bin
bl31_0x00040000.bin disk              lib                spl                uboot.img
bl31_0x00069000.bin doc               licenses           System.map        u-boot.lds
bl31_0x0006b000.bin Documentation    MAINTAINERS       tee.bin           u-boot.map
bl31_0xfdcc1000.bin drivers          Makefile          test              u-boot-nodtb.bin
bl31_0xfdcce000.bin dts              make.sh           tools             u-boot.srec
bl31_0xfdcdd000.bin env              net               tpl               u-boot.sym
bl31.elf           examples          post              u-boot            u-boot.bin
board             fit              PREUPLOAD.cfg     u-boot.bin        u-boot.cfg
cmd              fs               README            u-boot.cfg        u-boot.cfg.configs
common           include          rk356x_spl_loader_v1.13.112.bin
```

第二步：编译 kernel

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
Choose an option
```

```
uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第二个 kernel，点击回车即可开始 kernel 内核的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
Choose an option
```

```
uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
kernel
```

```
processing option: kernel
```

```
=====Start building kernel=====
```

```
TARGET_KERNEL_ARCH =
```

```
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
```

```
TARGET_KERNEL_DTS =rk3568-evb1-ddr4-v10-linux
```

```
TARGET_KERNEL_CONFIG_FRAGMENT =
```

```
=====
```

```
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.
```

```
1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
```

```
#
```

```
# configuration written to .config
```

```
#
```

```
scripts/kconfig/conf --syncconfig Kconfig
```

内核编译完如下图所示：

```
Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdcce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar            build.config.gki_kprobes         ini           OWNERS
arch         build.config.gki_kprobes.aarch64 init          README
block        build.config.gki_kprobes.x86_64  ipc          resource.img
boot.img     build.config.gki.x86_64          Kbuild       samples
boot.its     build.config.x86_64             Kconfig      scripts
build.config.aarch64    built-in.a      kern          securit
build.config.allmodconfig  certs          kernel        security
build.config.allmodconfig.aarch64  COPYING       kernel.img    sound
build.config.allmodconfig.arm      CREDITS       lib           System.map
build.config.allmodconfig.x86_64   crypto        LICENSES      tools
build.config.arm              d             logo.bmp     usr
build.config.common            Documentation  logo_kernel.bmp virt
build.config.gki              dr            MAINTAINERS  vmlinux
build.config.gki.aarch64      dri           Makefile     vmlinux.o
build.config.gki-debug.aarch64  drive         mm           zboot.img
build.config.gki-debug.x86_64   drivers
build.config.gki_kasan         firmware
build.config.gki_kasan.aarch64  fs
topeet@ubuntu:~/Linux/rk356x_linux$
```

第三步：编译 Recovery

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```



```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第四个 recovery，点击回车即可开始 recovery 的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk356x
#OUTPUT_DIR=output/rockchip_rk356x_recovery
#CONFIG=rockchip_rk356x_recovery_defconfig
```

编译完成如下图所示：

```
Entry Point: unavailable
Hash algo: sha256
Hash value: 22e0ed02b9a9c1f7573c07c449f4a094788b1b8de8ab4dafc8521abc71cd244c
Image 3 (resource)
Description: unavailable
Created: Mon Jan 1 22:52:09 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
Init Ramdisk: ramdisk
FDT: fdt
done.
you take 0:06.68 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步：构建 Ubuntu

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后将光标移动到第三个 rootfs，点击回车会进入到文件系统镜像选择界面，如下图所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22  
  
buildroot  
debian10  
debian11  
ubuntu20  
ubuntu22  
yocto
```

这里总共有六种镜像供编译选择，由于本小节要编译的是 ubuntu，所以将光标移动到第 4 个 ubuntu20 或者第 5 个 ubuntu22，大家根据自己的需求选择合适的镜像即可。按下回车即可开始 ubuntu 镜像的构建，构建过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22  
  
buildroot  
debian10  
debian11  
ubuntu20  
ubuntu22  
yocto
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
ubuntu20
processing option: ubuntu20
=====Start building rootfs(ubuntu20) to .rootfs=====
=====Start building ubuntu20=====
Building for ubuntu20
Extract image
[sudo] password for topeet:
```

需要注意，由于 ubuntu 镜像的构建中的解压步骤需要 root 权限，所以需要输入一次 root 用户的密码，构建完成如下所示：

```
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Copying files into the device: done
Writing superblocks and filesystem accounting information: done

Rootfs Image: rootfs.img
Running build_ubuntu20 succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

会在 ubuntu 目录下生成名为 rootfs.img 的文件系统烧写镜像，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls ubuntu/
binary      mk-rootfs.sh  rootfs.img      ubuntu-jammy-arm64.tar.xz
mk-image.sh post-build.sh ubuntu-focal-arm64.tar.xz
topeet@ubuntu:~/Linux/rk356x_linux$
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，将打包要用到的所有固件移动到 rockdev 目录下，然后打包为完整的 update.img 镜像。

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第六个 firmware，点击回车就会运行相应的脚本将打包要用到的所有固件移动到 rockdev 目录下，具体打印如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
firmware
processing option: firmware
/usr/bin/fakeroot
Source buildroot/build/envsetup.sh
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
=====
make: Entering directory '/home/topeet/Linux/rk356x_linux/buildroot'
GEN /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/Makefile
/home/topeet/Linux/rk356x_linux/buildroot/build/defconfig_hook.py -m /home/topeet/Linux/rk356x_linux/buildroot/configs/rockchip_rk3568_defconfig /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/.rockchipconfig
```

打包成 update.img 所需要的镜像就已经拷贝到了 rockdev 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img          misc.img  parameter.txt  rootfs.img  userdata.img
MiniLoaderAll.bin oem.img  recovery.img   uboot.img
```

然后进行最后的打包步骤，在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```


然后将光标移动到第七个 updateimg, 点击回车 rockdev 目录下全部固件打包成为一个整体的 update.img 镜像, 具体打印如下所示:

```
iTOP-RK3568 building script, http://www.topeet.com

Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ff9e00,userspace=0x3ff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

命令运行完成之后, 在 rockdev 目录下就生成了打包而来的 update.img 镜像, 如下图所示:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.5.1.2 命令行

本小节单独编译镜像的顺序如下所示:

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 ubuntu

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 ubuntu 镜像 update.img。

第一步：编译 uboot

在 linux 源码目录下输入以下命令编译 U-Boot

```
./build.sh uboot
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh uboot
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
```

Uboot 编译完成如下图所示：

```
*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIALL.ini
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.ini

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 23:23:55 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api                config.mk          Kbuild            scripts            u-boot.dtb
arch               configs            Kconfig           snapshot.commit   u-boot-dtb.bin
bl31_0x00040000.bin disk              lib               spl               uboot.img
bl31_0x00069000.bin doc               Licenses          System.map        u-boot.lds
bl31_0x0006b000.bin Documentation    MAINTAINERS       tee.bin           u-boot.map
bl31_0xfdcc1000.bin drivers          Makefile          test              u-boot-nodtb.bin
bl31_0xfdcce000.bin dts              make.sh           tools             u-boot.srec
bl31_0xfdcdd000.bin env              net               tpl              u-boot.sym
bl31.elf           examples          post              u-boot            u-boot.bin
board             fit              PREUPLOAD.cfg    u-boot.bin        u-boot.cfg
cmd              fs               README            u-boot.cfg        u-boot.cfg.configs
common           include          rk356x_spl_loader_v1.13.112.bin
```

第二步：编译 kernel

在 linux 源码目录下输入以下命令编译 Kernel

```
./build.sh kernel
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh kernel
kernel
processing option: kernel
=====Start building kernel=====
TARGET_KERNEL_ARCH =
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
TARGET_KERNEL_DTS =rk3568-evb1-ddr4-v10-linux
TARGET_KERNEL_CONFIG_FRAGMENT =
=====
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
#
# configuration written to .config
#
```

内核编译完如下图所示：

```
Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdcce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar           build.config.gki_kprobes         ini          OWNERS
arch         build.config.gki_kprobes.aarch64 init         README
block       build.config.gki_kprobes.x86_64  ipc          resource.img
boot.img     build.config.gki.x86_64          Kbuild      samples
boot.its     build.config.x86_64             Kconfig     scripts
build.config.aarch64    built-in.a                      kern         security
build.config.allmodconfig  certs                           kernel      sound
build.config.allmodconfig.aarch64  COPYING                       kernel.img  System.map
build.config.allmodconfig.arm      CREDITS                      lib         tools
build.config.allmodconfig.x86_64    crypto                       LICENSES   usr
build.config.arm              d                             logo.bmp   virt
build.config.common            Documentation                 logo_kernel.bmp  vmlinux
build.config.gki              dr                           MAINTAINERS  vmlinux.o
build.config.gki.aarch64      dri                          Makefile     zboot.img
build.config.gki-debug.aarch64  drive                        modules.builtin
build.config.gki-debug.x86_64   drivers                      modules.order
build.config.gki_kasan         firmware                     Module.symvers
build.config.gki_kasan.aarch64    fs
```

第三步：编译 Recovery

在 linux 源码目录下输入以下命令编译 Recovery


```
./build.sh recovery
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh recovery
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk356x
```

编译完成如下图所示:

```
Image 3 (resource)
Description: unavailable
Created: Mon Jan 1 23:25:39 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
Init Ramdisk: ramdisk
FDT: fdt
done.
you take 0:06.85 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步: 编译 ubuntu

由于 ubuntu 有 ubuntu20 和 ubuntu22 两个版本, 所以提供了两个命令, 分别对应 ubuntu20 和 ubuntu22:

```
./build.sh ubuntu20
```

```
./build.sh ubuntu22
```

这里以 ubuntu20 为例进行编译, 编译过程如下所示:

```
topeet@topeet:~/Linux/3588s-linux$ ./build.sh ubuntu20
ubuntu20
processing option: ubuntu20
=====Start building rootfs(ubuntu20) to .rootfs=====
=====Start building ubuntu20=====
Building for ubuntu20
Deleted binary directory.
Extract image
```

需要注意, 由于 ubuntu 镜像的构建中的解压步骤需要 root 权限, 所以需要输入一次 root 用户的密码, 编译完成如下图所示:


```
Adding build-info to /etc/os-release...
Fixing up /etc/fstab...
Fixing up rootfs type: auto
Adding dirs and links...
0+0 records in
0+0 records out
0 bytes copied, 2.762e-05 s, 0.0 kB/s
mke2fs 1.46.5 (30-Dec-2021)
Discarding device blocks: done
Creating filesystem with 1298432 4k blocks and 325120 inodes
Filesystem UUID: 67dfd0bb-5224-42a8-8964-c568bd810def
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Copying files into the device: done
Writing superblocks and filesystem accounting information: done

Rootfs Image: rootfs.img
Running build_ubuntu20 succeeded.
Running build_rootfs succeeded.
topeet@topeet:~/Linux/3588s-linux$
```

会在 ubuntu 目录下生成名为 rootfs.img 的文件系统烧写镜像，如下图所示：

```
topeet@topeet:~/Linux/3588s-linux$ ls ubuntu/
binary      mk-rootfs.sh  rootfs.img      ubuntu-jammy-arm64.tar.xz
mk-image.sh post-build.sh ubuntu-focal-arm64.tar.xz
topeet@topeet:~/Linux/3588s-linux$
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，进入工程目录根目录执行以下命令自动完成所有固件打包到 rockdev 目录下。

输入以下命令打包所有启动系统需要的镜像。进入 rockdev 目录下可以看到打包的单独镜像。

```
./build.sh firmware
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh firmware
firmware
processing option: firmware
/usr/bin/fakeroot
Source buildroot/build/envsetup.sh
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
```

输入以下命令，打包 update.img 镜像。

```
./build.sh updateimg
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh updateimg
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ffa000,userspace=0x3ff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.5.2 全自动编译

5.5.2.1 图形化界面

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

Choose an option

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

然后选择第五个 all，就会进入到文件系统类型选择页面，如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

Choose an option

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall

iTOP-RK3568 building script, <http://www.topeet.com>

Compile update.img image | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22

Choose an option

buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto

由于本小节全自动编译的是 ubuntu 系统，所以这里可以根据大家想编译的 ubuntu 版本选择第四个 ubuntu20 或者 ubuntu22，然后回车，等待编译完成即可，脚本会自动编译 uboot kernel recovery ubuntu 并打包为一个完整的 update.img 镜像，编译完成之后如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

Compile update.img image | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22

Choose an option

buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto

```
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
Running build_buildroot_all succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

在 rockdev 目录下就生成了打包完成的 update.img 镜像如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img  update.img
MiniLoaderAll.bin  oem.img      recovery.img  uboot.img   userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.5.2.2 命令行

进入源码根目录执行以下命令自动完成所有的编译。如果想自动全编译 ubuntu 文件系统，输入以下命令：

```
./build.sh ubuntu20_update
./build.sh ubuntu22_update
```

上面两个命令分别对应 ubuntu20 版本和 ubuntu22 版本，输入上面的命令后，会自动编译 uboot kernel recovery ubuntu 并打包。打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img  update.img
MiniLoaderAll.bin  oem.img      recovery.img  uboot.img   userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.6 编译 Debian

本小节将介绍 Debian 镜像的编译流程，分为单独编译和完整全自动编译。

5.6.1 单独编译

5.6.1.1 图形化界面

本小节单独编译镜像的顺序如下所示：

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 debian

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 debian 镜像 update.img。

第一步：编译 uboot

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

光标默认就在 uboot，所以直接回车即可开始 uboot 的编译，编译过程如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
# configuration written to .config
#
```

Uboot 编译完成如下图所示：

```
*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIAL.INI
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.INI

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 21:17:38 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api          config.mk    Kbuild      scripts      u-boot.dtb
arch         configs      Kconfig     snapshot.commit  u-boot-dtb.bin
bl31_0x00040000.bin disk         lib          spl           uboot.img
bl31_0x00069000.bin doc          licenses    System.map    u-boot.lds
bl31_0x0006b000.bin Documentation MAINTAINERS tee.bin       u-boot.map
bl31_0xfdcc1000.bin drivers      Makefile    test          u-boot-nodtb.bin
bl31_0xfdcc0000.bin dts          make.sh     tools         u-boot.srec
bl31_0xfdc00000.bin env          net         tpl           u-boot.sym
bl31.elf     examples     post        u-boot       u-boot.bin
board        fit          PREUPLOAD.cfg u-boot        u-boot.cfg
cmd          fs           README      u-boot        u-boot.cfg.configs
common       include      rk356x_spl_loader_v1.13.112.bin
```

第二步：编译 kernel

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后将光标移动到第二个 kernel，点击回车即可开始 kernel 内核的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
kernel
```

```
processing option: kernel
```

```
=====Start building kernel=====
```

```
TARGET_KERNEL_ARCH =
```

```
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
```

```
TARGET_KERNEL_DTS =rk3568-evb1-ddr4-v10-linux
```

```
TARGET_KERNEL_CONFIG_FRAGMENT =
```

```
=====
```

```
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
```

```
#
```

```
# configuration written to .config
```

```
#
```

```
scripts/kconfig/conf --syncconfig Kconfig
```

内核编译完如下图所示：

```

Hash value: 7e03bdc8b85b1dc49dfc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdcce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar           build.config.gki_kprobes         ini          OWNERS
arch         build.config.gki_kprobes.aarch64 init         README
block       build.config.gki_kprobes.x86_64  ipc         resource.img
boot.img     build.config.gki.x86_64         Kbuild      samples
boot.its    build.config.x86_64             Kconfig     scripts
build.config.aarch64    built-in.a    kern        securit
build.config.allmodconfig  certs        kernel     security
build.config.allmodconfig.aarch64  COPYING     kernel.img  sound
build.config.allmodconfig.arm    CREDITS     lib        System.map
build.config.allmodconfig.x86_64  crypto      LICENSES   tools
build.config.arm            d           logo.bmp   usr
build.config.common        Documentation logo_kernel.bmp  virt
build.config.gki           dr          MAINTAINERS  vmlinux
build.config.gki.aarch64   dri         Makefile    vmlinux.o
build.config.gki-debug.aarch64  drive      mm          zboot.img
build.config.gki-debug.x86_64  drivers
build.config.gki_kasan      firmware
build.config.gki_kasan.aarch64  fs
topeet@ubuntu:~/Linux/rk356x_linux$

```

第三步：编译 Recovery

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```

topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh

```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | Choose an option | recovery | all | firmware | updateimg | cleanall
```

```
uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第四个 recovery，点击回车即可开始 recovery 的编译，编译过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | Choose an option | recovery | all | firmware | updateimg | cleanall
```

```
uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk356x
#OUTPUT_DIR=output/rockchip_rk356x_recovery
#CONFIG=rockchip_rk356x_recovery_defconfig
```

编译完成如下图所示：


```
Image 2 (ramdisk)
Description: unavailable
Created:    Mon Jan 1 22:52:09 2024
Type:      RAMDisk Image
Compression: uncompressed
Data Size: 7479307 Bytes = 7304.01 KiB = 7.13 MiB
Architecture: AArch64
OS:        Linux
Load Address: 0xffffffff02
Entry Point: unavailable
Hash algo:  sha256
Hash value:  22e0ed02b9a9c1f7573c07c449f4a094788b1b8de8ab4dafc8521abc71cd244c
Image 3 (resource)
Description: unavailable
Created:    Mon Jan 1 22:52:09 2024
Type:      Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo:  sha256
Hash value:  e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel:      kernel
Init Ramdisk: ramdisk
FDT:         fdt
done.
you take 0:06.68 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步：构建 Debian

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第三个 rootfs，点击回车会进入到文件系统镜像选择界面，如下图所示：

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall

uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22
buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto
```

这里总共有五种镜像供编译选择，由于本小节要编译的是 debian，所以将光标移动到第 2 个 debian10 或者第 3 个 debian11，大家根据自己的需求选择合适的镜像即可。按下回车即可开始 ubuntu 镜像的构建，构建过程如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option
Compile single rootfs img | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22
buildroot
debian10
debian11
ubuntu20
ubuntu22
yocto
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
debian10
processing option: debian10
=====Start building rootfs(debian10) to .rootfs=====
=====Start building debian10=====
Building for debian10
Extract image
```

需要注意，由于 debian 镜像的构建中的解压步骤需要 root 权限，所以需要输入一次 root 用户的密码，构建完成如下所示：

```
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Copying files into the device: done
Writing superblocks and filesystem accounting information: done

Rootfs Image: rootfs.img
Running build_debian10 succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

会在 debian 目录下生成名为 rootfs.img 的文件系统烧写镜像，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls debian/
binary          linaro-buster-arm64.tar.xz  mk-rootfs.sh  rootfs.img
linaro-bullseye-arm64.tar.xz  mk-image.sh               post-build.sh
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，将打包要用到的所有固件移动到 rockdev 目录下，然后打包为完整的 update.img 镜像。

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后将光标移动到第六个 firmware，点击回车就会运行相应的脚本将打包要用到的所有固件移动到 rockdev 目录下，具体打印如下所示：

iTOP-RK3568 building script, <http://www.topeet.com>

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
firmware  
processing option: firmware  
/usr/bin/fakeroot  
Source buildroot/build/envsetup.sh  
Top of tree: /home/topeet/Linux/rk356x_linux  
=====  
#TARGET_BOARD=rk3568  
#OUTPUT_DIR=output/rockchip_rk3568  
#CONFIG=rockchip_rk3568_defconfig  
=====  
make: Entering directory '/home/topeet/Linux/rk356x_linux/buildroot'  
GEN /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/Makefile  
/home/topeet/Linux/rk356x_linux/buildroot/build/defconfig_hook.py -m /home/topeet/Linux/rk356x_linux/buildroo  
t/configs/rockchip_rk3568_defconfig /home/topeet/Linux/rk356x_linux/buildroot/output/rockchip_rk3568/.rockchi  
pconfig
```

打包成 update.img 所需要的镜像就已经拷贝到了 rockdev 目录，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/  
boot.img      misc.img      parameter.txt  rootfs.img    userdata.img  
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img  
topeet@ubuntu:~/Linux/rk356x_linux$
```

然后进行最后的打包步骤，在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
Choose an option
```

```
uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```

然后将光标移动到第七个 updateimg，点击回车 rockdev 目录下全部固件打包成为一个整体的 update.img 镜像，具体打印如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall
```

```
Choose an option
```

```
uboot
kernel
rootfs
recovery
all
firmware
updateimg
cleanall
```



```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniLoaderAll.bin
bootloader,Add file: ./Image/MiniLoaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/uboot.img
uboot,Add file: ./Image/uboot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ff9e00,userspace=0x3ff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

命令运行完成之后，在 rockdev 目录下就生成了打包而来的 update.img 镜像，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.6.1.2 命令行

本小节单独编译镜像的顺序如下所示：

单独编译 uboot -> 单独编译 kernel -> 单独编译 recovery -> 单独编译 debian

各个镜像编译完成后，最终打包镜像 update.img 的方式来编译 debian 镜像 update.img。

第一步：编译 uboot

在 linux 源码目录下输入以下命令编译 U-Boot

```
./build.sh uboot
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh uboot
uboot
processing option: uboot
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j40
#
```

Uboot 编译完成如下图所示:

```
*****boot_merger ver 1.2*****
Info:Pack loader ok.
pack loader okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKBOOT/RK3568MINIAL.INI
/home/topeet/Linux/rk356x_linux/u-boot

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk356x_spl_loader_v1.13.112.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/topeet/Linux/rk356x_linux/rkbin/RKTRUST/RK3568TRUST.INI

Platform RK3568 is build OK, with new .config(make rk3568_defconfig -j40)
/home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
Mon Jan 1 23:23:55 PST 2024
Running build_uboot succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

编译完成后生成 uboot.img 到 u-boot 目录, 如下图所示:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls u-boot/
api          config.mk    Kbuild      scripts      u-boot.dtb
arch         configs     Kconfig     snapshot.commit  u-boot-dtb.bin
bl31_0x00040000.bin  disk        lib          spl          u-boot.img
bl31_0x00069000.bin  doc         Licenses    System.map    u-boot.lds
bl31_0x0006b000.bin  Documentation  MAINTAINERS tee.bin       u-boot.map
bl31_0xfdcc1000.bin  drivers      Makefile    test         u-boot-nodtb.bin
bl31_0xfdcce000.bin  dts          make.sh     tools        u-boot.srec
bl31_0xfdcdd000.bin  env          net         tpl          u-boot.sym
bl31.elf        examples     post        u-boot       u-boot
board          fit          PREUPLOAD.cfg  u-boot.bin   u-boot.cfg
cmd           fs           README      u-boot.cfg   u-boot.cfg.configs
common        include      rk356x_spl_loader_v1.13.112.bin
```

第二步: 编译 kernel

在 linux 源码目录下输入以下命令编译 Kernel

```
./build.sh kernel
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh kernel
kernel
processing option: kernel
=====Start building kernel=====
TARGET_KERNEL_ARCH =
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
TARGET_KERNEL_DTS   =rk3568-evb1-ddr4-v10-linux
TARGET_KERNEL_CONFIG_FRAGMENT =
=====
Using prebuilt GCC toolchain: /home/topeet/Linux/rk356x_linux/prebuilts/gcc/linux-x86/aarch64/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu-
#
# configuration written to .config
#
```

内核编译完如下图所示:


```

Hash value: 7e03bdc8b85b1dc49d4fc5271475d4fb9594b23dfad6c9b581e4907d0b0d8e8e4
Image 1 (kernel)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Kernel Image
Compression: uncompressed
Data Size: 25731080 Bytes = 25128.01 KiB = 24.54 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: ab833f93ee1333bdce71cc504168f906d00180fc8094281bc73d2373430000d
Image 2 (resource)
Description: unavailable
Created: Mon Jan 1 21:21:57 2024
Type: Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
Running build_kernel succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$

```

编译完成后映像文件会打包成 boot.img 放到 kernel 目录。如下图所示

```

topeet@ubuntu:~/Linux/rk356x_linux$ ls kernel/
android      build.config.gki_kasan.x86_64    include      net
ar           build.config.gki_kprobes         ini          OWNERS
arch        build.config.gki_kprobes.aarch64  init        README
block       build.config.gki_kprobes.x86_64   ipc         resource.img
boot.img    build.config.gki.x86_64          Kbuild      samples
boot.its    build.config.x86_64             Kconfig     scripts
build.config.aarch64    built-in.a    kern        securit
build.config.allmodconfig  certs        kernel      security
build.config.allmodconfig.aarch64  COPYING     kernel.img  sound
build.config.allmodconfig.arm    CREDITS     lib         System.map
build.config.allmodconfig.x86_64  crypto      LICENSES    tools
build.config.arm             d           logo.bmp    usr
build.config.common          Documentation  logo_kernel.bmp  virt
build.config.gki            dr           MAINTAINERS  vmlinux
build.config.gki.aarch64    dri          Makefile     vmlinux.o
build.config.gki-debug.aarch64  drive       mm           zboot.img
build.config.gki-debug.x86_64  drivers
build.config.gki_kasan      firmware
build.config.gki_kasan.aarch64  fs
topeet@ubuntu:~/Linux/rk356x_linux$

```

第三步：编译 Recovery

在 linux 源码目录下输入以下命令编译 Recovery

```
./build.sh recovery
```

```

topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh recovery
recovery
processing option: recovery
=====Start building recovery=====
TARGET_RECOVERY_CONFIG=rockchip_rk356x_recovery
=====
config is rockchip_rk356x_recovery
found kernel image
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk356x

```

编译完成如下图所示：

```
Image 3 (resource)
Description: unavailable
Created:    Mon Jan 1 23:25:39 2024
Type:      Multi-File Image
Compression: uncompressed
Data Size: 176640 Bytes = 172.50 KiB = 0.17 MiB
Hash algo: sha256
Hash value: e9f8f64f945e496005b4b98f0f1d3cc76256e6e894d3def1dd7ed95fa764ec06
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel:     kernel
Init Ramdisk: ramdisk
FDT:        fdt
done.
you take 0:06.85 to build recovery
Running build_recovery succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

第四步：编译 debian

由于 ubuntu 有 debian10 和 debian11 两个版本，所以提供了两个命令，分别对应 debian10 和 debian11：

```
./build.sh debian10
```

```
./build.sh debian11
```

这里以 debian10 为例进行编译，编译过程如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh debian10
debian10
processing option: debian10
=====Start building rootfs(debian10) to .rootfs=====
=====Start building debian10=====
Deleted binary directory.
Building for debian10
Extract image
```

需要注意，由于 debian 镜像的构建中的解压步骤需要 root 权限，所以需要输入一次 root 用户的密码，编译完成如下图所示：

```
Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Copying files into the device: done
Writing superblocks and filesystem accounting information: done

Rootfs Image: rootfs.img
Running build_debian10 succeeded.
Running build_rootfs succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

会在 ubuntu 目录下生成名为 rootfs.img 的文件系统烧写镜像，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls debian/
binary          linaro-buster-arm64.tar.xz  mk-rootfs.sh  rootfs.img
linaro-bullseye-arm64.tar.xz  mk-image.sh               post-build.sh
topeet@ubuntu:~/Linux/rk356x_linux$
```

第五步：打包镜像

上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后,进入工程目录根目录执行以下命令自动完成所有固件打包到 rockdev 目录下。

输入以下命令打包所有启动系统需要的镜像。进入 rockdev 目录下可以看到打包的单独镜像。

```
./build.sh firmware
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh firmware
firmware
processing option: firmware
/usr/bin/fakeroot
Source buildroot/build/envsetup.sh
Top of tree: /home/topeet/Linux/rk356x_linux
=====
#TARGET_BOARD=rk3568
#OUTPUT_DIR=output/rockchip_rk3568
#CONFIG=rockchip_rk3568_defconfig
```

输入以下命令,打包 update.img 镜像。

```
./build.sh updateimg
```

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh updateimg
updateimg
processing option: updateimg
Make update.img
start to make update.img...
Android Firmware Package Tool v2.0
----- PACKAGE -----
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0x28b,userspace=0x1
Add file: ./Image/MiniloaderAll.bin
bootloader,Add file: ./Image/MiniloaderAll.bin done,offset=0x1000,size=0x719c0,userspace=0xe4
Add file: ./Image/parameter.txt
parameter,Add file: ./Image/parameter.txt done,offset=0x73000,size=0x1f4,userspace=0x1
Add file: ./Image/u-boot.img
u-boot,Add file: ./Image/u-boot.img done,offset=0x73800,size=0x400000,userspace=0x800
Add file: ./Image/misc.img
misc,Add file: ./Image/misc.img done,offset=0x473800,size=0xc000,userspace=0x18
Add file: ./Image/boot.img
boot,Add file: ./Image/boot.img done,offset=0x47f800,size=0x18d7e00,userspace=0x31b0
Add file: ./Image/recovery.img
recovery,Add file: ./Image/recovery.img done,offset=0x1d57800,size=0x1ffa000,userspace=0x3ff4
Add file: ./Image/rootfs.img
rootfs,Add file: ./Image/rootfs.img done,offset=0x3d51800,size=0x61d40000,userspace=0xc3a80
Add file: ./Image/oem.img
oem,Add file: ./Image/oem.img done,offset=0x65a91800,size=0x1100000,userspace=0x2200
Add file: ./Image/userdata.img
userdata,Add file: ./Image/userdata.img done,offset=0x66b91800,size=0x500000,userspace=0xa00
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.0*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

打包完成会在 rockdev 目录下生成 update.img, 如下图所示:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/  
boot.img      misc.img      parameter.txt  rootfs.img    update.img  
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img  
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.6.2 全自动编译

5.6.2.1 图形化界面

首先在 linux 源码目录下输入以下命令进入编译的 UI 界面，进入之后如下所示：

```
./build.sh
```

```
topeet@ubuntu:~/Linux/rk356x_linux$  
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

然后选择第五个 all，就会进入到文件系统类型选择页面，如下所示：

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile image | uboot | kernel | rootfs | recovery | all | firmware | updateimg | cleanall  
  
uboot  
kernel  
rootfs  
recovery  
all  
firmware  
updateimg  
cleanall
```

```
iTOP-RK3568 building script, http://www.topeet.com
```

```
Choose an option  
Compile update.img image | buildroot | debian10 | debian11 | ubuntu20 | ubuntu22  
  
buildroot  
debian10  
debian11  
ubuntu20  
ubuntu22  
vocto
```

由于本小节全自动编译的是 debian 系统，所以这里可以根据大家想编译的 ubuntu 版本选择第四个 debian10 或者 debian11，然后回车，等待编译完成即可，脚本会自动编译 uboot kernel recovery debian 并打包为一个完整的 update.img 镜像，编译完成之后如下所示：

```
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
Running build_buildroot_all succeeded.
topeet@ubuntu:~/Linux/rk356x_linux$
```

在 rockdev 目录下就生成了打包完成的 update.img 镜像如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.6.2.2 命令行

进入源码根目录执行以下命令自动完成所有的编译。如果想自动全编译 ubuntu 文件系统，输入以下命令：

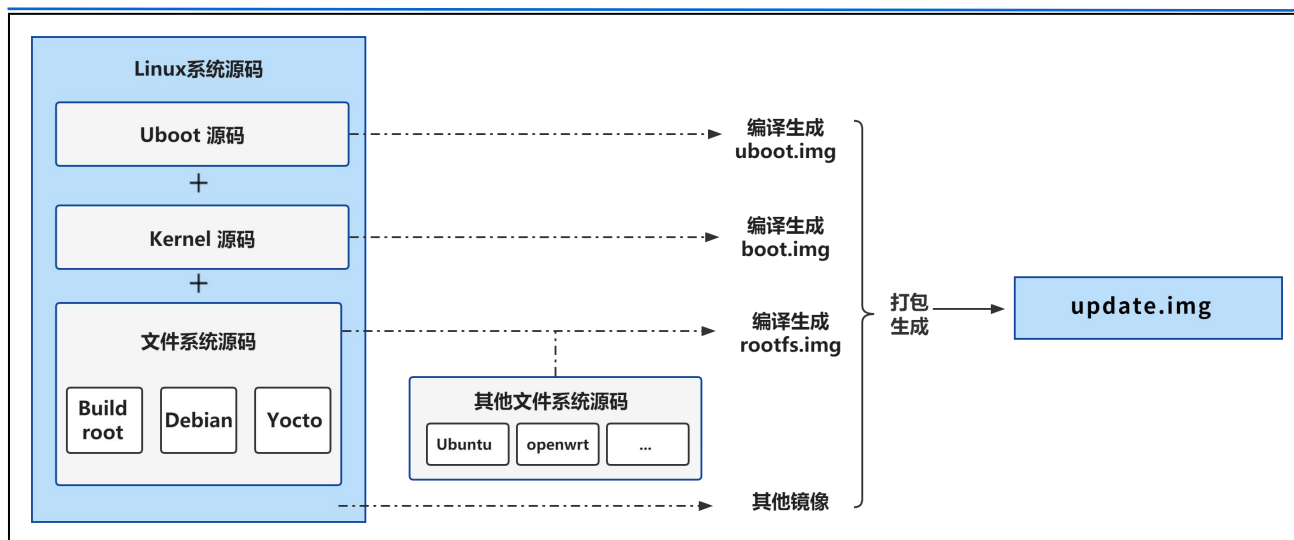
```
./build.sh debian10_update
./build.sh debian11_update
```

上面两个命令分别对应 debian10 版本和 debian11 版本，输入上面的命令后，会自动编译 uboot kernel recovery ubuntu 并打包。打包完成会在 rockdev 目录下生成 update.img，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux$ ls rockdev/
boot.img      misc.img      parameter.txt  rootfs.img    update.img
MiniLoaderAll.bin  oem.img      recovery.img   uboot.img     userdata.img
topeet@ubuntu:~/Linux/rk356x_linux$
```

5.7 编译其他系统

因为瑞芯微官方的 SDK 包里只包含了 buildroot 系统、debian 系统和 yocto 系统源码，瑞芯微官方 SDK 包的 build.sh 编译脚本只能编译此三种系统。根据上面三小节内容可知除了文件系统之外的内核和 uboot 等源码使用的是相同的，所以想要编译其他 Linux 系统，如 Ubuntu，openwrt 等其他系统（可以参考《【北京迅为】itop-3568 开发板文件系统构建手册》手册进行系统构建），内核和 uboot 等的编译可以参考 buildroot 系统的单独编译章节进行编译。由此可以总结为下图内容，如下图所示：



若想在迅为提供的 Ubuntu 系统基础上进行修改可参考本手册 9.1 章节进行自己 Ubuntu 系统的定制。

其他 Linux 系统定制构建完，最后可根据本手册 9.2 小节打包生成整体 update.img 固件包。

第 6 章 编译 Linux 源码问题总结

问题一：如何查看编译命令？

源码根目录下的 build.sh 是编译脚本，在源码根目录执行如下命令可以查看编译命令的帮助信息。

```
./build.sh -h
```

```
root@ubuntu:/home/topeet/Linux/rk356x_linux# ./build.sh -h
Usage: build.sh [OPTIONS]
Available options:
BoardConfig*.mk  -switch to specified board config
lunch            -list current SDK boards and switch to specified board config
uboot            -build uboot
spl              -build spl
loader          -build loader
kernel          -build kernel
modules         -build kernel modules
toolchain       -build toolchain
rootfs          -build default rootfs, currently build buildroot as default
buildroot       -build buildroot rootfs
ramboot        -build ramboot image
multi-npu_boot  -build boot image for multi-npu board
yocto           -build yocto rootfs
debian          -build debian10 buster/x11 rootfs
distro          -build debian10 buster/wayland rootfs
pcba            -build pcba
recovery        -build recovery
all             -build uboot, kernel, rootfs, recovery image
cleanall        -clean uboot, kernel, rootfs, recovery
firmware        -pack all the image we need to boot up system
updateimg       -pack update image
otapackage      -pack ab update otapackage image (update_ota.img)
sdpackage       -pack update sdcard package image (update_sdcard.img)
save            -save images, patches, commands used to debug
allsave         -build all & firmware & updateimg & save
check           -check the environment of building
info            -see the current board building information
app/<pkg>        -build packages in the dir of app/*
external/<pkg>   -build packages in the dir of external/*

Default option is 'allsave'.
root@ubuntu:/home/topeet/Linux/rk356x_linux#
```

问题二：如何查看部分模块详细编译命令？

查看部分模块详细编译命令，例如编译内核，详细编译命令如下所示：

```
./build.sh -h kernel
```

```
root@ubuntu:/home/topeet/Linux/rk356x_linux# ./build.sh -h kernel
###Current SDK Default [ kernel ] Build Command###
cd kernel
make ARCH=arm64 rockchip_linux_defconfig
make ARCH=arm64 rk3568-evb1-ddr4-v10-linux.img -j12
root@ubuntu:/home/topeet/Linux/rk356x_linux#
root@ubuntu:/home/topeet/Linux/rk356x_linux#
```

问题三：编译 Debian 报错

如果编译出现如下错误:

```
root@ubuntu:/home/topeet/Linux/rk356x_linux# cd debian/
root@ubuntu:/home/topeet/Linux/rk356x_linux/debian# RELEASE=buster TARGET=desktop ARCH=arm64 ./mk-base-debian.sh
Starting Download.....
sudo lb clean --purge
[2022-01-21 23:45:08] lb_clean --purge
P: Cleaning chroot
rm -f linaro-buster-alip-*
rm -rf config
I: create configuration
[2022-01-21 23:45:09] lb config --apt-indices false --apt-recommends false --apt-secure false --architectures arm64 --archive-areas main contrib non-free --backports false --binary-file-system ext4 --binary-images tar --bootappend-live hostname=linaro-alip username=linaro --bootstrap-qemu-arch arm64 --bootstrap-qemu-static /usr/bin/qemu-aarch64-static --cache false --chroot-file-system none --compression gzip --debootstrap-options --variant=minbase --include=apt-transport-https,gnupg --distribution buster --gzip-options -9 --rsyncable --iso-publisher Linaro; http://www.linaro.org/; linaro-dev@lists.linaro.org --iso-volume Linaro Buster $(date +%Y%m%d-%H:%M) --linux-flavours none --linux-packages none --mode debian --security true --system normal --updates true
lb config: unrecognized option '--debootstrap-options'
lb config: unrecognized option '--variant=minbase --include=apt-transport-https,gnupg'
lb config: unrecognized option '--updates'
root@ubuntu:/home/topeet/Linux/rk356x_linux/debian#
root@ubuntu:/home/topeet/Linux/rk356x_linux/debian#
```

需要将网盘资料“iTOP-3568 开发板\01_【iTOP-RK3568 开发板】基础资料【V1.7 版本】\02_iTOP-RK3568 开发板烧写工具及驱动”目录下的 live-build_3.0_a69-1_all.deb.zip 下载下来, 然后解压后生成 live-build_3.0_a69-1_all.deb 文件, 然后输入以下命令安装, 然后再重新编译 debian。

```
dpkg -i live-build_3.0_a69-1_all.deb
```

```
root@ubuntu:/home/topeet# dpkg -i live-build_3.0_a69-1_all.deb
(Reading database ... 187440 files and directories currently installed.)
Preparing to unpack live-build_3.0_a69-1_all.deb ...
Unpacking live-build (3.0~a69-1) over (3.0~a57-1ubuntu34.1) ...
Setting up live-build (3.0~a69-1) ...
Processing triggers for man-db (2.8.3-2ubuntu0.1) ...
root@ubuntu:/home/topeet#
```

问题四：编译 Debian 报错

如果编译出现如下错误:

```
topeet@ubuntu:~/Linux/rk356x_linux$ ./build.sh debian
processing option: debian
=====Start building debian for arm64=====
Starting Download.....
sudo lb clean --purge
[sudo] password for topeet:
[2022-06-30 05:48:29] lb_clean --purge
P: Cleaning chroot
rm -f linaro-buster-alip-*
rm -rf config
I: create configuration
[2022-06-30 05:48:30] lb config --apt-indices false --apt-recommends false --apt-secure false --architectures arm64 --archive-areas main contrib non-free --backports false --binary-file-system ext4 --binary-images tar --bootappend-live hostname=linaro-alip username=linaro --bootstrap-qemu-arch arm64 --bootstrap-qemu-static /usr/bin/qemu-aarch64-static --cache false --chroot-file-system none --compression gzip --debootstrap-options --variant=minbase --include=apt-transport-https,gnupg --distribution buster --gzip-options -9 --rsyncable --iso-publisher Linaro; http://www.linaro.org/; linaro-dev@lists.linaro.org --iso-volume Linaro Buster $(date +%Y%m%d-%H:%M) --linux-flavours none --linux-packages none --mode debian --security true --system normal --updates true
P: Considering defaults defined in /etc/live/build.conf
E: Architecture(s) arm64 not yet supported (FIXME)
ERROR: Running build_debian failed!
ERROR: exit code 1 from line 704:
RELEASE=$RK_DEBIAN_VERSION TARGET=desktop ARCH=$ARCH ./mk-base-debian.sh
```

进入 Debian 目录下, 输入以下命令安装依赖包。

```
apt-get install binfmt-support qemu-user-static live-build
```

```
dpkg -i ubuntu-build-service/packages/*
```

```
apt-get install -f
```

问题五：编译瑞芯微原厂 yocto 源码, 编译报错

如果编译过程中遇到如下图所示的错误:

第 7 章 Android11/12/Linux 设备树简介

Android11 源码默认设备树文件：

```
rk_android11.0_sdk/kernel/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dts
```

Android12 源码默认设备树文件：

```
rk_android12.0_sdk/kernel-4.19/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dts
```

Linux 源码默认设备树文件：

```
rk356x_linux/kernel/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10-linux.dts
```

设备树是一种描述硬件的数据结构，很形象的来说，就像是树一样，把硬件描述了出来，如果我们需要修改节点，通过头文件一层一层地找节点即可。比如 Linux 源码默认设备树文件 rk3568-evb1-ddr4-v10-linux.dts 中的头文件包括以下两个设备树，去到以下两个设备树中会看到它们头文件所包含的其他设备树文件，可以这样一层层的找需要修改的节点。

```
// SPDX-License-Identifier: (GPL-2.0+ OR MIT)
/*
 * Copyright (c) 2020 Rockchip Electronics Co., Ltd.
 *
 */

#include "rk3568-evb1-ddr4-v10.dtsi"
#include "rk3568-linux.dtsi"

#include <dt-bindings/display/rockchip_vop.h>

&vp0 {
    cursor-win-id = <ROCKCHIP_VOP2_CLUSTER0>;
};

&vp1 {
    cursor-win-id = <ROCKCHIP_VOP2_CLUSTER1>;
};
```


第 8 章 如何修改默认配置并保存

执行 SDK 包中 `build.sh` 编译脚本编译内核和 `uboot` 时会先 `make` 内核和 `uboot` 的默认的配置文件生产 `.config`，然后使用 `.config` 文件里面的配置来编译内核和 `uboot` 镜像。所以要想修改配置需修改默认的配置文件才行。

在使用 `make menuconfig` 进行配置之前，先使用命令 “`make XXX_defconfig`” 生成 `.config`，修改完图形化配置界面的设置并保存，那么 `.config` 文件会被更新，需将修改后的 `.config` 覆盖回默认配置文件。具体可根据以下章节进行操作。

8.1 Android11/12/Linux 默认配置文件

Android11 的默认配置文件

内核: `kernel/arch/arm64/configs/rockchip_defconfig`

Uboot: `u-boot/configs/rk3568_defconfig`

Android12 的默认配置文件

内核: `kernel-4.19/arch/arm64/configs/rockchip_defconfig`

Uboot: `u-boot/configs/rk3568_defconfig`

Linux 的默认配置文件

内核: `kernel/arch/arm64/configs/rockchip_linux_defconfig`

Uboot: `u-boot/configs/rk3568_defconfig`

8.2 Android11/12/Linux 编译驱动到 uboot

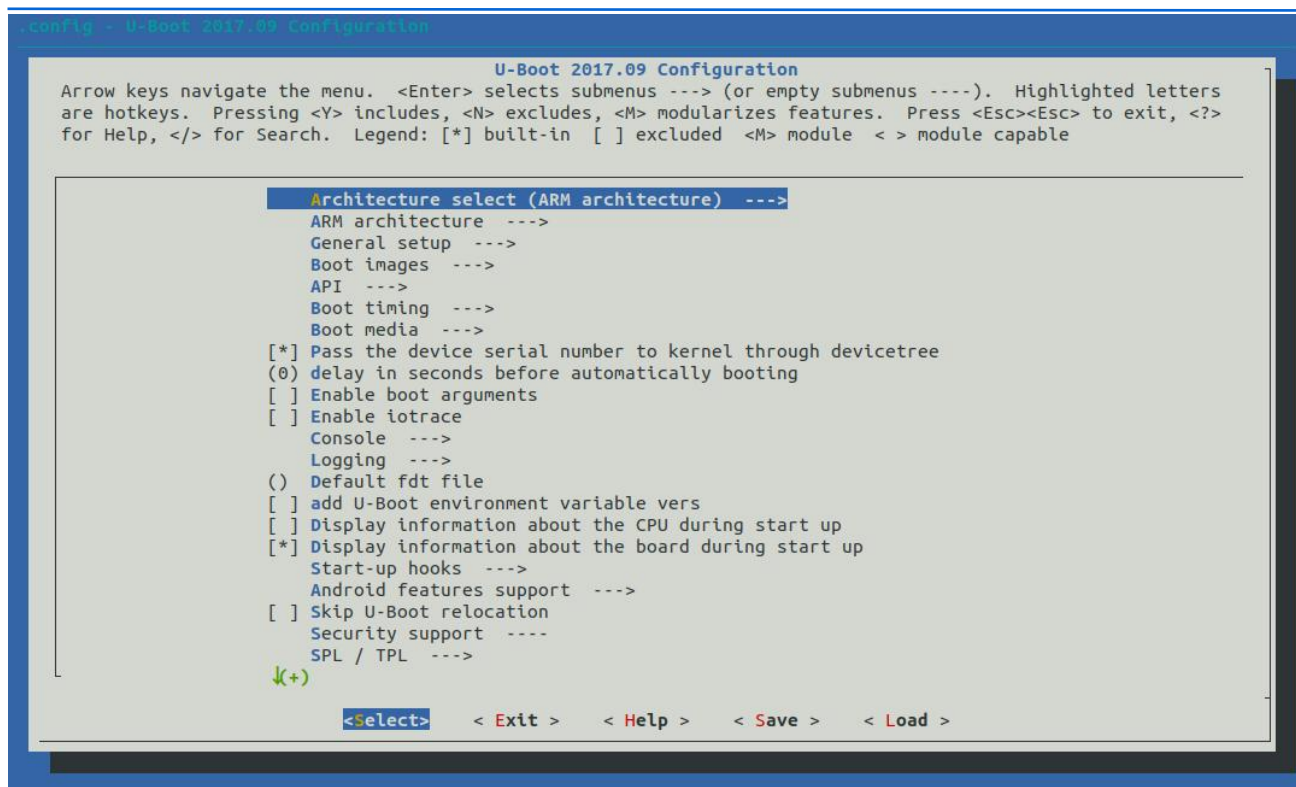
如果想要配置自己的 `uboot`，可以运行以下命令进行操作，在 SDK 源码目录下：

```
cd u-boot
```

```
make rk3568_defconfig
```

```
make menuconfig
```

运行结果如下图所示：



Exit 退出图形配置界面后运行以下命令将修改保存到默认配置文件。

```
cp .config configs/rk3568_defconfig //保存修改配置到默认配置文件
```

8.3 Android11/12/Linux 编译驱动到内核

在平时的驱动开发中，经常需要在内核中配置某种功能，为了方便大家开发和学习，本小节讲解如何在内核中添加驱动。具体的讲解原理讲解请参考本手册的驱动教程。

Android11 源码如果想要修改内核，可以运行以下命令进行修改：

```
cd kernel/
make ARCH=arm64 rockchip_defconfig
make ARCH=arm64 menuconfig
```

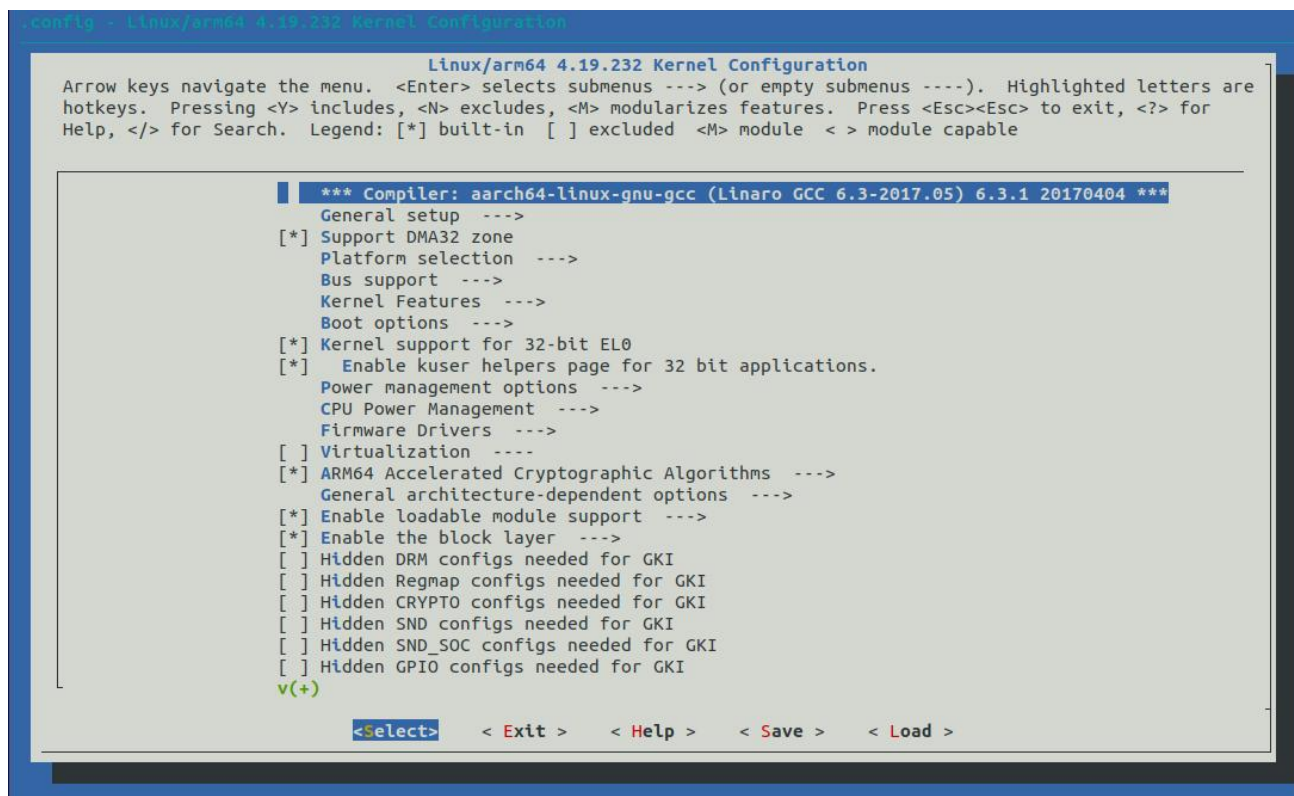
Android12 源码如果想要修改内核，可以运行以下命令进行修改：

```
cd kernel-4.19/
make ARCH=arm64 rockchip_defconfig
make ARCH=arm64 menuconfig
```

Linux 源码如果想要修改内核，可以运行以下命令进行修改，如下图所示：

```
cd kernel/
make ARCH=arm64 rockchip_linux_defconfig
make ARCH=arm64 menuconfig
```

运行结果如下图所示：



Exit 退出图形配置界面后运行以下命令将修改保存到默认配置文件。

```
cp .config arch/arm64/configs/rockchip_defconfig //Android11、Android12
```

```
cp .config arch/arm64/configs/rockchip_linux_defconfig //Linux
```

第 9 章 编译 Linux_NVR_SDK 源码

9.1 编译环境说明

本手册使用的是迅为提供的编译环境 ubuntu18.04，在网盘资料“**iTOP-3568 开发板\02_【iTOP-RK3568 开发板】开发资料\02_iTOP-3568 开发板搭建好编译环境的 Ubuntu18.04 虚拟机镜像**”路径下。这里的 ubuntu 默认开发环境已经搭建好。

客户也可以自行搭建编译环境，建议使用 Ubuntu18.04 系统或者更高版本，以下介绍如何自行搭建编译环境。

硬件要求：

Ubuntu 为 64 位系统，硬盘空间要大于 40G。

软件要求：

Ubuntu18.04 系统，需要在 Ubuntu18.04 安装依赖软件包，输入以下命令安装：

```
sudo apt-get install repo git ssh make gcc libssl-dev liblz4-tool expect g++ patchelf chrpath  
gawk texinfo chrpath diffstat binfmt-support qemu-user-static live-build bison flex fakeroot cmake  
gcc-multilib g++-multilib unzip device-tree-compiler python-pip ncurses-dev pyelftools
```

9.2 获取 Linux_NVR_SDK 源码

NVR_Linux 源码是基于完整的 Linux 源码裁剪而来，针对类 NVR 产品优化了多视频播放能力等。迅为提供的 Linux_NVR_SDK 源码网盘下载路径为：**iTOP-3568 开发板\01_【iTOP-RK3568 开发板】基础资料【V1.7 版本】\07_iTOP-RK3568 开发板 Linux_NVR_SDK 源码**

1 首先下载网盘资料的源码 rk3568_nvr_linux_20220528.tar.gz，使用 SSH 软件将其拷贝到 Ubuntu 系统。

源码名称说明：源码会逐渐更新，发布的源码日期会有所不同，具体以网盘中实际名称为准。

2 输入以下命令解压压缩包。

```
tar -vxf rk3568_nvr_linux_20220528.tar.gz
```

```
root@ubuntu:/home/topeet/Linux# ls  
rk3568_nvr_linux rk3568_nvr_linux_20220528.tar.gz rk3568_linux  
root@ubuntu:/home/topeet/Linux#  
root@ubuntu:/home/topeet/Linux#
```

3 解压完毕生成 rk3568_nvr_linux 文件夹，如下图所示：


```
root@ubuntu:/home/topeet/Linux# ls
rk3568_nvr_linux rk3568_nvr_linux_20220528.tar.gz rk356x_linux
root@ubuntu:/home/topeet/Linux#
```

4 输入以下命令进入 rk3568_nvr_linux 文件夹，如下图所示：

```
cd rk3568_nvr_linux
```

```
ls
```

```
root@ubuntu:/home/topeet/Linux# ls
rk3568_nvr_linux rk3568_nvr_linux_20220528.tar.gz rk356x_linux
root@ubuntu:/home/topeet/Linux#
root@ubuntu:/home/topeet/Linux# cd rk3568_nvr_linux
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# ls
build          repo
build_emmc.sh  rkbin
build_spi_nand.sh Rockchip_RK356X_Quick_Start_Linux_NVR_V1.2.0_20211029_CN.pdf
docs           rockdev
IMAGE         tools
kernel        u-boot
mkfirmware.sh
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux#
```

9.3 Linux_NVR_SDK 简介

Linux_NVR_SDK 旨在适配 NVR 以及显控，会议系统等产品开发，基于 Linux 的一套精简 SDK。区别于完整的桌面 Linux 系统，安防后端 SDK 更偏向于嵌入式需求。Linux_NVR_SDK 简介易用，优化存储，内存资源占用，并对多路视屏并发解码拼接显示等功能做专项优化。

Linux_NVR_SDK 源码目录如下所示：

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# tree -L 1
.
├── build
├── build_emmc.sh -> build/tools/build_emmc.sh
├── build_spi_nand.sh -> build/tools/build_spi_nand.sh
├── docs
├── IMAGE
├── kernel
├── mkfirmware.sh -> build/tools/mkfirmware.sh
├── repo
├── rkbin
├── Rockchip_RK356X_Quick_Start_Linux_NVR_V1.2.0_20211029_CN.pdf -> docs/.Socs/RK356X/Rockchip_RK356X_Quick_Start_Linux_NVR_V1.2.0_20211029_CN.pdf
├── rockdev
├── tools
└── u-boot

9 directories, 4 files
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux#
```

Linux_NVR_SDK 源码目录结构解释如下：

build：存放编译脚本，根文件系统以及工具链等。

build_emmc.sh：编译 EMMC 脚本

build_spi_nand.sh：编译 NAND 脚本

docs：存放开发指导文件，平台支持列表，工具使用文档，Linux 开发指南等。

IMAGE：存放每次完整编译产生的镜像，补丁，编译信息等。

kernel: 存放 kernel 代码
mkfirmware.sh
repo: 存放版本 repo 信息
rkbin: 存放 Rockchip 相关 Binary 和工具
Rockchip_RK356X_Quick_Start_Linux_NVR_V1.2.0_20211029_CN.pdf
rockdev: 存放编译输出的分立固件
tools: 存放 SDK 配套的工具
u-boot: 存放 uboot 代码

区别于桌面版本的 Linux 源码，Linux_NVR_SDK 只保留了 kernel ,u-boot,docs,rkbin,tools 目录，新增 build 目录。

build 目录结构如下所示：

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/build# tree -L 1
.
├── app
├── parameter-nvr-emmc.txt
├── parameter-nvr-spinand.txt
├── patch
├── rootfs
├── rootfs.ext4
├── rootfs.img -> rootfs.ext4
├── rootfs.tar.gz
├── toolchain
├── toolchain.tar.gz
├── tools
├── userdata
├── userdata.ext4
└── userdata.img -> userdata.ext4
```

build 目录结构解释如下：

app:用于应用程序编译，默认带 RKMPI 发布版本以及 CMAKE 编译脚本
parameter-nvr-emmc.txt: EMMC 设备参考分区表
parameter-nvr-spinand.txt: SPI NAND 设备参考分区表
patch: 源码默认自带的 Patch,目前主要用于设置 uboot 的 toolchain 路径
rootfs: 文件系统
rootfs.ext4: 文件系统 ext4 格式
rootfs.img: 文件系统 img 格式
rootfs.tar.gz: 提供的参考的 rootfs,包括基础的 C 库，以及 gdb,ubifs,nfs 调试库等。
toolchain: 工具链，用于编译源码以及应用
toolchain.tar.gz: 工具链压缩包
tools: 用于打包 rootfs 的脚本，编译脚本等。
userdata: 用于打包 data.img，可用于调试

userdata.ext4: userdata ext4 格式

userdata.img: userdata ext4 img 格式

9.4 Linux_NVR_SDK 编译源码

Linux_NVR_SDK 源码根据存储介质的不同, 区分为 EMMC 和 SPI NAND 两种编译配置。
iTOP-RK3568 开发板核心板上存储介质为 EMMC, 所以这里以 EMMC 编译为例。

输入以下命令查看编译命令:

```
./build_emmc.sh -h
```

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# ./build_emmc.sh -h
/usr/bin/fakeroot
=====Start check sdk env =====
patching file Makefile
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file Makefile.rej
patching file make.sh
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file make.sh.rej
Running check_env succeeded.
processing option: -h
Usage: build.sh [OPTIONS]
uboot          -build uboot
kernel         -build kernel
userdata       -build userdata
rootfs         -build default rootfs, currently build buildroot as default
all            -build uboot, kernel, rootfs, userdata image
cleanall       -clean uboot, kernel, rootfs, userdata
update         -pack update image
env            -check sdk env
cleanenv       -clean sdk env

Default option is 'all'.
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux#
```

在源码的根目录下, 输入以下命令完整编译源码, 如下所示:

```
./build_emmc.sh
```

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# ./build_emmc.sh
/usr/bin/fakeroot
=====Start check sdk env =====
patching file Makefile
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file Makefile.rej
patching file make.sh
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file make.sh.rej
Running check_env succeeded.
processing option: all
=====
TARGET_ARCH=arm64
TARGET_PLATFORM=rk3568
TARGET_UBOOT_CONFIG=rk3568
TARGET_KERNEL_CONFIG=rockchip_linux_defconfig
TARGET_KERNEL_DTS=rk3568-nvr-demo-v12-linux
=====
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
```


编译生成的镜像统一放在 rockdev 目录下，如下图所示：

```
*****RKImageMaker ver 1.66*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
Running build_save succeeded.
Running build_all succeeded.
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# cd rockdev/
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/rockdev# ls
boot.img          parameter.txt      uboot.img          userdata.img
MiniLoaderAll.bin rootfs.img         update.img
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/rockdev#
```

Linux_NVR_SDK 源码也可以单独编译，以下介绍如何单独编译各种模块，及打包。

输入以下命令，单独编译 U-boot，如下图所示：

```
./build_emmc.sh uboot
```

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# ./build_emmc.sh uboot
/usr/bin/fakeroot
=====Start check sdk env =====
patching file Makefile
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file Makefile.rej
patching file make.sh
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file make.sh.rej
Running check_env succeeded.
processing option: uboot
=====Start building uboot=====
TARGET_UBOOT_CONFIG=rk3568
=====
## make rk3568_defconfig -j8
```

输入以下命令，单独编译内核，如下图所示：

```
./build_emmc.sh kernel
```

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# ./build_emmc.sh kernel
/usr/bin/fakeroot
=====Start check sdk env =====
patching file Makefile
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file Makefile.rej
patching file make.sh
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file make.sh.rej
Running check_env succeeded.
processing option: kernel
=====Start building kernel=====
TARGET_ARCH          =arm64
TARGET_KERNEL_CONFIG =rockchip_linux_defconfig
```

输入以下命令，执行命令之后，会将 build/rootfs 目录打包成特定格式的 img 镜像。客户可以在 build/rootfs/ 目录下自行增删根文件系统内容。

```
./build_emmc.sh rootfs
```



```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# ./build_emmc.sh rootfs
/usr/bin/fakeroot
=====Start check sdk env =====
patching file Makefile
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file Makefile.rej
patching file make.sh
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file make.sh.rej
Running check_env succeeded.
processing option: rootfs
Making rootfs/ (auto size:121M)
Creating filesystem with parameters:
    Size: 126877696
    Block size: 4096
    Blocks per group: 32768
```

在编译完 U-boot, kernel, rootfs 各部分编译完成之后, 输入以下命令打包镜像:

```
./build_emmc.sh update
```

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# ./build_emmc.sh update
/usr/bin/fakeroot
=====Start check sdk env =====
patching file Makefile
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file Makefile.rej
patching file make.sh
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file make.sh.rej
Running check_env succeeded.
processing option: update
create rootfs.img...done.
create parameter...done.
/home/topeet/Linux/rk3568_nvr_linux/build/parameter-nvr-emmc.txt
0x00000800@0x00002000(vnm),0x00002000@0x00002800(uboot),0x00006000@0x00004800(b
oot),0x00006400@0x0000a800(rootfs),-@0x00006e800(userdata:grow)
create uboot.img...done.
uboot format type is fit, so ignore trust.img...
```

编译生成的镜像统一放在 rockdev 目录下, 如下图所示:

```
*****RKImageMaker ver 1.66*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!
Making ./Image/update.img OK.
Running build_updateimg succeeded.
Running build_save succeeded.
Running build_all succeeded.
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# cd rockdev/
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/rockdev# ls
boot.img          parameter.txt     uboot.img        userdata.img
MiniLoaderAll.bin rootfs.img        update.img
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/rockdev#
```

9.5 Linux_NVR_SDK 编译应用

在源码 build/app 目录下有发布版本的 RKMPI 以及编译配置, 目前的编译方式只支持 Cmake 脚本。如果要编译其他应用, 可以参考 build/app/build/build.sh 脚本配置编译工具链。

以编译 RKMPI 应用程序为例，进行示范：

1. 使能 sdk 环境(同一个终端只需执行一次)

```
./build_emmc.sh env
```

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# ./build_emmc.sh env
/usr/bin/fakeroot
processing option: env
=====Start check sdk env =====
patching file Makefile
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file Makefile.rej
patching file make.sh
Reversed (or previously applied) patch detected! Skipping patch.
1 out of 1 hunk ignored -- saving rejects to file make.sh.rej
Running check_env succeeded.
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux#
```

2. 编译 RKMPI，输入以下命令

```
cd build/app/build
```

```
ls
```

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux# cd build/app/build
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/build/app/build# ls
aarch64.linux.cross.cmake build.sh
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/build/app/build#
```

- 3 接着输入以下命令编译：

```
./build.sh ../RKMPI_Release/SDK/rockit/
```

```
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/build/app/build# ./build.sh ../RKMPI_Release/SDK/rockit/
./build.sh: line 22: -I/home/topeet/Linux/rk3568_nvr_linux/build/app/build/../RKMPI_Release/SDK/rockit//sdk/deps/include: No such file or directory
/home/topeet/Linux/rk3568_nvr_linux/build/app/build/../RKMPI_Release/SDK/rockit/
-- CMAKE_C_COMPILER="aarch64-linux-gcc" is specified.
-- CMAKE_C_COMPILER="aarch64-linux-gcc" is specified.
-- The C compiler identification is GNU 9.3.0
-- The CXX compiler identification is GNU 9.3.0
-- Check for working C compiler: /home/topeet/Linux/rk3568_nvr_linux/build/toolchain/bin/aarch64-linux-gcc
-- Check for working C compiler: /home/topeet/Linux/rk3568_nvr_linux/build/toolchain/bin/aarch64-linux-gcc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /home/topeet/Linux/rk3568_nvr_linux/build/toolchain/bin/aarch64-linux-g++
-- Check for working CXX compiler: /home/topeet/Linux/rk3568_nvr_linux/build/toolchain/bin/aarch64-linux-g++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
```

- 4 编译生成的可执行文件会在 build/app/RKMPI_Release/SDK/bin 目录下，如下图所示：

```
cd ../RKMPI_Release/SDK/bin/
```

```
ls
```



```
[ 92%] Built target rk_mpi_vplay_test
Scanning dependencies of target rk_mpi_ainplay_test
[ 94%] Building CXX object example/samples/CMakeFiles/rk_mpi_ainplay_test.dir/rk_m
pi_ainplay_test.cpp.o
[ 96%] Linking CXX executable /home/topeet/Linux/rk3568_nvr_linux/build/app/RKMPI
_Release/SDK/bin/rk_mpi_ainplay_test
[ 96%] Built target rk_mpi_ainplay_test
Scanning dependencies of target rk_mpi_acapture_test
[ 98%] Building CXX object example/samples/CMakeFiles/rk_mpi_acapture_test.dir/r
k_mpi_acapture_test.cpp.o
[100%] Linking CXX executable /home/topeet/Linux/rk3568_nvr_linux/build/app/RKMPI
_Release/SDK/bin/rk_mpi_acapture_test
[100%] Built target rk_mpi_acapture_test
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/build/app/build#
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/build/app/build# cd ../RKMPI_Rel
ease/SDK/bin/
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/build/app/RKMPI_Release/SDK/bin#
ls
rk_mpi_acapture_test  rk_mpi_ainplay_test  rk_mpi_sys_test      rk_mpi_vi_test
rk_mpi_adec_test      rk_mpi_avio_test      rk_mpi_tde_test      rk_mpi_vo_test
rk_mpi_aenc_test      rk_mpi_mb_test        rk_mpi_vdec_test      rk_mpi_vplay_test
rk_mpi_ai_test        rk_mpi_mmz_test      rk_mpi_venc_test      rk_mpi_vpss_test
rk_mpi_ao_test        rk_mpi_rgn_test      rk_mpi_vgs_test
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/build/app/RKMPI_Release/SDK/bin#
root@ubuntu:/home/topeet/Linux/rk3568_nvr_linux/build/app/RKMPI_Release/SDK/bin#
```

5 首先将编译生成的可执行文件拷贝到 Linux_NVR_SDK 源码的 buildrootfs/usr/bin 目录下，然后重新编译 Linux_NVR_SDK 源码,编译完成之后将新生成的 Linux_NVR_SDK 镜像进行烧写，在开发板的/usr/bin 目录下可以看到之前编译生成的可执行文件。

第 10 章 定制 Ubuntu 和 Debian 系统

在前面的章节中讲解了 debian 和 ubuntu 的编译，但有些时候需要进行一些系统定制，例如添加一些当前系统中没有的文件，或者安装一些当前系统没有的软件，在本小节将对上述两种定制内容进行讲解(本小节以 ubuntu20 为例进行演示)。

首先来演示如何向文件系统中添加文件，在 Linux SDK 源码的 ubuntu 和 debian 目录下各存在一个 overlay 目录，如下图所示：

```
topeet@topeet:~/rk356x_linux$ ls ubuntu/  
mk-image.sh mk-rootfs.sh mount.sh overlay post-build.sh ubuntu-focal-arm64.tar.xz ubuntu-jammy-arm64.tar.xz  
topeet@topeet:~/rk356x_linux$ ls debian/  
linaro-bullseye-arm64.tar.xz linaro-buster-arm64.tar.xz mk-image.sh mk-rootfs.sh mount.sh overlay post-build.sh  
topeet@topeet:~/rk356x_linux$
```

如果想要向 debian 系统中添加文件就将文件添加到 debian/overlay 目录下，如果想要向 ubuntu 系统中添加文件就将文件添加到 ubuntu/overlay 目录下，这里以 ubuntu 为例进行演示，首先来到 ubuntu/overlay 目录下，如下图所示：

```
topeet@topeet:~/rk356x_linux$ cd ubuntu/overlay/  
topeet@topeet:~/rk356x_linux/ubuntu/overlay$ ls  
topeet@topeet:~/rk356x_linux/ubuntu/overlay$
```

这里添加一个名为 test 的文件，添加拷贝完成如下图所示：

```
topeet@topeet:~/rk356x_linux/ubuntu/overlay$ mkdir test  
topeet@topeet:~/rk356x_linux/ubuntu/overlay$ ls  
test  
topeet@topeet:~/rk356x_linux/ubuntu/overlay$
```

然后根据 4.5、4.6、5.5 或者 5.6 小节重新编译 ubuntu 和 debian，将生成的系统烧写到开发板上，在开发板的根目录就有了名为 test 的文件夹，如下图所示：

```
topeet@topeet:/$  
bin boot data dev etc home lib lost+found media mnt opt proc root run sbin sdcard snap srv sys test tmp udisk usr var  
root@topeet:/$
```

至此，关于文件的添加就讲解完成了，然后来演示软件的安装，首先回到 Linux SDK 目录下，在 ubuntu 和 debian 文件夹下，都有一个名为 mk-rootfs.sh 的脚本文件，如下图所示：

```
topeet@topeet:~/rk356x_linux$ ls ubuntu/  
mk-image.sh mk-rootfs.sh mount.sh overlay post-build.sh ubuntu-focal-arm64.tar.xz ubuntu-jammy-arm64.tar.xz  
topeet@topeet:~/rk356x_linux$ ls debian/  
linaro-bullseye-arm64.tar.xz linaro-buster-arm64.tar.xz mk-image.sh mk-rootfs.sh mount.sh overlay post-build.sh  
topeet@topeet:~/rk356x_linux$
```

通过对该脚本的修改即可实现软件的默认安装，这里以 ubuntu 系统为例进行演示，打开 ubuntu/mk-rootfs.sh 脚本文件，找到 “`sudo chroot "${chroot_dir}"`” 内容，如下图所示：

```
30 sudo mount -t proc /proc ${chroot_dir}/proc  
31 sudo mount -t sysfs /sys ${chroot_dir}/sys  
32 sudo mount -o bind /dev ${chroot_dir}/dev  
33 sudo mount -o bind /dev/pts ${chroot_dir}/dev/pts  
34  
35 sudo chroot "${chroot_dir}" /bin/bash -c "apt update"  
36  
37 sudo umount -lf ${chroot_dir}/dev/pts 2> /dev/null || true  
38 sudo umount -lf ${chroot_dir}/* 2> /dev/null || true
```

该内容最后的 “apt update” 会在构建文件系统时进行调用，同理如果想要安装软件或者执行其他命令，只需要按照格式进行添加即可，例如安装 vim 软件就需要添加以下内容，添加完成如下所示：

apt install vim

```
30 sudo mount -t proc /proc ${chroot_dir}/proc
31 sudo mount -t sysfs /sys ${chroot_dir}/sys
32 sudo mount -o bind /dev ${chroot_dir}/dev
33 sudo mount -o bind /dev/pts ${chroot_dir}/dev/pts
34
35 sudo chroot "${chroot_dir}" /bin/bash -c "apt update"
36 sudo chroot "${chroot_dir}" /bin/bash -c "apt install vim"
37
38 sudo umount -lf ${chroot_dir}/dev/pts 2> /dev/null || true
39 sudo umount -lf ${chroot_dir}/* 2> /dev/null || true
```

然后根据 4.5、4.6、5.5 或者 5.6 小节重新编译 ubuntu 和 debian，至此，关于软件的默认安装就讲解完成了，大家可以根据上述内容，对 ubuntu 和 debian 文件系统实现定制。

附录一 视频链接

为了方便大家更快的使用学习 RK3568，北京迅为电子录制了 RK3568 开发板学习系统编译与烧写系列教程，如下图所示：



【北京迅为】RK3568开发板系统编译与烧写（必会）

RK3568 开发板学习系统编译与烧写系列教程如下所示：

1_本期视频介绍以及观看建议

视频链接：<https://www.bilibili.com/video/BV1pv4y1K71W?p=1>

2_RK3568 开发板启动模式以及引导顺序（必看）

视频链接：<https://www.bilibili.com/video/BV1pv4y1K71W?p=2>

3_Android11 源码包编译-获取 Android11 源码

视频链接：<https://www.bilibili.com/video/BV1pv4y1K71W?p=3>

4_Android11 源码包编译-整体编译（必看）

视频链接：<https://www.bilibili.com/video/BV1pv4y1K71W?p=4>

5_Android11 源码包编译-单独编译 uboot

视频链接：<https://www.bilibili.com/video/BV1pv4y1K71W?p=5>

6_Android11 源码包编译-单独编译内核和设备树

视频链接：<https://www.bilibili.com/video/BV1pv4y1K71W?p=6>

7_Android11 源码包编译-单独编译 Android11 文件系统

视频链接：<https://www.bilibili.com/video/BV1pv4y1K71W?p=7>

8_Android11 源码包编译-打包 update.img 镜像

视频链接：<https://www.bilibili.com/video/BV1pv4y1K71W?p=8>

9_系统镜像烧写-安装 RKTool 驱动（必看）

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=9>

10_系统镜像烧写-设置拨码开关（必看）

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=10>

11_Android11 镜像烧写-整体烧写 Android11 系统镜像（必看）

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=11>

12_Anroid 系统固件作用讲解（必看）

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=12>

13_完整固件 update.img 拆包和解包方法（必看）

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=13>

14_Android11 镜像烧写-使用独立镜像文件进行烧写

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=14>

15_Android11 镜像烧写-单独烧写 uboot、内核、设备树等镜像

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=15>

16_Android12 源码包编译-获取 Android12 源码

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=16>

17_Android12 源码包编译-整体编译（必看）

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=17>

18_Android12 源码包编译-单独编译 uboot

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=18>

19_Android12 源码包编译-单独编译内核和设备树

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=19>

20_Android12 源码包编译-单独编译 Andorid12 文件系统

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=20>

21_Android12 源码包编译-打包 update.img 镜像

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=21>

22_Andorid12 镜像烧写-整体烧写 Android12 系统镜像（必看）

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=22>

23_Android12 镜像烧写-使用独立镜像文件进行烧写

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=23>

24_Android12 镜像烧写-单独烧写 uboot、内核、设备树等镜像

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=24>

25_Linux 源码包介绍

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=25>

26_编译 Linux 源码说明

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=26>

27_Linux 源码包编译-获取 Linux 源码

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=27>

28_Linux 系统编译-整体编译 buildroot, debian, yocto 系统 (必看)

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=28>

29_Linux 系统编译-单独编译 uboot

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=29>

30_Linux 系统编译-单独编译内核和设备树

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=30>

31_Linux 系统编译-单独编译 recovery

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=31>

32_Linux 系统编译-单独编译 Linux 文件系统

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=32>

33_Linux 系统编译-打包 builddroot, yocto, debian 文件系统

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=33>

34_linux 系统编译-编译 ubuntu 文件系统

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=34>

35_Linux 系统编译-打包 ubuntu 系统镜像

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=35>

36_Linux 系统镜像烧写-烧写完整固件 update.img

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=36>

37_Linux 系统镜像烧写-单独烧写 uboot 内核设备树等镜像

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=37>

38_TF 卡烧写 (适用 Linux 和 Android 系统)

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=38>

39_使用虚拟机 ubuntu 系统给开发板烧写系统镜像

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=39>

40_救砖模式

视频链接: <https://www.bilibili.com/video/BV1pv4y1K71W?p=40>

